

分数:



2025—2026 学年春季学期

模式识别课程实验报告

结课作业

题目	模式识别课程实验报告
评语	

班级 09012302

学号 2023302359

姓名 曹军

成绩

摘 要

模式识别方法的性能不仅取决于分类器形式,还受到特征表示、概率假设、样本规模 and 评价准则的共同影响。围绕课程中的参数估计、非参数估计、线性判别、K-L 变换和聚类分析等核心内容,本文构建了由低维统计数据、标准机器学习数据集和视觉应用任务组成的实验体系。首先,以男女身高体重数据为主要对象,建立最小错误率 Bayes 分类器和最小风险 Bayes 分类器,并进一步引入 Parzen 窗、 k 近邻和 Fisher 线性判别方法,对不同特征组合、先验概率、协方差假设和损失函数设置下的分类性能进行比较。其次,利用 K-L 变换分析无监督主成分方向与监督判别方向的差异,并在人脸图像上实现特征脸提取与识别。随后,采用 C 均值和 Ward 分级聚类方法研究样本空间中的无监督分组结构。为提高实验的完整性,本文还在模拟高斯数据、UCI Iris、MNIST、CIFAR 等扩展数据集上测试相关方法,并补充开集目标检测和视觉分类器奖励引导的无人机数字轨迹两个拓展实验。

实验结果表明,在低维、近似高斯且类别分布较清晰的数据上,参数 Bayes 分类器和线性判别方法能够获得较稳定的分类效果;当数据分布复杂或图像维度较高时,特征表达质量对识别性能的影响更加显著。非参数方法能够在不强加分布假设的情况下刻画局部结构,但其效果依赖窗宽、邻居数和样本密度。K-L 变换能够有效压缩高维图像信息,但其最大方差方向并不一定等价于最优分类方向。聚类实验进一步说明,无监督分组结果反映的是特征空间结构,而不必然等同于真实语义类别。两个拓展实验表明,模式识别方法还可以扩展到开放类别拒识和智能体控制评价中,体现出从传统统计分类到现代视觉决策系统的应用联系。

关键词: 模式识别; Bayes 分类器; 非参数估计; Fisher 线性判别; K-L 变换; 开集检测; 视觉分类器

目 录

摘要	II
图目录	VII
表目录	X
1 实验环境与数据集	1
1.1 实验环境	1
1.2 数据集说明	1
2 实验一：Bayes 分类器实验	2
2.1 模式识别理论知识	2
2.1.1 分类问题与 Bayes 公式	2
2.1.2 高斯分布假设与最大似然参数估计	2
2.1.3 最小风险 Bayes 分类器	3
2.1.4 非参数估计方法	4
2.2 实际数据集与参数计算	5
2.3 程序框图与代码实现	6
2.4 实验步骤与结果分析	7
2.4.1 步骤一：单个特征下的最小错误率 Bayes 分类	7
2.4.2 步骤二：两个特征下的 Bayes 分类	8
2.4.3 步骤三：不同先验概率对决策规则的影响	8
2.4.4 步骤四：最小风险 Bayes 决策	9
2.4.5 步骤五：非参数估计实验	10
2.5 综合实验结果与分析总结	11
2.6 扩展数据集实验	12
2.6.1 模拟高斯数据实验	13
2.6.2 UCI Iris 数据集实验	14
2.6.3 MNIST 手写数字数据集实验	15
2.6.4 CIFAR-10 与 CIFAR-100 图像数据集实验	17
2.6.5 扩展数据集结果汇总	19
2.7 体会	20
3 实验二：非参数估计、Fisher 线性判别与留一法实验	21
3.1 模式识别理论知识	21
3.1.1 非参数估计与 Parzen 窗 Bayes 分类器	21
3.1.2 Fisher 线性判别	21
3.1.3 多类情形下的 LDA 扩展	22
3.1.4 留一法错误率估计	22

3.2	实际数据集与实验设置	23
3.3	程序框图与代码实现	24
3.4	实验步骤与结果分析	25
3.4.1	步骤一: Parzen 窗 Bayes 与参数 Bayes 比较	25
3.4.2	步骤二: Fisher 线性判别与 Bayes 分类器比较	25
3.4.3	步骤三: 留一法错误率估计	27
3.5	扩展数据集实验	27
3.5.1	模拟高斯数据	27
3.5.2	UCI Iris 数据	28
3.5.3	MNIST 与 CIFAR 图像数据	29
3.6	综合实验结果与分析总结	32
3.7	体会	33
4	实验三: K-L 变换特征提取实验	34
4.1	模式识别理论知识	34
4.2	实际数据集与参数计算	35
4.3	程序框图与代码实现	36
4.4	实验步骤与结果分析	37
4.4.1	步骤一: 不考虑类别信息的 K-L/PCA 变换	37
4.4.2	步骤二: 利用类平均向量提取判别信息	38
4.4.3	步骤三: 与 Fisher、Bayes 分类方法比较	39
4.5	扩展数据集实验	39
4.5.1	模拟高斯数据	40
4.5.2	UCI Iris 数据	40
4.5.3	MNIST 与 CIFAR 图像数据	41
4.6	综合实验结果与分析总结	44
4.7	体会	45
5	实验四: 基于 K-L 变换的应用实验	46
5.1	模式识别理论知识	46
5.1.1	人脸图像的向量表示	46
5.1.2	K-L 变换与特征脸	46
5.1.3	低维特征空间中的分类	47
5.2	人脸数据集说明	47
5.3	实验设置与程序实现	48
5.4	程序框图	49
5.5	实验步骤与结果分析	49
5.5.1	平均脸与特征脸提取结果	49
5.5.2	特征脸数量与识别性能	50

5.5.3	特征脸空间分布与混淆情况	52
5.5.4	预测样例分析	53
5.6	补充实验：飞机图像数据集上的 K-L 变换	53
5.7	实验结果综合分析	55
5.8	体会	56
6	实验五：聚类分析实验	57
6.1	模式识别理论知识	57
6.1.1	C 均值聚类	57
6.1.2	聚类指标	57
6.1.3	分级聚类	58
6.2	实际数据集与实验设置	58
6.3	程序框图与代码实现	59
6.4	实验步骤与结果分析	59
6.4.1	步骤一：训练集上 C=2 的 C 均值聚类与初始值影响	59
6.4.2	步骤二：不同类别数下的聚类指标	61
6.4.3	步骤三：Ward 分级聚类	62
6.4.4	步骤四：加入 test2 后重复实验	64
6.5	综合实验结果与分析总结	66
6.6	体会	66
7	补充实验一：基于 DT-ProtoYOLO 的开集目标检测	67
7.1	模式识别理论知识	67
7.1.1	闭集识别与开集识别	67
7.1.2	原型表示	67
7.1.3	双线性阈值判别	68
7.2	数据集与实验设置	68
7.3	方法设计与程序框图	69
7.4	实验结果与分析	70
8	补充实验二：视觉分类器奖励引导的无人机数字轨迹	74
8.1	模式识别理论知识	74
8.1.1	视觉分类器作为模式评价函数	74
8.1.2	CNN 数字识别的理论基础	74
8.1.3	无监督技能发现与下游迁移	75
8.2	数据集与实验设置	76
8.3	方法设计与程序框图	77
8.4	实验结果与分析	77
9	总体总结	79

A 核心程序代码	80
A.1 实验一核心代码	80
A.2 实验二核心代码	83
A.3 实验三核心代码	86
A.4 实验四核心代码	88
A.5 实验五核心代码	91
A.6 补充实验一核心代码	93
A.7 补充实验二核心代码	94

图 目 录

1	男女训练样本身高-体重散点分布	5
2	实验一程序流程图	6
3	单特征高斯密度估计与 Bayes 判决阈值	7
4	二维高斯 Bayes 分类器决策边界	8
5	不同先验概率下的 Bayes 决策边界比较	9
6	Parzen 窗和 k 近邻分类边界	10
7	二维高斯 Bayes 分类器在 test2 上的混淆矩阵	11
8	模拟二类高斯数据上的参数 Bayes 与非参数 Bayes 决策边界	13
9	模拟三类高斯数据上的参数 Bayes 与非参数 Bayes 决策边界	13
10	模拟二类高斯数据测试集混淆矩阵	14
11	模拟三类高斯数据测试集混淆矩阵	14
12	UCI Iris 数据集 PCA 二维投影下的分类边界	15
13	UCI Iris 测试集混淆矩阵	15
14	MNIST 数据集 k 近邻 Bayes 测试集混淆矩阵	16
15	MNIST 测试样本 PCA 投影及分类正确性	16
16	MNIST 部分测试样本预测效果	17
17	CIFAR-10 参数高斯 Bayes 测试集混淆矩阵	18
18	CIFAR 数据集 PCA 投影及分类正确性	18
19	CIFAR 部分测试样本预测效果	19
20	扩展数据集上四类 Bayes 分类方法准确率比较	20
21	实验二程序流程图	24
22	参数高斯 Bayes 与 Parzen 窗 Bayes 决策边界比较	25
23	Fisher 线性判别与 Bayes 分类器决策边界比较	26
24	训练样本在 Fisher 投影方向上的分布	26
25	训练错误率、留一法错误率与 test2 错误率比较	27
26	模拟数据上的 Fisher/LDA 与 Parzen 决策边界	28
27	模拟数据 Fisher/LDA 测试集混淆矩阵	28
28	UCI Iris 数据上的 LDA 与非参数方法	29
29	MNIST 数据上的 Fisher/LDA 分类结果	29
30	MNIST 数据 LDA 部分测试样本预测结果	30
31	CIFAR 图像数据上的 Fisher/LDA 分类结果	30
32	CIFAR 数据 LDA 部分测试样本预测结果	31
33	扩展数据集上三类分类器测试错误率比较	32
34	实验三程序流程图	36
35	K-L 主方向、类均值差方向与 Fisher 方向比较	37
36	训练样本在 K-L 第一主成分方向上的投影分布	38

37	训练样本在类均值差方向上的投影分布	38
38	K-L、类均值差、Fisher 与 Bayes 分类边界比较	39
39	模拟数据的 PCA/K-L 投影与 LDA 投影比较	40
40	模拟二类高斯数据在第一主成分上的投影分布	40
41	UCI Iris 数据的 K-L/PCA 特征提取结果	41
42	图像数据 K-L/PCA 累计方差贡献率	41
43	MNIST 数据的 K-L/PCA 特征提取结果	42
44	MNIST 数据 PCA 特征分类的部分测试样本预测结果	42
45	CIFAR 数据的 PCA/K-L 投影与 LDA 投影比较	43
46	CIFAR 数据 PCA 特征分类的部分测试样本预测结果	43
47	扩展数据集上 PC1、PCA 特征和 LDA 的分类准确率比较	44
48	Olivetti Faces 数据集样本示例	48
49	基于 K-L 变换的特征脸识别流程	49
50	训练集平均脸	49
51	K-L/PCA 提取的前 15 个特征脸	50
52	特征脸数量与累计方差贡献率	50
53	不同特征脸数量下的人脸识别准确率	51
54	测试人脸在前两个特征脸系数维度上的预测分布	52
55	Eigenfaces+SVM 在人脸测试集上的归一化混淆矩阵	52
56	Eigenfaces+SVM 人脸预测正确与错误样例	53
57	飞机分类补充数据集样本示例	54
58	飞机图像补充实验中提取的前 10 个主成分	54
59	飞机图像补充实验中不同主成分数下的测试准确率	54
60	飞机图像补充实验 PCA+1NN 归一化混淆矩阵	55
61	实验五聚类分析流程图	59
62	训练集 C 均值聚类结果, $C = 2$	60
63	不同初始中心对 C 均值聚类结果的影响	61
64	训练集上不同类别数的 C 均值聚类指标	62
65	训练集 Ward 分级聚类树状图	63
66	训练集 Ward 分级聚类结果, $C = 2$	63
67	训练集与 test2 合并后的 C 均值聚类结果, $C = 2$	64
68	训练集与 test2 合并后不同类别数的 C 均值聚类指标	64
69	训练集与 test2 合并后的 Ward 分级聚类结果, $C = 2$	65
70	DT-ProtoYOLO 总体架构图	69
71	DT-ProtoYOLO 定量结果可视化	71
72	DT-ProtoYOLO 开集检测定性示例	72
73	基于 CNN 的轨迹图像视觉分类器结构	75

74	视觉分类器奖励引导的无人机数字轨迹生成流程	77
75	补充实验二轨迹生成与技能可视化结果	78

表 目 录

1	实验软件环境	1
2	本仓库可用数据集	1
3	实验一最小风险 Bayes 决策损失表	3
4	单特征 Bayes 分类准确率 (先验概率 0.5 vs. 0.5)	7
5	二维特征 Bayes 分类准确率 (先验概率 0.5 vs. 0.5)	8
6	不同先验概率下二维完整协方差 Bayes 分类准确率	9
7	最小风险 Bayes 分类准确率	9
8	参数估计与非参数估计分类性能比较	10
9	实验一主要方法汇总结果	11
10	实验一扩展数据集 Bayes 分类结果	19
11	实验二分类器错误率汇总	27
12	实验二扩展数据集错误率汇总	31
13	实验三 K-L 变换与相关分类方法结果汇总	39
14	实验三扩展数据集 K-L/PCA 特征提取结果	43
15	Olivetti Faces 人脸数据集基本信息	48
16	不同特征脸数量下的人脸识别结果	51
17	120 个特征脸下的人脸识别结果	51
18	飞机图像补充实验结果	55
19	实验五使用的数据集统计	58
20	训练集 C 均值聚类与真实性别的对应关系	60
21	训练集 C 均值不同初始值实验结果	61
22	训练集不同类别数下的 C 均值聚类指标	62
23	训练集 Ward 分级聚类与真实性别的对应关系	63
24	训练集与 test2 合并后不同类别数下的 C 均值聚类指标	65
25	合并数据 C 均值聚类与真实性别的对应关系	65
26	合并数据 Ward 分级聚类与真实性别的对应关系	65
27	实验五聚类结果汇总	66
28	补充实验一主要模块与设置	69
29	补充实验一开集检测评价指标	70
30	补充实验一论文定量结果	70
31	补充实验二主要模块与设置	76
32	补充实验二评价指标与结果记录	77

1 实验环境与数据集

1.1 实验环境

本实验使用仓库内 Python 虚拟环境 `.venv`。主要软件环境如下：

表 1: 实验软件环境

项目	版本或说明
Python	3.11, 仓库内虚拟环境 <code>.venv</code>
NumPy / SciPy	数值计算与矩阵运算
Pandas	数据读取、整理与统计
Matplotlib / Seaborn	实验图表绘制
Scikit-learn	基线模型、评价指标、辅助算法
OpenCV / Pillow	图像读取与预处理
Jupyter	交互式实验记录

1.2 数据集说明

表 2: 本仓库可用数据集

编号	数据集	路径	状态
1	男女身高体重数据集	experiment/男女 data 数据集/ data/	已具备
2	模拟高斯数据集	datasets/02_simulated_ gaussian/	已生成
3	UCI Iris	datasets/03_uci_iris/	已下载并划分
4	MNIST	datasets/04_mnist/	已下载
5	CIFAR-10 CIFAR-100	/ datasets/05_cifar/	已下载并解压
6	飞机分类数据集	experiment/plane_dataset_4_1/	已具备

2 实验一：Bayes 分类器实验

2.1 模式识别理论知识

2.1.1 分类问题与 Bayes 公式

模式识别中的分类问题定义为：给定样本特征向量 $\mathbf{x}$ ，判定其所属类别。本实验的类别为女生 ω_F 和男生 ω_M ，特征可以只取身高或体重，也可以同时取二维向量

$$\mathbf{x} = [\text{身高}, \text{体重}]^T.$$

Bayes 分类器根据类条件概率密度和先验概率计算类别后验概率。根据 Bayes 公式，类别后验概率为

$$P(\omega_i|\mathbf{x}) = \frac{p(\mathbf{x}|\omega_i)P(\omega_i)}{p(\mathbf{x})}, \quad i \in \{F, M\}.$$

其中 $p(\mathbf{x}|\omega_i)$ 是类条件概率密度， $P(\omega_i)$ 是先验概率。由于对两个类别比较时 $p(\mathbf{x})$ 相同，最小错误率 Bayes 分类器只需比较

$$p(\mathbf{x}|\omega_i)P(\omega_i).$$

因此，若 $p(\mathbf{x}|\omega_F)P(\omega_F) > p(\mathbf{x}|\omega_M)P(\omega_M)$ ，则判为女生；否则判为男生。程序实现时，为避免概率密度数值过小导致下溢，等价地比较对数判别函数

$$g_i(\mathbf{x}) = \ln p(\mathbf{x}|\omega_i) + \ln P(\omega_i).$$

训练阶段根据已标注样本估计各类别在特征空间中的概率分布；测试阶段将待测样本代入两个类别的判别式，并选择后验概率较大的类别。对于身高体重数据，若样本靠近女生分布中心，则 $p(\mathbf{x}|\omega_F)$ 较大；若样本靠近男生分布中心，则 $p(\mathbf{x}|\omega_M)$ 较大。先验概率 $P(\omega_i)$ 表示分类前对类别比例的设定。

2.1.2 高斯分布假设与最大似然参数估计

在参数估计实验中，假设每一类样本服从正态分布。单特征时采用一维高斯分布：

$$p(x|\omega_i) = \frac{1}{\sqrt{2\pi}\sigma_i} \exp \left[-\frac{(x - \mu_i)^2}{2\sigma_i^2} \right].$$

二维特征时采用多维高斯分布：

$$p(\mathbf{x}|\omega_i) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}_i|^{1/2}} \exp \left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i) \right].$$

最大似然估计（MLE）就是选择最能解释训练样本的参数。对高斯分布而言，均值和协方差的最大似然估计为

$$\hat{\boldsymbol{\mu}}_i = \frac{1}{N_i} \sum_{k=1}^{N_i} \mathbf{x}_k, \quad \hat{\boldsymbol{\Sigma}}_i = \frac{1}{N_i} \sum_{k=1}^{N_i} (\mathbf{x}_k - \hat{\boldsymbol{\mu}}_i)(\mathbf{x}_k - \hat{\boldsymbol{\mu}}_i)^T.$$

因此，本实验中的参数估计实例选择最大似然估计：程序分别对女生类和男生类计算样本均值、方差或协方差矩阵，再把这些参数代入高斯 Bayes 判别函数，得到可用于分类的概率密度模型。如果认为身高和体重相关，则使用完整协方差矩阵；如果认为二者不相关，则只保留协方差矩阵对角线，相当于把二维密度拆成两个独立的一维密度相乘。

完整协方差模型能够描述“身高越高体重往往越重”的线性相关关系，因此决策边界可以随数据分布方向发生旋转；对角协方差模型忽略相关性，假设身高和体重分别独立提供判别信息，模型更简单、参数更少。在训练样本数量有限时，复杂模型不一定总是带来更好的测试性能，这也是本实验比较相关和不相关假设的原因。

2.1.3 最小风险 Bayes 分类器

最小错误率 Bayes 分类器把所有错误看作同等代价，决策目标是使错误概率最小。当不同类型错误造成的代价不同时，需要使用最小风险 Bayes 分类器。最小风险准则不直接比较后验概率大小，而是先计算采取某一决策后的条件风险，再选择条件风险最小的决策。本实验给出如下损失表：

表 3: 实验一最小风险 Bayes 决策损失表

真实类别	判为女生	判为男生
女生 ω_F	0	2
男生 ω_M	1	0

设共有 M 个类别， α_i 表示采取第 i 个决策， λ_{ij} 表示真实类别为 ω_j 而采取决策 α_i 时的损失，则条件风险定义为

$$R_i(\mathbf{x}) = \sum_{j=1}^M \lambda_{ij} P(\omega_j | \mathbf{x}).$$

其中 $P(\omega_j | \mathbf{x})$ 为样本 $\mathbf{x}$ 属于真实类别 ω_j 的后验概率， λ_{ij} 为在该真实类别下采取决策 α_i 的损失。最小风险 Bayes 决策选择条件风险最小的决策：

$$\alpha^*(\mathbf{x}) = \arg \min_i R_i(\mathbf{x}).$$

对于本实验的二分类问题，若决策 α_F 表示判为女生，决策 α_M 表示判为男生，则 $M = 2$,

条件风险可按损失表展开为

$$R_F(\mathbf{x}) = \lambda_{FF}P(\omega_F|\mathbf{x}) + \lambda_{FM}P(\omega_M|\mathbf{x}) = 0 \cdot P(\omega_F|\mathbf{x}) + 1 \cdot P(\omega_M|\mathbf{x}) = P(\omega_M|\mathbf{x}),$$

$$R_M(\mathbf{x}) = \lambda_{MF}P(\omega_F|\mathbf{x}) + \lambda_{MM}P(\omega_M|\mathbf{x}) = 2 \cdot P(\omega_F|\mathbf{x}) + 0 \cdot P(\omega_M|\mathbf{x}) = 2P(\omega_F|\mathbf{x}).$$

因此，当

$$R_F(\mathbf{x}) < R_M(\mathbf{x}) \iff P(\omega_M|\mathbf{x}) < 2P(\omega_F|\mathbf{x})$$

时判为女生，否则判为男生。后验概率仍由 Bayes 公式计算：

$$P(\omega_i|\mathbf{x}) = \frac{p(\mathbf{x}|\omega_i)P(\omega_i)}{\sum_{j \in \{F, M\}} p(\mathbf{x}|\omega_j)P(\omega_j)}.$$

表 3 中女生被误判为男生的代价为 2，男生被误判为女生的代价为 1，因此最小风险决策会扩大女生判决区域，以降低高损失错误的发生次数。

2.1.4 非参数估计方法

非参数估计不预先规定类条件概率密度必须服从某一种参数化分布。与高斯 Bayes 中“先假设分布形式，再估计均值和协方差”的过程不同，非参数方法主要利用训练样本在特征空间中的局部排列情况完成密度估计或分类。本实验实现 Parzen 窗和 k 近邻两种方法。

Parzen 窗密度估计。 Parzen 窗是一种概率密度估计方法。对类别 ω_i ，设该类训练样本为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_i}$ ，采用高斯核函数时，类条件概率密度估计为

$$\hat{p}(\mathbf{x}|\omega_i) = \frac{1}{N_i} \sum_{k=1}^{N_i} \frac{1}{(2\pi)^{d/2}h^d} \exp \left[-\frac{1}{2} \left\| \frac{\mathbf{x} - \mathbf{x}_k}{h} \right\|^2 \right].$$

其中 d 为特征维数， h 为窗宽。每个训练样本都可以看作一个以自身为中心的高斯核，所有核函数叠加后得到该类别的概率密度估计。窗宽 h 决定单个样本影响范围： h 较小时，密度估计更依赖局部样本，决策边界可能更曲折； h 较大时，密度估计更平滑，但可能削弱局部结构。本实验在二维身高体重特征上使用高斯核 Parzen 窗，得到 $\hat{p}(\mathbf{x}|\omega_i)$ 后代入 Bayes 判别规则，比较

$$\hat{p}(\mathbf{x}|\omega_i)P(\omega_i).$$

k 近邻分类。 k 近邻方法不显式估计概率密度。对待分类样本 $\mathbf{x}$ ，先计算它与训练样本之间的距离，选取距离最近的 k 个样本，再按邻居类别多数投票确定类别：

$$\hat{\omega}(\mathbf{x}) = \arg \max_{\omega_i} \sum_{\mathbf{x}_k \in N_k(\mathbf{x})} I(y_k = \omega_i),$$

其中 $N_k(\mathbf{x})$ 表示 $\mathbf{x}$ 的 k 个最近邻集合， $I(\cdot)$ 为指示函数。本实验使用欧氏距离和 $k = 5$ 。

k 较小时分类结果对个别样本敏感, k 较大时局部细节被平均。Parzen 窗和 k 近邻均依赖样本局部分布, 但 Parzen 窗先估计密度再按 Bayes 规则分类, k 近邻直接根据邻域样本标签分类。

2.2 实际数据集与参数计算

训练集由 50 个女生样本和 50 个男生样本组成, 测试集 `test1.txt` 包含 15 个女生和 20 个男生, `test2.txt` 包含 50 个女生和 250 个男生。训练样本在二维平面上的分布如图 1 所示, 男生样本整体更偏向较高、较重区域, 但两类在边界附近存在重叠, 这也是分类错误的主要来源。

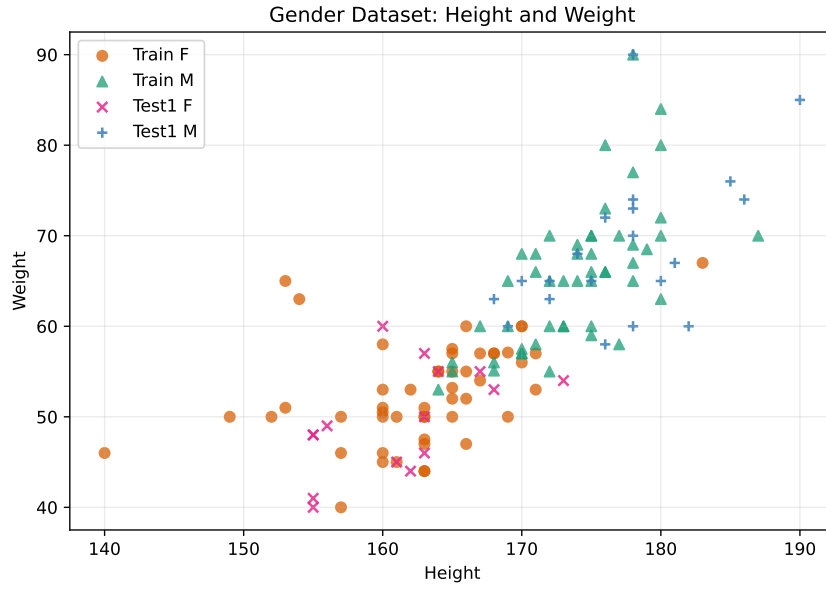


图 1: 男女训练样本身高-体重散点分布

由训练集使用最大似然估计得到的二维高斯参数如下:

$$\hat{\mu}_F = [162.840, 52.596]^T, \quad \hat{\Sigma}_F = \begin{bmatrix} 43.934 & 15.525 \\ 15.525 & 31.129 \end{bmatrix},$$

$$\hat{\mu}_M = [173.920, 65.502]^T, \quad \hat{\Sigma}_M = \begin{bmatrix} 20.754 & 23.058 \\ 23.058 & 59.898 \end{bmatrix}.$$

从均值看, 男生训练样本的平均身高和平均体重均高于女生; 从协方差看, 两类样本的身高与体重均呈正相关, 说明较高的样本往往也较重。男生协方差中的非对角元素更大, 表示男生样本中身高和体重的线性相关更明显。

本实验围绕同一组训练数据逐步改变分类器设计因素, 具体设置如下:

1. 单特征实验: 分别只使用身高或体重, 观察单一特征对性别分类的贡献。

2. 双特征实验：同时使用身高和体重，比较完整协方差和对角协方差两种高斯模型。
3. 先验概率实验：在二维完整协方差模型下改变 $P(F)$ 和 $P(M)$ ，观察决策边界偏移和错误率变化。
4. 最小风险实验：引入损失表，使分类器不只追求错误率最低，还考虑不同错误的代价。
5. 非参数实验：使用 Parzen 窗和 k 近邻，与参数化高斯 Bayes 分类器进行对比。

评价指标包括训练集、`test1.txt` 和 `test2.txt` 上的准确率，以及 `test2` 上的混淆矩阵。由于 `test2` 中男生样本远多于女生样本，单纯观察总体准确率可能掩盖某一类的识别情况，因此需要同时关注女生召回率和男生召回率。

2.3 程序框图与代码实现

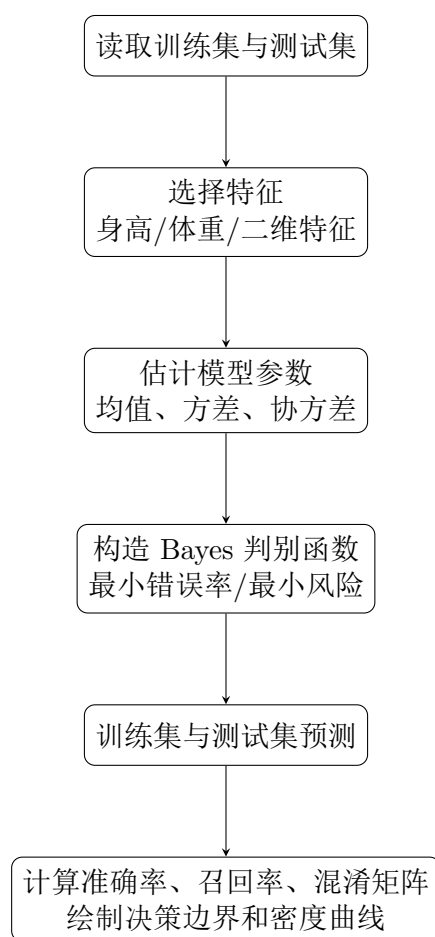


图 2: 实验一程序流程图

关键程序位于 `scripts/experiment1_bayes.py`。数据读取时先把测试集中的性别标签统一为 F 和 M，避免大小写差异影响评价。相关实现见附录代码 1。

参数估计部分直接实现高斯分布的最大似然估计。`covariance_mode="full"` 表示考虑身高体重相关，`covariance_mode="diag"` 表示假设二者不相关。相关实现见附录代码 2。

最小错误率 Bayes 分类器使用对数判别函数，避免直接计算很小的概率密度导致数值下溢。相关实现见附录代码 3。

最小风险 Bayes 分类器先由对数后验得分恢复后验概率，再根据损失表计算条件风险。相关实现见附录代码 4。

非参数方法中，Parzen 窗使用高斯核估计类条件密度， k 近邻使用欧氏距离投票。相关实现见附录代码 5。

2.4 实验步骤与结果分析

2.4.1 步骤一：单个特征下的最小错误率 Bayes 分类

首先分别只使用身高和只使用体重作为特征，在一维正态分布假设下估计每类均值和方差，并建立最小错误率 Bayes 分类器。图 3 给出了训练样本直方图、估计得到的一维高斯密度曲线以及等先验条件下的分类阈值。

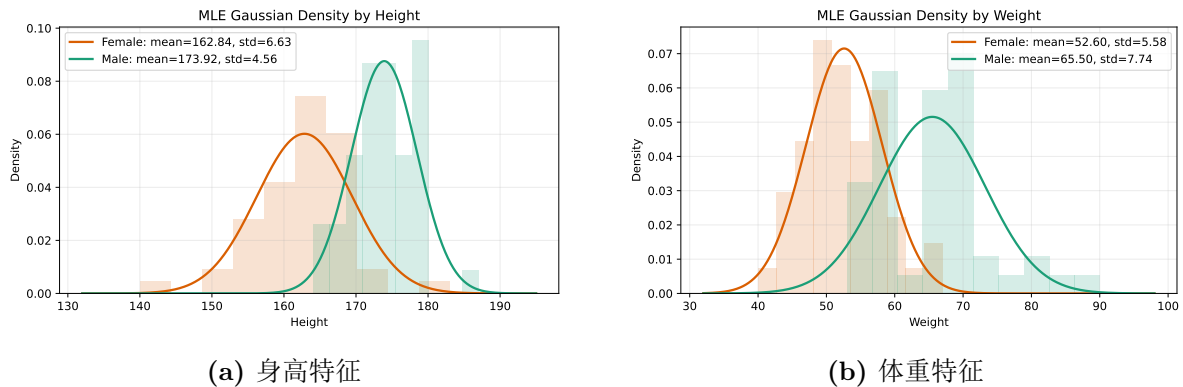


图 3: 单特征高斯密度估计与 Bayes 判决阈值

在等先验条件下，单特征 Bayes 分类器可以近似理解为“超过某个阈值更像男生，否则更像女生”。由训练集估计出的高斯密度交点可得，身高特征在主要样本范围内的判决阈值约为 168.42 cm，体重特征的主要判决阈值约为 59.07 kg。由于男女两类的方差并不完全相同，高斯曲线在远离样本主体区域也可能出现第二个交点，但该交点处样本密度极低，对实际测试样本分类影响很小。

表 4: 单特征 Bayes 分类准确率（先验概率 0.5 vs. 0.5）

特征	训练集	test1	test2
身高	86.00%	94.29%	91.00%
体重	82.00%	94.29%	85.33%

可以看到，身高单特征在训练集和 test2 上均优于体重单特征，说明该数据集中身高对性别区分更稳定。体重也有明显判别能力，但两类体重分布重叠更大，因此在样本较多且类别比例不均衡的 test2 上错误率更高。

2.4.2 步骤二：两个特征下的 Bayes 分类

然后同时使用身高和体重两个特征，分别在“相关”和“不相关”假设下建立二维高斯 Bayes 分类器。完整协方差模型对应相关假设，对角协方差模型对应不相关假设。等先验下完整协方差模型的二维决策边界如图 4 所示。

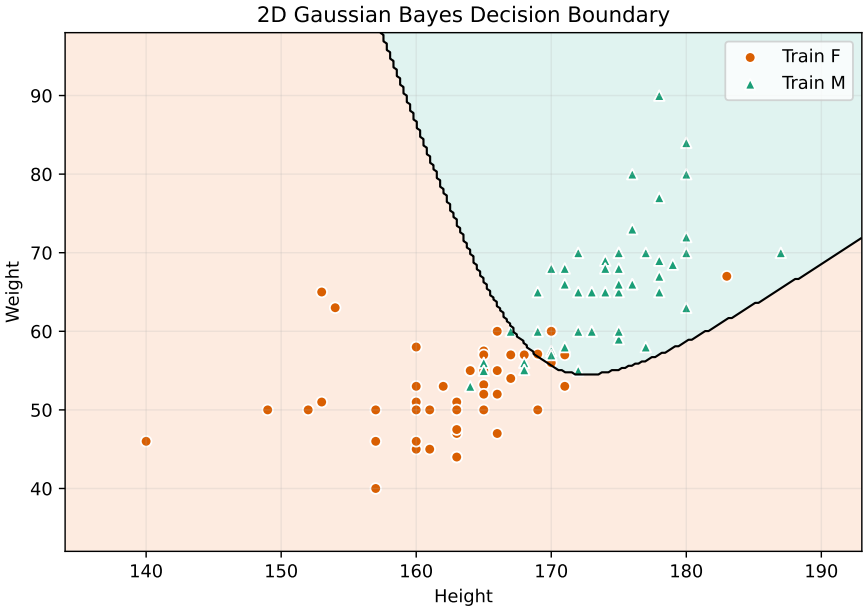


图 4：二维高斯 Bayes 分类器决策边界

表 5：二维特征 Bayes 分类准确率（先验概率 0.5 vs. 0.5）

协方差假设	训练集	test1	test2
完整协方差（相关）	88.00%	97.14%	89.33%
对角协方差（不相关）	88.00%	97.14%	90.33%

二维特征在 test1 上达到 97.14%，高于单特征实验，说明身高和体重联合使用可以提供更多判别信息。但在 test2 上，对角协方差略优于完整协方差，说明完整协方差虽然表达能力更强，也可能更受训练样本规模和测试集分布差异影响；在样本较少时，较简单的不相关模型反而可能具有更好的泛化表现。

2.4.3 步骤三：不同先验概率对决策规则的影响

在 Bayes 判别函数中，先验概率通过 $\ln P(\omega_i)$ 改变决策边界的位置。图 5 展示了先验从均衡逐渐偏向女生时，二维 Bayes 决策区域的变化。

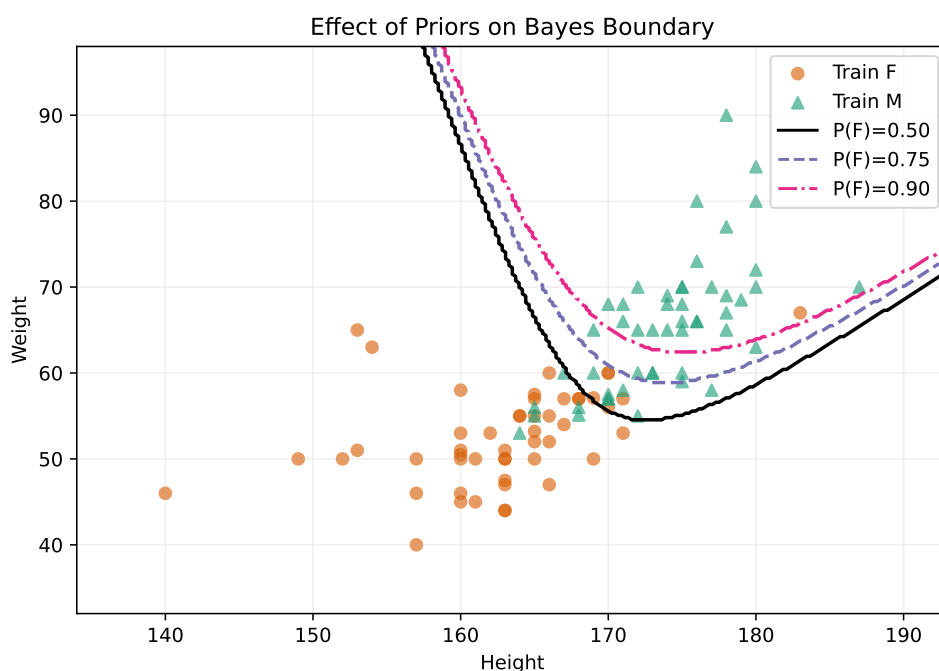


图 5: 不同先验概率下的 Bayes 决策边界比较

表 6: 不同先验概率下二维完整协方差 Bayes 分类准确率

先验概率 $P(F)$ vs. $P(M)$	训练集	test1	test2
0.50 vs. 0.50	88.00%	97.14%	89.33%
0.75 vs. 0.25	86.00%	85.71%	80.00%
0.90 vs. 0.10	79.00%	80.00%	67.33%

由于 test2 中男生占多数，若人为设置更偏向女生的先验，会扩大女生判决区域，使更多男生被误判为女生，因此总体准确率明显下降。这说明先验概率不是随意调节的超参数，而应尽量反映真实应用场景中的类别比例或任务偏好。

2.4.4 步骤四：最小风险 Bayes 决策

在表 3 的损失设定下，女生被错判为男生的损失为 2，男生被错判为女生的损失为 1。因此分类器会更倾向于减少女生漏判。实验结果如下：

表 7: 最小风险 Bayes 分类准确率

方法	训练集	test1	test2
最小风险 Bayes	84.00%	94.29%	84.33%

test2 上女生 50 个样本中有 49 个被正确识别，仅 1 个女生被判为男生；但男生中有 46 个被判为女生。结果表明，在该损失设定下，最小风险决策降低了女生被误判为

男生的次数，同时增加了男生被误判为女生的次数。因此，最小错误率准则和最小风险准则对应不同的优化目标：前者最小化平均错误率，后者最小化期望损失。

2.4.5 步骤五：非参数估计实验

最后实现 Parzen 窗和 k 近邻方法，并与参数化二维 Bayes 分类器比较。图 6 展示了非参数方法得到的决策边界。

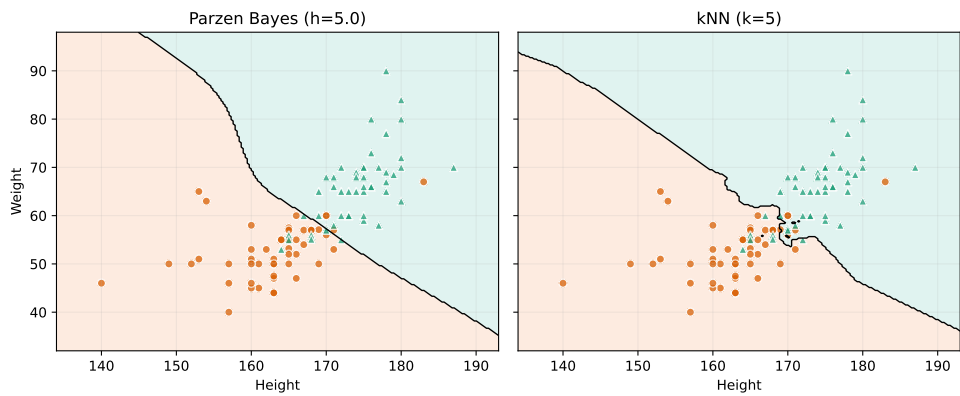


图 6: Parzen 窗和 k 近邻分类边界

表 8: 参数估计与非参数估计分类性能比较

方法	训练集	test1	test2
二维高斯 Bayes（完整协方差）	88.00%	97.14%	89.33%
Parzen 窗 Bayes	87.00%	97.14%	90.33%
k 近邻	89.00%	100.00%	90.33%

Parzen 窗和 k 近邻都不强制数据服从高斯分布，因此边界形状更加灵活。在本数据集中，非参数方法在 test2 上略优于完整协方差高斯 Bayes，但差距不大，说明身高体重数据整体接近可由简单概率模型描述，同时局部非参数方法能对边界附近样本做一定修正。

2.5 综合实验结果与分析总结

表 9: 实验一主要方法汇总结果

方法	设置	训练准确率	test1 准确率	test2 准确率
单特征 Bayes	身高特征; $P(F) = 0.50$, $P(M) = 0.50$	86.00%	94.29%	91.00%
单特征 Bayes	体重特征; $P(F) = 0.50$, $P(M) = 0.50$	82.00%	94.29%	85.33%
二维相关 Bayes	身高 + 体重; 完整协方差; $P(F) = 0.50$, $P(M) = 0.50$	88.00%	97.14%	89.33%
二维不相关 Bayes	身高 + 体重; 对角协方差; $P(F) = 0.50$, $P(M) = 0.50$	88.00%	97.14%	90.33%
不同先验 Bayes	身高 + 体重; 完整协方差; $P(F) = 0.75$, $P(M) = 0.25$	86.00%	85.71%	80.00%
不同先验 Bayes	身高 + 体重; 完整协方差; $P(F) = 0.90$, $P(M) = 0.10$	79.00%	80.00%	67.33%
最小风险 Bayes	身高 + 体重; 完整协方差; $P(F) = 0.50$, $P(M) = 0.50$; $\lambda_{MF} = 2$, $\lambda_{FM} = 1$	84.00%	94.29%	84.33%
Parzen Bayes	身高 + 体重; 高斯核; $h = 5.0$; $P(F) = 0.50$, $P(M) = 0.50$	87.00%	97.14%	90.33%
k 近邻	身高 + 体重; $k = 5$	89.00%	100.00%	90.33%

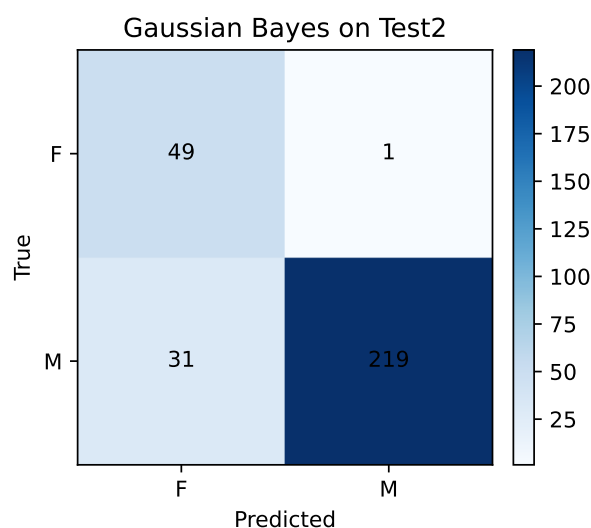


图 7: 二维高斯 Bayes 分类器在 test2 上的混淆矩阵

以 test2 为例, 二维完整协方差 Bayes 分类器将 50 个女生中的 49 个正确识别为女生, 女生召回率为 98.00%; 同时将 250 个男生中的 219 个正确识别为男生, 男生召

回率为 87.60%。这说明该分类器对女生样本较敏感，但仍会把一部分靠近边界、身高体重偏低的男生判为女生。二维对角协方差、Parzen 窗和 k 近邻在 test2 上的准确率均为 90.33%，对应女生召回率为 96.00%、男生召回率为 89.20%，整体更均衡。

综合来看，本实验可以得到以下结论：

- 身高和体重都具有性别判别能力，其中身高单特征更稳定；二维联合特征通常优于单特征。
- 完整协方差模型能够描述身高和体重的相关性，但在训练样本较少时，对角协方差模型可能泛化更稳。
- 先验概率会直接改变决策边界。若先验与测试集类别比例不一致，可能显著降低总体准确率。
- 最小风险 Bayes 决策适合错误代价不对称的任务；它不一定使错误率最低，但能降低指定任务中的期望损失。
- 非参数方法减少了分布假设，边界更灵活，但需要选择窗宽或邻居数等超参数，并且对样本规模和局部分布更敏感。

2.6 扩展数据集实验

除男女身高体重数据集外，实验一还按照作业要求补充了模拟高斯数据、UCI Iris、MNIST、CIFAR-10 和 CIFAR-100 数据集上的 Bayes 分类实验。扩展实验仍围绕“参数估计、非参数估计、最小错误率、最小风险”四类 Bayes 决策展开，用于考察同一类分类准则在不同数据复杂度下的适用性。

不同数据集的维数、类别数和样本复杂度差异较大，因此实验设置如下：

- 模拟数据按 iid 原则由二维多元正态分布生成，包括二类数据和三类数据，每类 200 个样本，训练集与测试集比例为 60% 和 40%。
- UCI Iris 使用 4 个连续特征，包括花萼长度、花萼宽度、花瓣长度和花瓣宽度，类别数为 3。
- MNIST、CIFAR-10 和 CIFAR-100 属于高维图像数据。为避免高维协方差矩阵奇异，并控制计算量，先对图像向量进行标准化和 PCA 降维，再在 64 维特征上进行分类。
- 参数估计方法使用高斯 Bayes 分类器。低维数据使用完整协方差矩阵，高维图像数据使用对角协方差矩阵。
- 非参数估计方法中，低维数据使用 Parzen 窗估计类条件概率密度，图像数据使用 k 近邻估计后验概率。
- 最小风险 Bayes 分类器采用统一损失设定：正确分类损失为 0，一般错分损失为 1；第一类样本被误判为其他类时损失为 2。

2.6.1 模拟高斯数据实验

模拟数据用于验证 Bayes 分类器在已知数据生成机制下的表现。二类数据和三类数据均由二维高斯分布独立同分布采样生成。每类共有 200 个样本，其中 60% 作为训练样本，40% 作为测试样本。由于数据真实来源就是高斯模型，参数高斯 Bayes 在该数据集上具有较强的模型匹配性；Parzen 窗 Bayes 不使用显式高斯假设，主要依靠样本局部密度估计。

图 8 和图 9 给出了二类和三类模拟数据上的决策边界。二类数据中，两类分布中心相距较远，边界主要穿过两类样本的重叠区域。三类数据中，多个类别之间存在更复杂的相邻关系，因此决策区域被划分为三个部分，边界交汇处更容易产生错分。

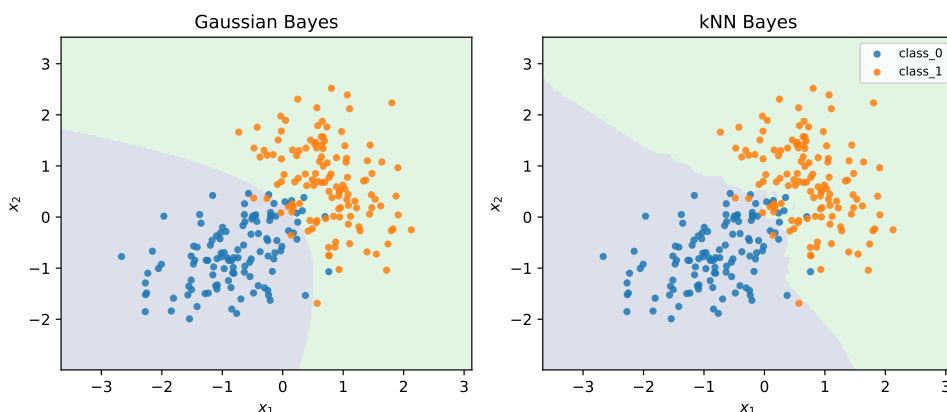


图 8: 模拟二类高斯数据上的参数 Bayes 与非参数 Bayes 决策边界

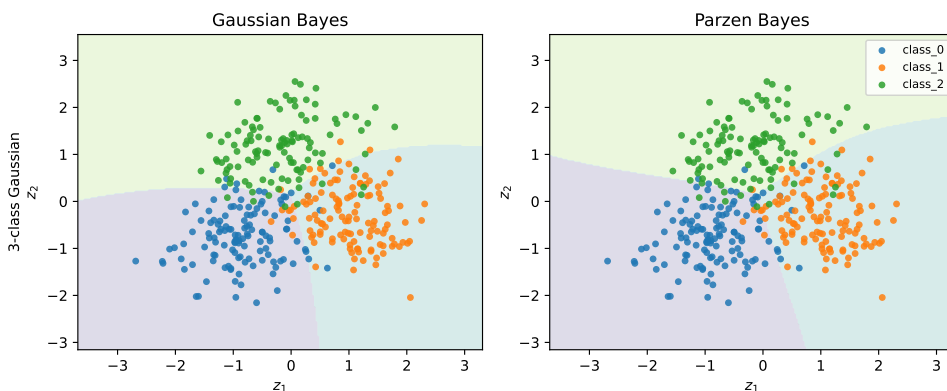


图 9: 模拟三类高斯数据上的参数 Bayes 与非参数 Bayes 决策边界

二类模拟数据的测试集混淆矩阵如图 10 所示。参数高斯 Bayes 和 Parzen 窗 Bayes 均能较好地地区分两类样本，错误主要发生在两类分布重叠区域。参数最小错误率 Bayes 在二类测试集上达到 97.50%，参数最小风险 Bayes 达到 98.75%；非参数最小错误率 Bayes 为 96.25%，非参数最小风险 Bayes 为 98.75%。三类模拟数据中，参数 Bayes 准确率为 90.00%，非参数 Parzen Bayes 为 89.17%，低于二类情形，原因是类别数增加后类间重叠和边界交会区域增多。

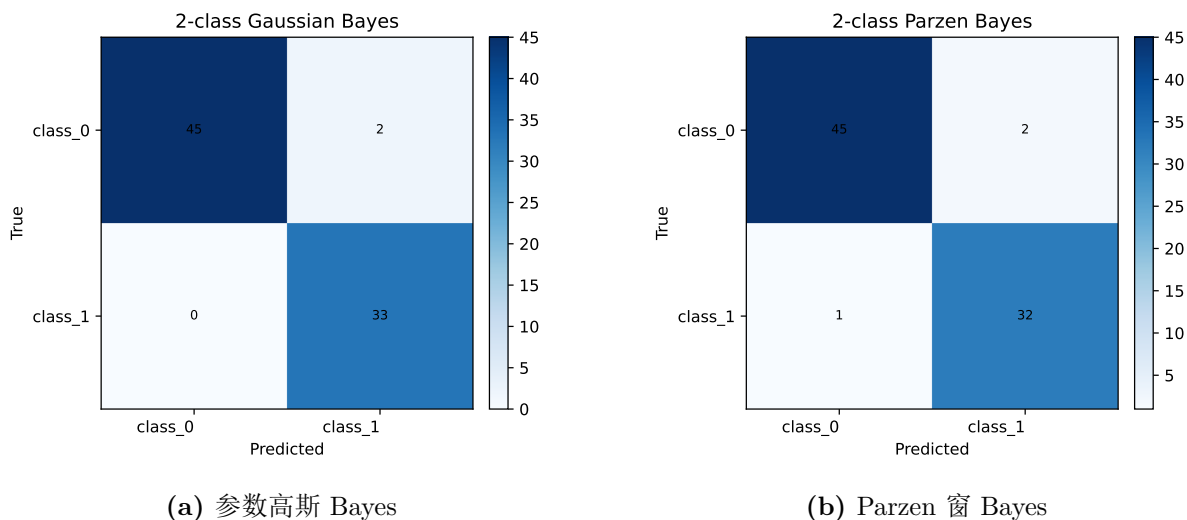


图 10: 模拟二类高斯数据测试集混淆矩阵

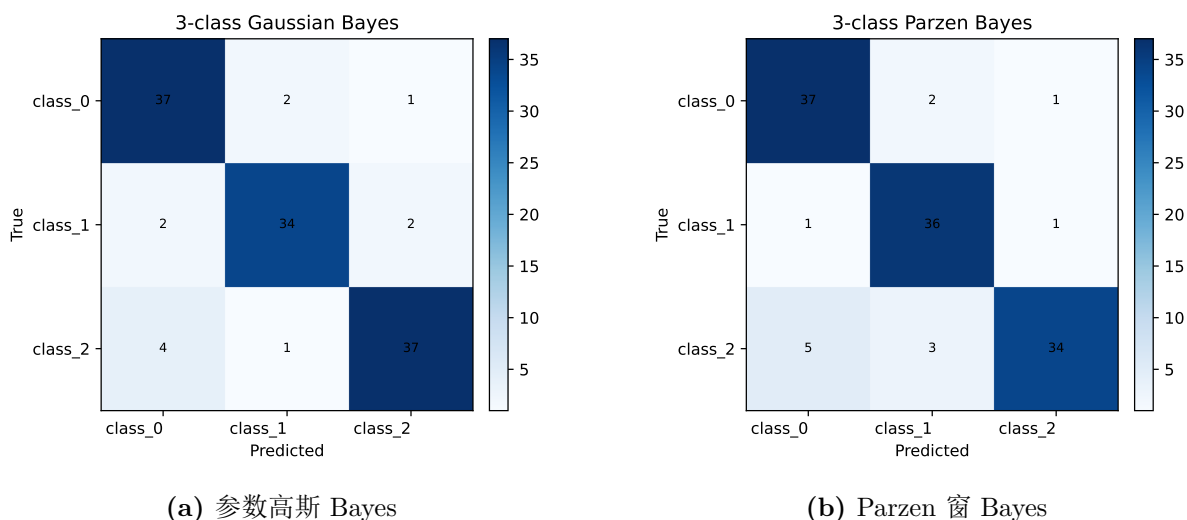


图 11: 模拟三类高斯数据测试集混淆矩阵

2.6.2 UCI Iris 数据集实验

UCI Iris 数据集包含 3 类鸢尾花，每个样本由花萼长度、花萼宽度、花瓣长度和花瓣宽度 4 个连续特征表示。本实验使用仓库中已划分好的训练集和测试集，分类器在 4 维原始特征标准化后进行训练。由于 Iris 数据类别结构清晰，尤其花瓣相关特征具有较强判别能力，因此它常用于检验传统分类器在低维连续数据上的效果。

图 12 将 Iris 样本投影到二维 PCA 平面后绘制参数 Bayes 和 Parzen Bayes 的分类边界。可以看到，一类样本与另外两类分离明显，后两类之间存在少量边界接近区域。图 13 给出了测试集混淆矩阵，参数高斯 Bayes 在测试集上达到 100.00%，Parzen 窗 Bayes 达到 96.67%。

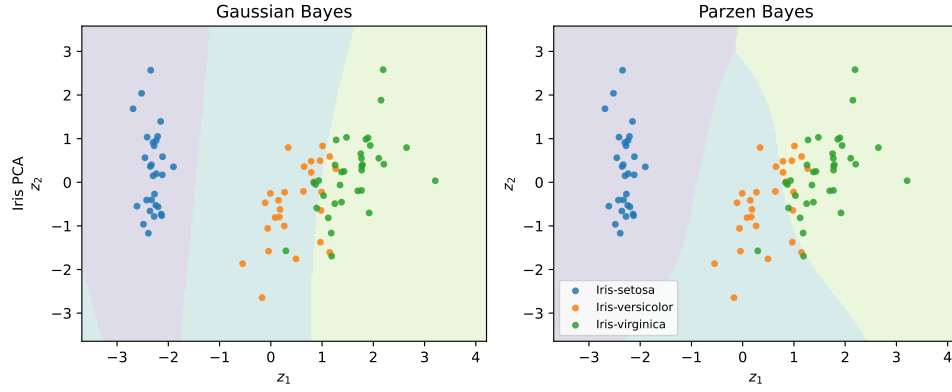
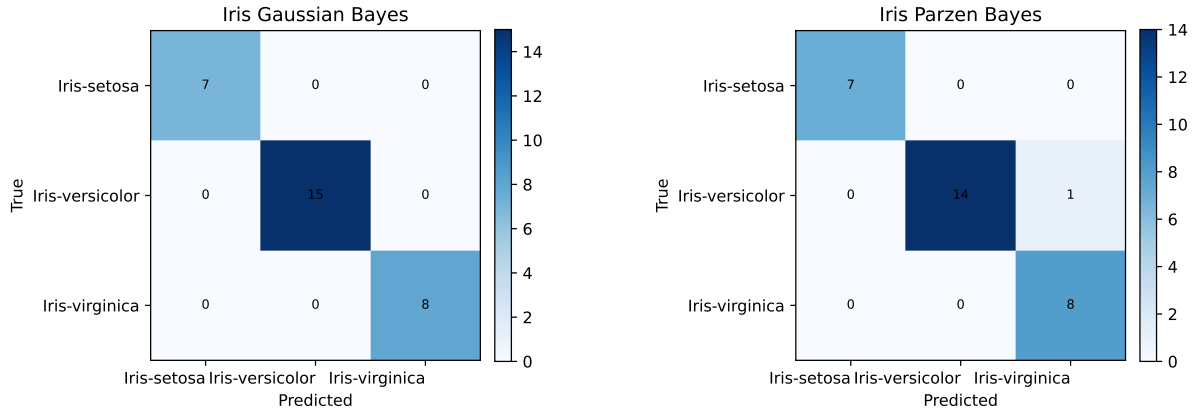


图 12: UCI Iris 数据集 PCA 二维投影下的分类边界



(a) 参数高斯 Bayes

(b) Parzen 窗 Bayes

图 13: UCI Iris 测试集混淆矩阵

2.6.3 MNIST 手写数字数据集实验

MNIST 数据集由 28×28 灰度手写数字图像组成，类别为数字 0-9。原始图像展开后为 784 维向量，如果直接估计完整协方差矩阵，容易出现维数过高、协方差矩阵奇异和计算量过大的问题。因此本实验先对图像向量进行标准化，再用 PCA 降到 64 维，累计方差贡献率为 66.50%。参数方法使用对角协方差高斯 Bayes，非参数方法使用 $k = 5$ 的 k 近邻。

图 14 给出了 MNIST 上 k 近邻分类器的混淆矩阵。图 15 展示测试样本在 PCA 前两维上的投影，蓝色点表示分类正确，红色叉号表示分类错误。图 16 展示部分测试样本的预测效果。

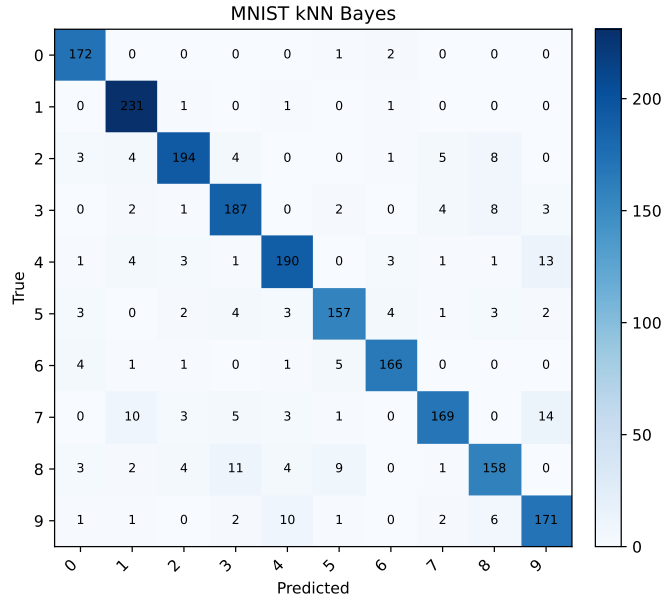


图 14: MNIST 数据集 k 近邻 Bayes 测试集混淆矩阵

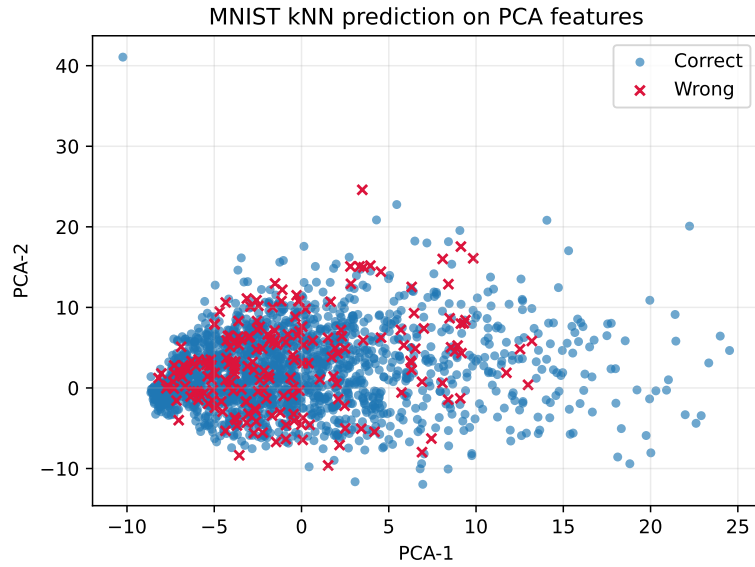


图 15: MNIST 测试样本 PCA 投影及分类正确性

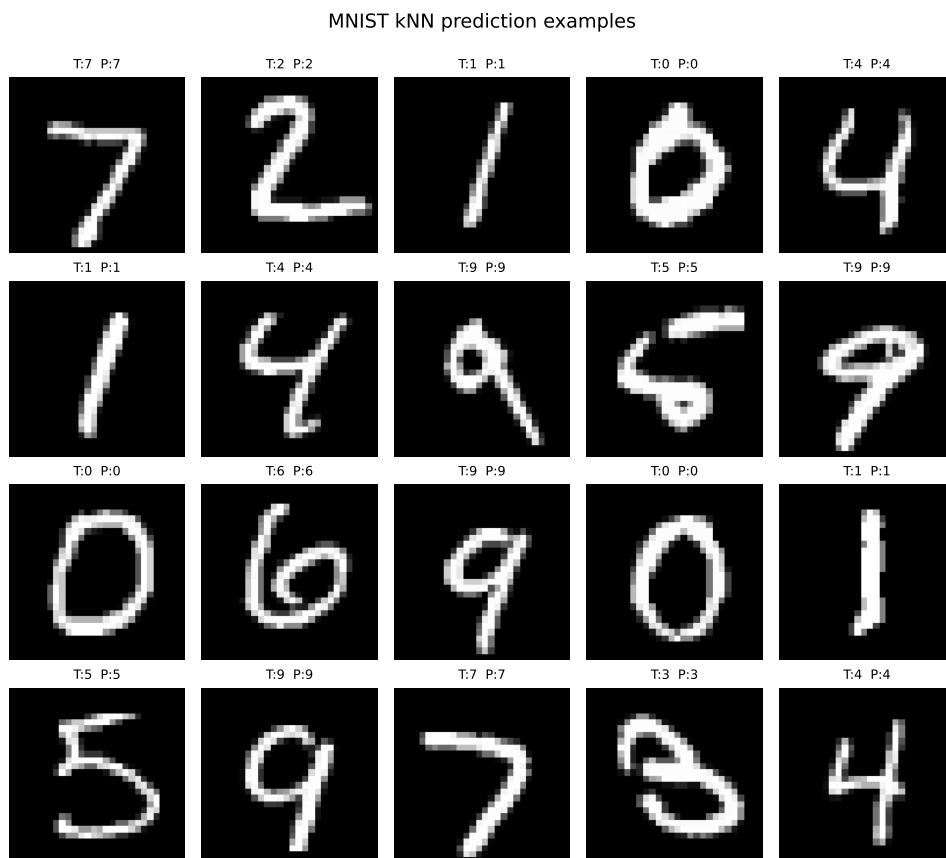


图 16: MNIST 部分测试样本预测效果

MNIST 上对角高斯 Bayes 准确率为 62.65%， k 近邻非参数方法达到 89.75%。该结果说明手写数字在 PCA 特征空间中并不符合简单的单峰高斯分布假设，不同书写风格会使同一数字类别形成多个局部结构； k 近邻利用局部邻域样本标签，因此在该数据集上明显优于对角高斯模型。

2.6.4 CIFAR-10 与 CIFAR-100 图像数据集实验

CIFAR-10 和 CIFAR-100 均为 32×32 彩色自然图像数据集。CIFAR-10 包含 10 类，CIFAR-100 包含 100 类，后者类别更细、类间差异更小。与 MNIST 相比，CIFAR 图像包含颜色、背景、姿态、尺度和纹理变化，原始像素特征与类别语义之间的关系更复杂。本实验同样采用标准化和 PCA 64 维特征，再使用对角高斯 Bayes 与 k 近邻进行分类。

图 17 给出了 CIFAR-10 参数高斯 Bayes 的测试集混淆矩阵，图 18 给出了 CIFAR-10 和 CIFAR-100 在 PCA 前两维上的分类正确性分布。可以看到，CIFAR 样本在低维 PCA 平面上重叠明显，错误样本大量分布在各类别混合区域中。

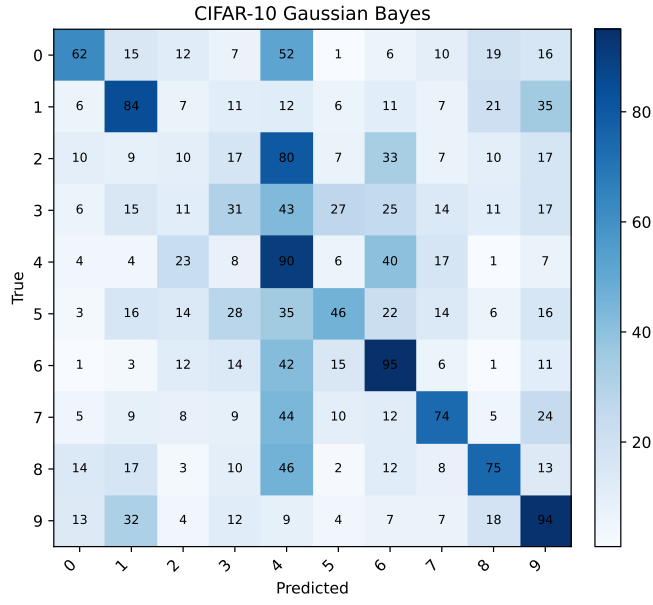
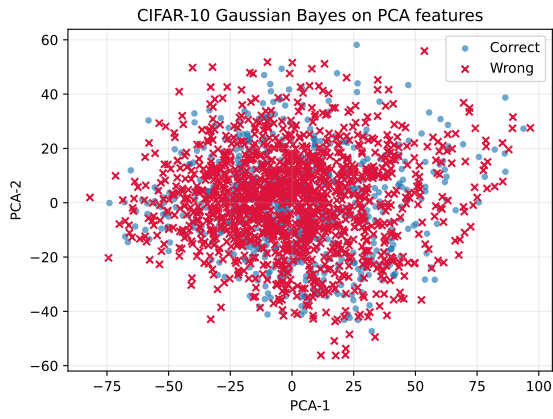
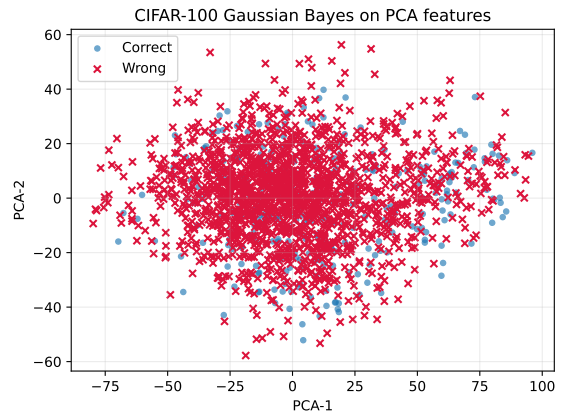


图 17: CIFAR-10 参数高斯 Bayes 测试集混淆矩阵



(a) CIFAR-10



(b) CIFAR-100

图 18: CIFAR 数据集 PCA 投影及分类正确性

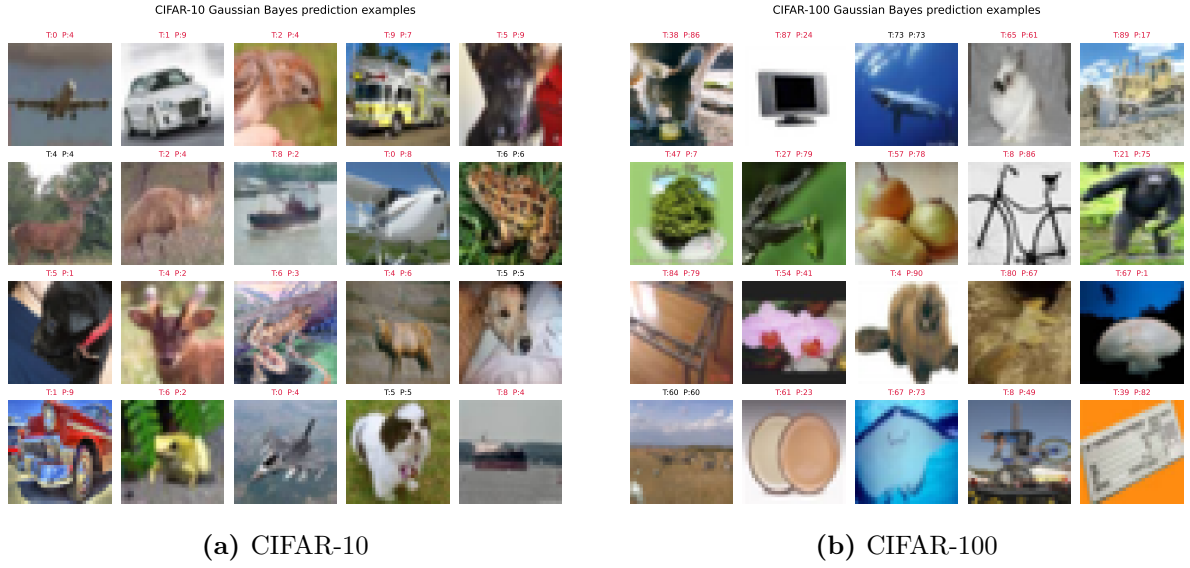


图 19: CIFAR 部分测试样本预测效果

CIFAR-10 上参数高斯 Bayes 准确率为 33.05%，CIFAR-100 上为 16.50%，均高于随机猜测，但明显低于 MNIST。非参数 k 近邻在 CIFAR-10 上为 30.30%，在 CIFAR-100 上为 10.65%。结果表明，对于复杂自然图像，仅使用 PCA 压缩后的像素特征难以形成稳定、可分的类别结构；传统 Bayes 分类器可以作为基线方法，但需要更有效的图像特征或深度模型才能获得较高性能。

2.6.5 扩展数据集结果汇总

各扩展数据集上的分类准确率汇总如表 10 所示，对应的准确率柱状图见图 20。

表 10: 实验一扩展数据集 Bayes 分类结果

数据集	实验设置	参数最小 错误率	参数最小 风险	非参数最小 错误率	非参数最小 风险
模拟二类高斯	原始二维特征；高斯 full； Parzen $h = 0.8$	97.50%	98.75%	96.25%	98.75%
模拟三类高斯	原始二维特征；高斯 full； Parzen $h = 0.8$	90.00%	90.00%	89.17%	85.00%
UCI Iris	4 维花萼/花瓣特征；高 斯 full；Parzen $h = 0.8$	100.00%	100.00%	96.67%	96.67%
MNIST	PCA 64 维；高斯 diag； kNN $k = 5$	62.65%	62.30%	89.75%	89.45%
CIFAR-10	PCA 64 维；高斯 diag； kNN $k = 7$	33.05%	33.20%	30.30%	28.85%
CIFAR-100	PCA 64 维；高斯 diag； kNN $k = 5$	16.50%	16.60%	10.65%	10.55%

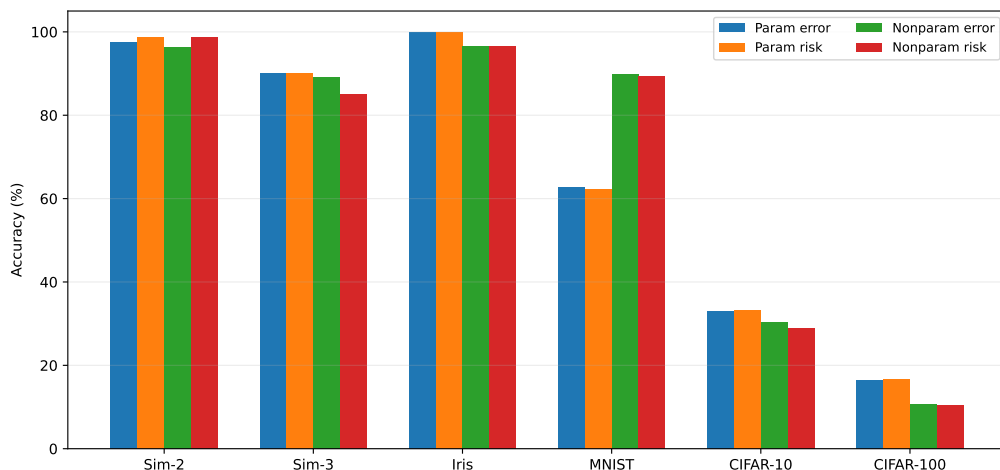


图 20: 扩展数据集上四类 Bayes 分类方法准确率比较

综合各数据集结果可得：当数据分布与高斯假设接近时，参数 Bayes 分类器效果较好；当类别分布呈现多峰局部结构时，非参数方法更有优势；当数据为复杂自然图像时，分类性能主要受特征表示限制，简单 PCA 像素特征不足以充分表达类别语义。

2.7 体会

通过本实验，我对 Bayes 分类器的认识从“会写判别公式”进一步变成了“理解公式背后的条件”。同样是比较后验概率，先验概率、密度估计方法、协方差假设和损失函数都会改变最后的分类结果。因此，分类器并不是孤立存在的，它和数据分布、特征选择以及任务目标是联系在一起的。

实验中比较有体会的一点是：模型假设不能脱离数据本身。高斯 Bayes 在一些分布清楚、维数较低的数据上表现较稳定，是因为模型假设和数据特点比较一致；但遇到更复杂的数据时，简单的高斯假设就会显得不够。非参数方法虽然少了一些分布假设，但也带来了窗宽、邻居数、样本规模等新的问题。这让我认识到，所谓“更复杂的方法”并不一定总是更好，关键还是看它是否适合当前数据。

另一个收获是对“最优分类”的理解更加具体了。最小错误率追求总体错误少，最小风险则强调错误代价小。实际任务中，如果不同错误的后果不同，就不能只看准确率一个指标。分类结果需要结合混淆矩阵、召回率和任务代价一起分析，这比单纯给出一个准确率更有意义。

3 实验二：非参数估计、Fisher 线性判别与留一法实验

3.1 模式识别理论知识

实验一主要从概率密度估计出发，在正态分布假设下建立参数化 Bayes 分类器。实验二继续使用男女身高体重数据，重点比较三类方法：非参数概率密度估计、直接设计线性分类器、以及留一法错误率估计。

3.1.1 非参数估计与 Parzen 窗 Bayes 分类器

参数估计方法需要先假设概率密度的形式，例如实验一中的二维高斯分布，然后估计均值和协方差。非参数估计则不强行假设类条件概率密度一定服从某种固定分布，而是直接利用训练样本本身估计密度。Parzen 窗就是典型的非参数密度估计方法。

对类别 ω_i ，设训练样本为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_i}$ ，使用高斯核 Parzen 窗估计类条件概率密度：

$$\hat{p}(\mathbf{x}|\omega_i) = \frac{1}{N_i} \sum_{k=1}^{N_i} \frac{1}{(2\pi)^{d/2} h^d} \exp \left[-\frac{1}{2} \left\| \frac{\mathbf{x} - \mathbf{x}_k}{h} \right\|^2 \right].$$

其中 $d = 2$ 表示使用身高和体重两个特征， h 是窗宽。本实验取 $h = 5.0$ 。窗宽越小，密度估计越依赖局部样本，边界更曲折，容易过拟合；窗宽越大，密度估计越平滑，可能忽略真实局部结构。

估计出 $\hat{p}(\mathbf{x}|\omega_F)$ 与 $\hat{p}(\mathbf{x}|\omega_M)$ 后，仍然使用 Bayes 判别函数：

$$g_i(\mathbf{x}) = \ln \hat{p}(\mathbf{x}|\omega_i) + \ln P(\omega_i).$$

若 $g_F(\mathbf{x}) > g_M(\mathbf{x})$ ，则判为女生，否则判为男生。本实验延续实验一中二维特征、等先验 $P(F) = P(M) = 0.5$ 的设置。

3.1.2 Fisher 线性判别

Fisher 线性判别是一种直接设计分类器的方法。它不显式估计概率密度，而是寻找一个投影方向 $\mathbf{w}$ ，把二维样本投影到一维轴上，使两类投影后的均值尽量分开，同时每一类内部尽量集中。

设女生和男生的均值向量分别为 $\boldsymbol{\mu}_F$ 、 $\boldsymbol{\mu}_M$ ，类内散度矩阵为

$$S_w = \sum_{\mathbf{x} \in F} (\mathbf{x} - \boldsymbol{\mu}_F)(\mathbf{x} - \boldsymbol{\mu}_F)^T + \sum_{\mathbf{x} \in M} (\mathbf{x} - \boldsymbol{\mu}_M)(\mathbf{x} - \boldsymbol{\mu}_M)^T.$$

Fisher 准则为

$$J(\mathbf{w}) = \frac{\mathbf{w}^T S_b \mathbf{w}}{\mathbf{w}^T S_w \mathbf{w}},$$

最优投影方向可写为

$$\mathbf{w} = S_w^{-1}(\boldsymbol{\mu}_M - \boldsymbol{\mu}_F).$$

本实验由训练集计算得到

$$\mathbf{w} = [0.002321, 0.001852]^T, \quad t = 0.500185.$$

分类时先计算投影值 $y = \mathbf{w}^T \mathbf{x}$ ，若 $y \geq t$ 则判为男生，否则判为女生。对应到二维平面中，Fisher 分类器的决策边界是一条直线：

$$\mathbf{w}^T \mathbf{x} = t.$$

3.1.3 多类情形下的 LDA 扩展

Fisher 线性判别最常见的推导形式是二分类。对于 Iris、MNIST、CIFAR 等多分类数据，可以将二分类 Fisher 判别推广为线性判别分析（Linear Discriminant Analysis, LDA）。设共有 C 个类别，第 i 类均值为 $\boldsymbol{\mu}_i$ ，总体均值为 $\boldsymbol{\mu}$ ，类内散度矩阵和类间散度矩阵分别为

$$S_w = \sum_{i=1}^C \sum_{\mathbf{x} \in \omega_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T,$$

$$S_b = \sum_{i=1}^C N_i (\boldsymbol{\mu}_i - \boldsymbol{\mu})(\boldsymbol{\mu}_i - \boldsymbol{\mu})^T.$$

LDA 的目标是在投影后增大类间散度、减小类内散度，可写为广义 Fisher 准则

$$J(W) = \frac{|W^T S_b W|}{|W^T S_w W|}.$$

当类别数为 2 时，LDA 退化为前述 Fisher 线性判别；当类别数大于 2 时，LDA 可以得到最多 $C - 1$ 个判别方向，并在这些线性判别函数上完成分类。本实验在扩展数据集上使用 LDA 作为 Fisher 方法的多类实现。

3.1.4 留一法错误率估计

留一法是一种交叉验证方法。对于 N 个训练样本，每次取出 1 个样本作为验证样本，用剩余 $N - 1$ 个样本重新训练分类器，再测试被取出的样本。重复 N 次后，错误率为

$$\epsilon_{LOO} = \frac{\text{留一过程中错分样本数}}{N}.$$

留一法的优点是几乎充分利用了训练数据，适合样本数量较少的情况；缺点是计算量较大，因为需要重复训练多次。本实验训练集共有 100 个样本，因此留一法需要对每个分类器重复训练和测试 100 次。

3.2 实际数据集与实验设置

本实验仍使用 FEMALE.TXT 和 MALE.TXT 作为训练集，共 100 个样本，其中女生 50 个、男生 50 个；使用 test1.txt 和 test2.txt 作为测试集。特征统一取二维向量

$$\mathbf{x} = [\text{身高}, \text{体重}]^T.$$

实验设置如下：

- 参数 Bayes 基线：二维高斯模型，完整协方差矩阵，等先验 $P(F) = P(M) = 0.5$ 。
- 非参数 Bayes：高斯核 Parzen 窗，窗宽 $h = 5.0$ ，等先验 $P(F) = P(M) = 0.5$ 。
- 线性分类器：Fisher 线性判别，使用训练集计算投影方向和阈值。
- 错误率估计：分别计算训练集错误率、test1 错误率、test2 错误率和留一法错误率。

为了进一步比较三类方法在不同数据复杂度下的表现，本实验还补充模拟高斯数据、UCI Iris、MNIST、CIFAR-10 和 CIFAR-100 数据集。扩展数据集的设置如下：

- 模拟高斯数据使用二维特征；参数、非参数和线性方法分别采用高斯 Bayes、Parzen 窗和 Fisher/LDA。
- UCI Iris 使用 4 维花萼、花瓣特征，先标准化后进行分类；为便于观察边界，图中将样本投影到 PCA 二维平面。
- MNIST、CIFAR-10 和 CIFAR-100 为高维图像数据，先将图像向量标准化并降到 PCA 64 维，再分别使用对角高斯 Bayes、 k 近邻和 LDA 分类。
- 图像数据训练样本数量较大，完整留一法计算量很高，因此在分层抽取的小训练子集上估计留一法错误率；测试错误率仍在独立测试集上计算。

3.3 程序框图与代码实现

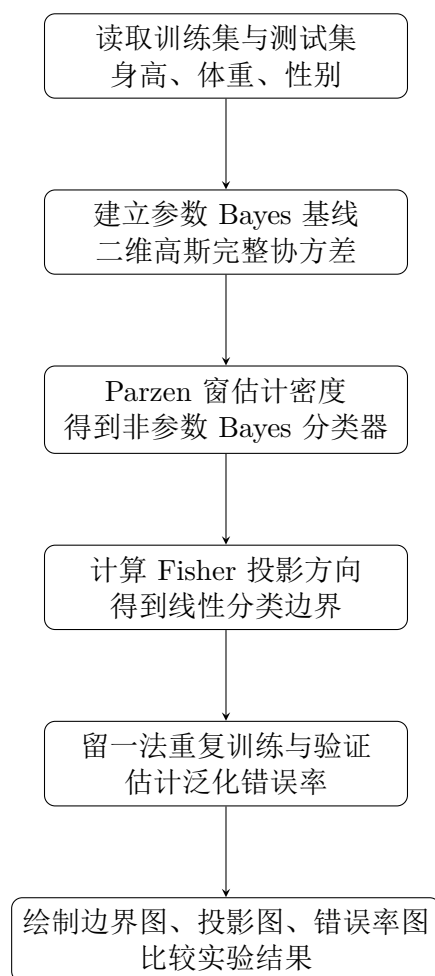


图 21: 实验二程序流程图

实验二程序位于 `scripts/experiment2_linear.py`。Parzen 窗部分先对每一类样本计算核密度，再代入 Bayes 判别函数。相关实现见附录代码 7。

Fisher 线性判别部分先计算两类均值和类内散度矩阵，再得到投影方向和阈值。相关实现见附录代码 8。

留一法部分每次屏蔽一个训练样本，用其余样本重新训练模型，再判断被屏蔽样本是否分类正确。相关实现见附录代码 9。

扩展数据集实验位于 `scripts/experiment2_extended_datasets.py`。其中低维数据使用 Parzen 窗作为非参数方法，图像数据使用 k 近邻，Fisher 多类扩展使用 LDA。相关实现见附录代码 10。

3.4 实验步骤与结果分析

3.4.1 步骤一：Parzen 窗 Bayes 与参数 Bayes 比较

首先选择实验一中的二维特征、等先验设置作为基线。参数方法使用二维高斯完整协方差模型，非参数方法使用高斯核 Parzen 窗估计类条件密度。两种方法的决策边界如图 22 所示。

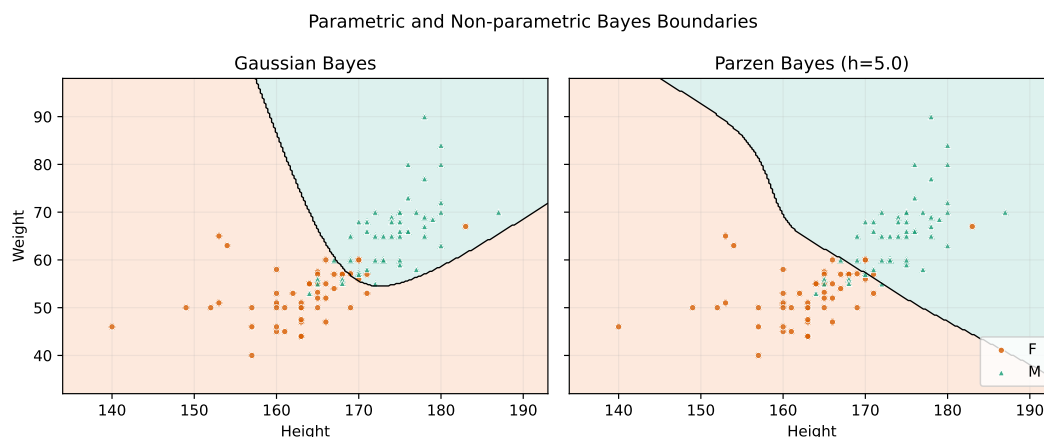


图 22: 参数高斯 Bayes 与 Parzen 窗 Bayes 决策边界比较

从图中可以看出，参数高斯 Bayes 的边界主要由两类整体均值和协方差决定，边界形状较规则；Parzen 窗 Bayes 的边界来自训练样本局部核密度叠加，因此更能反映边界附近样本的局部分布。实验中 Parzen 窗 Bayes 的 test2 错误率为 9.67%，低于参数高斯 Bayes 的 10.67%，说明非参数密度估计在该数据集上略有提升。

3.4.2 步骤二：Fisher 线性判别与 Bayes 分类器比较

接着使用身高和体重两个特征设计 Fisher 线性分类器，并将其与 Bayes 分类器画在同一平面上，如图 23 所示。

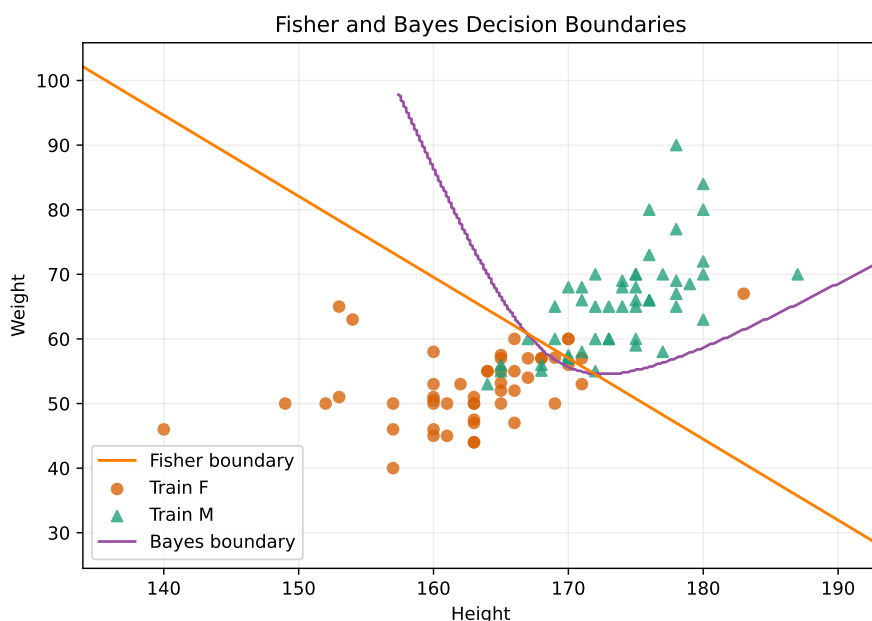


图 23: Fisher 线性判别与 Bayes 分类器决策边界比较

Fisher 分类器得到的是一条直线边界，形式简单，计算量小；Bayes 分类器基于类条件概率密度，边界可以是非线性的。由于男女身高体重数据在二维平面上大体呈左下与右上的分离趋势，线性边界已经能够取得较好效果。Fisher 方法在 test2 上错误率为 9.67%，与 Parzen 窗 Bayes 相同，略优于参数高斯 Bayes。

训练样本投影到 Fisher 方向后的分布如图 24 所示。投影后女生样本整体位于较低投影值区域，男生样本整体位于较高投影值区域，中间重叠区域对应分类中最容易出错的样本。

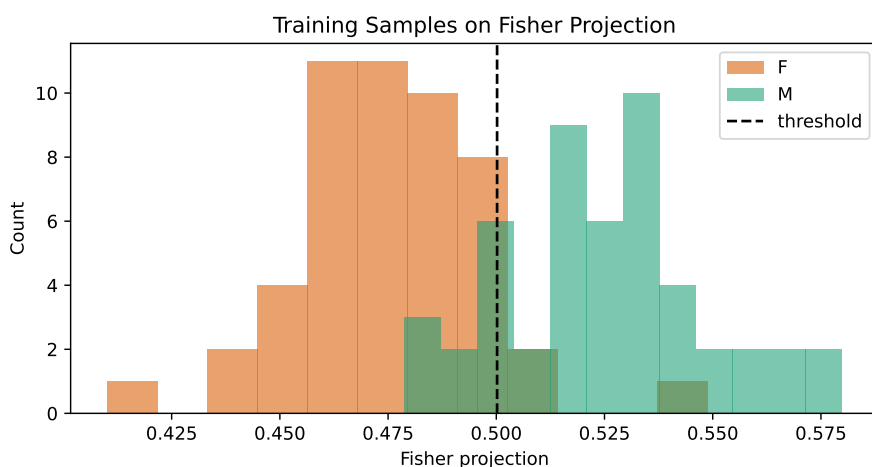


图 24: 训练样本在 Fisher 投影方向上的分布

3.4.3 步骤三：留一法错误率估计

最后对参数高斯 Bayes、Parzen 窗 Bayes 和 Fisher 线性判别分别进行留一法估计，并与训练错误率和 test2 错误率比较，如图 25 和表 11 所示。



图 25: 训练错误率、留一法错误率与 test2 错误率比较

表 11: 实验二分类器错误率汇总

方法	训练错误率	test1 错误率	test2 错误率	留一法错误率
参数高斯 Bayes	12.00%	2.86%	10.67%	12.00%
Parzen 窗 Bayes	13.00%	2.86%	9.67%	14.00%
Fisher 线性判别	12.00%	2.86%	9.67%	13.00%

留一法结果显示，参数高斯 Bayes、Parzen 窗 Bayes 和 Fisher 线性判别的留一法错误率分别为 12.00%、14.00% 和 13.00%。这些结果与训练错误率和 test2 错误率大体接近，说明留一法能够在训练样本较少时给出较合理的泛化误差估计。test1 错误率均为 2.86%，明显低于留一法和 test2 错误率，主要原因是 test1 样本量仅 35 个，偶然性更强；test2 样本量为 300 个，更能反映分类器在较大测试集上的表现。

3.5 扩展数据集实验

扩展实验保持“参数 Bayes 基线、非参数方法、Fisher/LDA 线性方法”三类分类器的比较方式。低维数据保留二维或四维连续特征，图像数据使用 PCA 特征，使传统分类器能够在可控计算量下完成训练和测试。

3.5.1 模拟高斯数据

模拟数据的真实分布为高斯分布，参数 Bayes 与数据生成假设匹配，Fisher/LDA 则只使用线性判别面。图 26 给出了二类 and 三类模拟数据上的 LDA 与 Parzen 决策区

域。二类数据中，两类样本主要由一条近似线性边界分开，因此 LDA 和 Parzen 的结果接近；三类数据中，类别边界出现交会区域，Parzen 边界对局部样本分布的响应更明显。

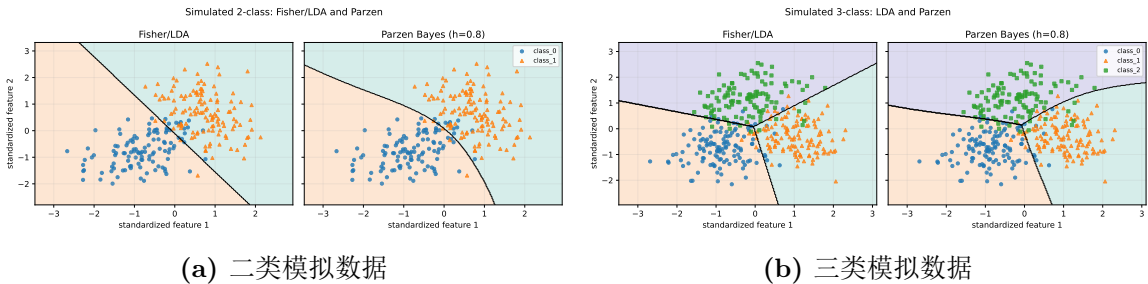


图 26: 模拟数据上的 Fisher/LDA 与 Parzen 决策边界

图 27 给出了 LDA 在模拟数据测试集上的混淆矩阵。二类模拟数据 LDA 测试错误率为 3.75%，三类模拟数据为 10.00%。类别数增加后，边界附近样本增多，错分主要集中在相邻类别之间。

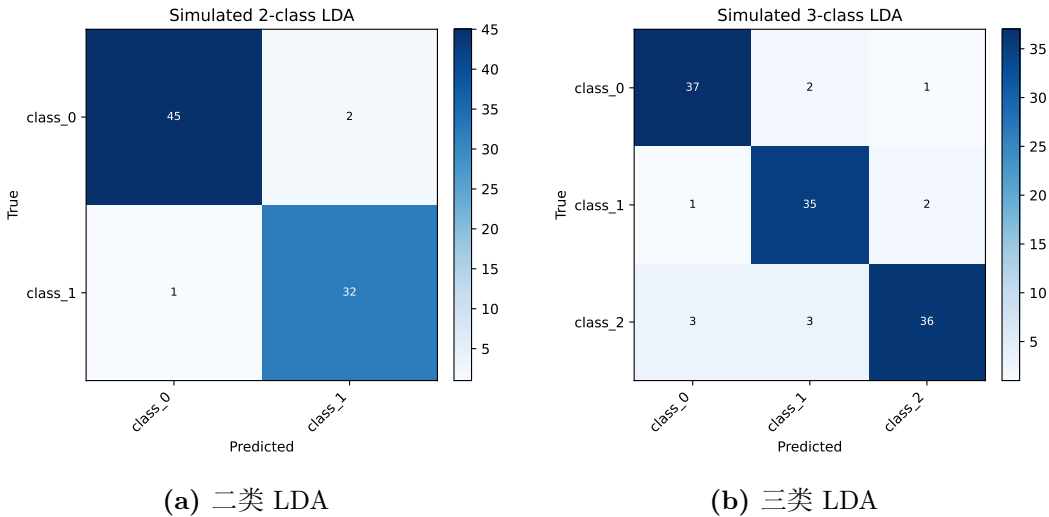


图 27: 模拟数据 Fisher/LDA 测试集混淆矩阵

3.5.2 UCI Iris 数据

Iris 数据具有 3 个类别和 4 个连续特征。图 28 将样本投影到 PCA 二维平面后绘制 LDA 与 k 近邻边界。Iris 中一类样本与另外两类分离明显，后两类之间存在少量重叠，因此 LDA 在测试集上达到 0.00% 错误率，Parzen 窗错误率为 3.33%。这说明当类别结构接近线性可分时，直接设计线性判别函数可以得到非常稳定的结果。

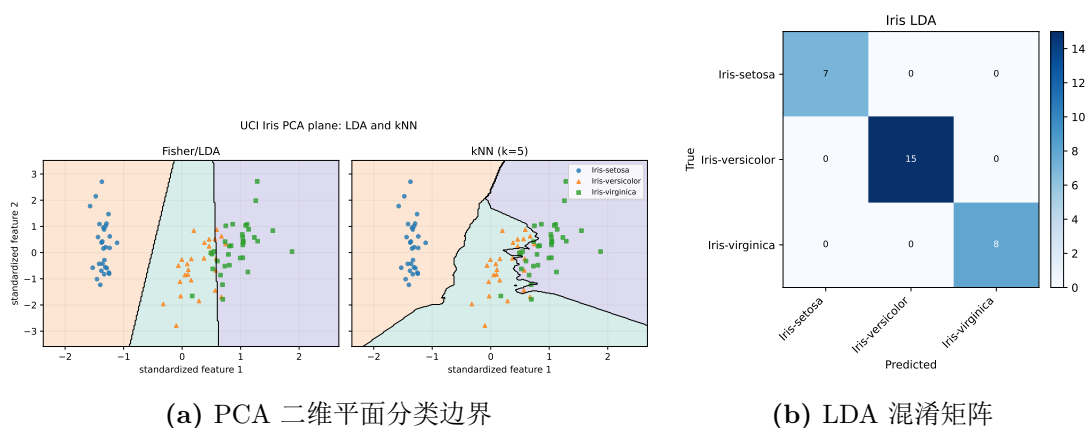


图 28: UCI Iris 数据上的 LDA 与非参数方法

3.5.3 MNIST 与 CIFAR 图像数据

MNIST 手写数字的主要变化与数字形状相关，PCA 64 维特征已经保留了较多有用结构。图 29 中，LDA 测试错误率为 19.55%， k 近邻错误率为 10.45%。这说明 MNIST 中同一数字的局部相似性较强，非参数近邻投票比单一线性判别函数更适合该数据。

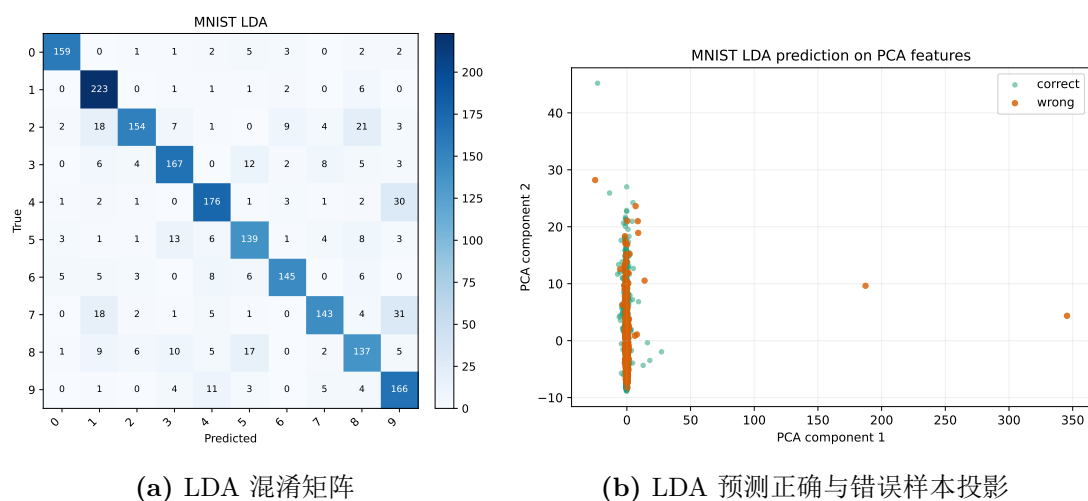


图 29: MNIST 数据上的 Fisher/LDA 分类结果

图 30 展示了 MNIST 测试样本中 LDA 的部分预测结果。图像上方标注中，T 表示真实类别，P 表示预测类别；绿色表示预测正确，红色表示预测错误。可以看到，错误样本多出现在笔画形状相近或书写方式不典型的数字之间，例如 4、7、9 等类别之间更容易混淆。



图 30: MNIST 数据 LDA 部分测试样本预测结果

CIFAR-10 和 CIFAR-100 的图像包含物体姿态、背景、颜色和纹理变化，PCA 特征不能直接形成清晰的类别边界。图 31 中，CIFAR-10 上 LDA 错误率为 63.67%，仍优于对角高斯 Bayes 的 66.28% 和 k 近邻的 70.44%；CIFAR-100 类别数更多、类间差异更细，LDA 错误率为 85.47%。这些结果说明，在线性特征空间中，传统分类器能够给出可解释基线，但对复杂自然图像的表达能力有限。

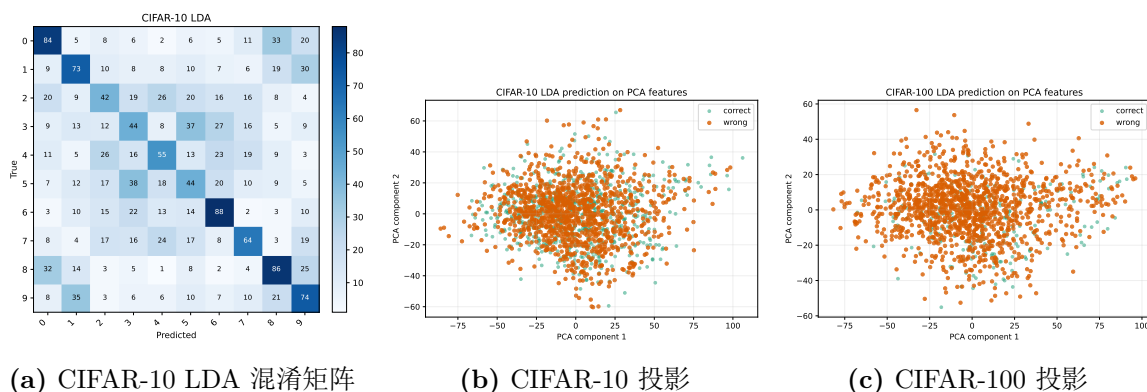


图 31: CIFAR 图像数据上的 Fisher/LDA 分类结果

图 32 给出了 CIFAR-10 和 CIFAR-100 的部分测试图像预测结果。与 MNIST 相比，CIFAR 图像中的目标尺寸、背景和纹理变化更复杂，许多样本即使人眼观察也需

要依赖整体语义信息，而 PCA 后的像素特征主要保留总体方差，难以稳定表达物体类别，因此红色错误样本比例明显更高。

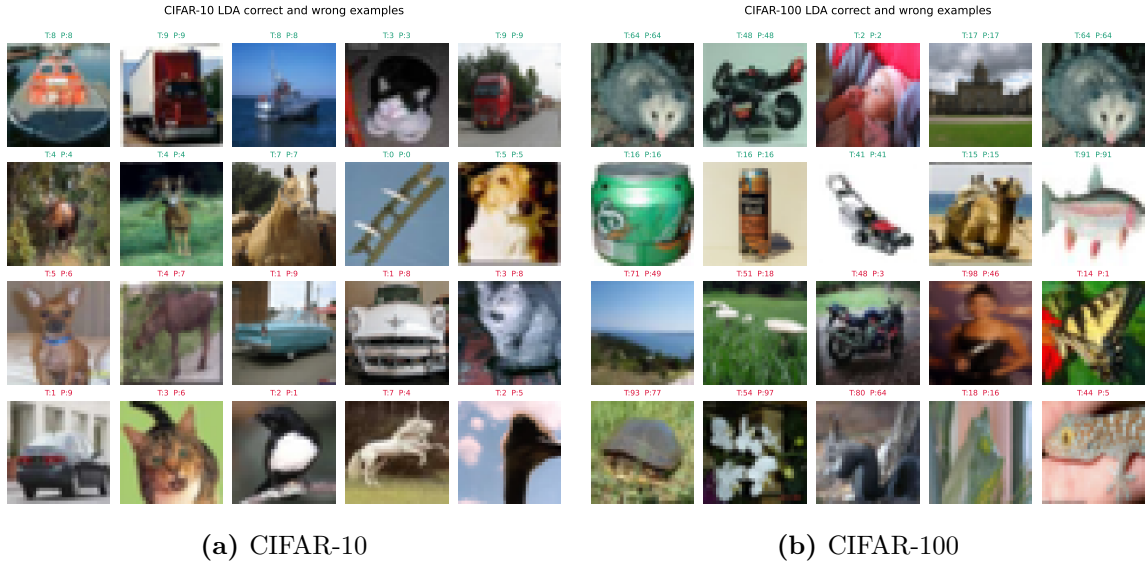


图 32: CIFAR 数据 LDA 部分测试样本预测结果

表 12: 实验二扩展数据集错误率汇总

数据集	特征与非参数设置	Gaussian 错误率	非参数错 误率	LDA 错 误率	留一样本
模拟二类高斯	标准化原始特征; Parzen $h=0.8$	2.50%	3.75%	3.75%	235
模拟三类高斯	标准化原始特征; Parzen $h=0.8$	10.00%	10.83%	10.00%	240
UCI Iris	标准化原始特征; Parzen $h=0.8$	0.00%	3.33%	0.00%	86
MNIST	PCA 64 维, 累计方差 67.07%; kNN $k=5$	37.80%	10.45%	19.55%	150
CIFAR-10	PCA 64 维, 累计方差 86.49%; kNN $k=7$	66.28%	70.44%	63.67%	120
CIFAR-100	PCA 64 维, 累计方差 87.62%; kNN $k=5$	85.13%	89.00%	85.47%	300

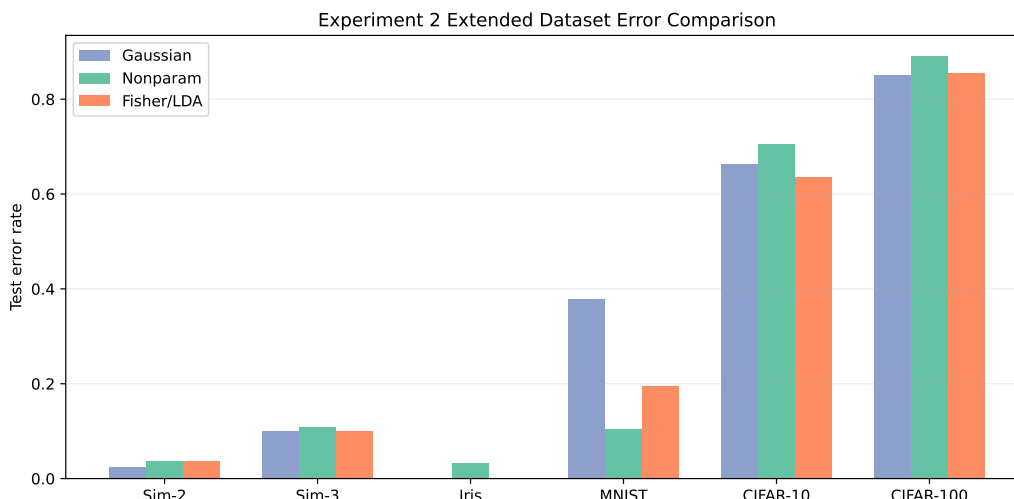


图 33: 扩展数据集上三类分类器测试错误率比较

表 12 和图 33 表明，低维连续数据上 Gaussian Bayes 和 Fisher/LDA 通常更稳定；图像数据上，MNIST 的局部邻域结构明显， k 近邻优于 LDA 和对角高斯 Bayes，而 CIFAR 数据对特征表达要求更高，简单 PCA 特征下三类传统分类器的错误率都较高。

3.6 综合实验结果与分析总结

- 参数高斯 Bayes 假设类条件概率密度服从二维正态分布，模型结构清晰，训练样本较少时较稳定，但边界形状受分布假设限制。
- Parzen 窗 Bayes 不预设固定分布形状，可以利用样本局部结构，test2 错误率由参数 Bayes 的 10.67% 降至 9.67%，但对窗宽 h 较敏感。
- Fisher 线性判别不估计概率密度，而是直接寻找最佳投影方向；在本数据集上，简单线性边界已能取得与 Parzen 窗 Bayes 相同的 test2 错误率。
- 留一法通过重复训练和验证估计泛化误差，适合小样本数据。图像数据上由于样本规模较大，本实验采用分层子集进行留一法估计。
- 扩展数据集说明，Fisher/LDA 在类间均值差异明显、类内分布较集中的数据上表现较好；非参数方法在局部相似性明显的数据上更有优势，但对样本量、距离度量和参数选择更敏感。

男女身高体重数据具有较明显的线性可分趋势，因此 Fisher 线性判别表现很好；同时，边界附近仍存在局部重叠，Parzen 窗等非参数方法能通过局部密度估计对部分样本做出更灵活的判断。参数 Bayes、非参数 Bayes 和 Fisher 线性判别在该数据集上的性能接近，分类结果主要受边界重叠样本影响。扩展到 MNIST、CIFAR 等图像数据后，分类器差异更多来自特征表达是否充分，PCA 特征虽然降低了维数，但并不能完全表示复杂图像的类别结构。

3.7 体会

这次实验让我更明显地感受到，分类器的形式简单并不一定代表效果差。对于男女身高体重和 Iris 这类结构清晰的数据，线性判别已经能够抓住主要差异；而在 MNIST、CIFAR 这类图像数据上，分类困难并不只来自分类器本身，更多来自特征是否真正表达了类别信息。留一法也提醒我，训练集上的一次结果并不充分，尤其在样本较少时，需要用交叉验证去估计模型面对新样本时可能出现的误差。

非参数方法的结果让我认识到，模型少一些分布假设并不等于使用起来更简单。Parzen 窗需要选择窗宽， k 近邻需要选择邻居数和距离度量；这些参数改变后，边界平滑程度和局部敏感性都会变化。因此在报告分类效果时，除了给出错误率，还应说明参数取值和特征预处理方式。

Fisher/LDA 的优点是计算过程清楚、判别方向容易解释，适合用来分析数据中主要的类间差异。它的局限也比较明显：如果类别边界本身不是线性的，或者原始特征没有把类别结构展开，再好的线性判别面也只能给出有限的改进。这一点在 CIFAR 数据上表现得很明显。

4 实验三：K-L 变换特征提取实验

4.1 模式识别理论知识

K-L 变换 (Karhunen-Loeve Transform) 在模式识别中通常对应主成分分析 PCA。它是一种无监督特征提取方法：不使用样本类别标签，只根据样本整体分布寻找一组正交方向，使样本投影后的方差尽可能集中在前几个方向上。

设原始样本为二维特征向量

$$\mathbf{x} = [\text{身高}, \text{体重}]^T,$$

样本总体均值为 $\boldsymbol{\mu}$ ，中心化样本为 $\mathbf{x}_i - \boldsymbol{\mu}$ 。总体协方差矩阵为

$$C = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T.$$

对协方差矩阵做特征值分解：

$$C\mathbf{u}_k = \lambda_k \mathbf{u}_k.$$

其中 λ_k 表示沿方向 $\mathbf{u}_k$ 的方差大小。最大特征值对应的特征向量 $\mathbf{u}_1$ 就是第一主成分方向，即样本整体变化最明显的方向。将样本投影到该方向上可得到一维新特征：

$$y = \mathbf{u}_1^T \mathbf{x}.$$

若选取前 m 个特征值对应的特征向量组成投影矩阵

$$U_m = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m],$$

则样本可变换为 m 维 K-L/PCA 特征：

$$\mathbf{z} = U_m^T (\mathbf{x} - \boldsymbol{\mu}).$$

前 m 个主成分保留的累计方差比例为

$$\eta_m = \frac{\sum_{k=1}^m \lambda_k}{\sum_{k=1}^d \lambda_k}.$$

该比例用于衡量降维后保留了多少总体变化信息。后续扩展数据集实验中，低维数据直接观察前两个主成分，图像数据选取前 64 个主成分，并比较第一主成分、多个主成分和监督 LDA 投影的分类效果。

需要注意的是，K-L/PCA 的目标是“保留总体方差”，不是“最大化类别可分性”。

如果类别差异刚好也是总体方差的主要来源，PCA 主方向会有较好的分类能力；如果最大方差来自与类别无关的因素，PCA 方向不一定适合分类。

为了利用类别信息，本实验还使用类平均向量提取判别信息。女生和男生均值分别为 μ_F 与 μ_M ，类均值差方向为

$$\mathbf{w}_{mean} = \frac{\mu_M - \mu_F}{\|\mu_M - \mu_F\|}.$$

该方向直接指向两类中心分离最大的方向，因此是一种简单的有监督判别投影。分类阈值取两类均值投影的中点：

$$t = \frac{1}{2} (\mathbf{w}_{mean}^T \mu_F + \mathbf{w}_{mean}^T \mu_M).$$

若 $\mathbf{w}_{mean}^T \mathbf{x} \geq t$ ，则判为男生，否则判为女生。

实验二中的 Fisher 线性判别同样利用类别信息，但它不仅考虑两类均值差，还考虑类内散度：

$$\mathbf{w}_{Fisher} = S_w^{-1}(\mu_M - \mu_F).$$

因此，PCA 主方向、类均值差方向和 Fisher 方向的关系可以概括为：PCA 关注总体方差，类均值差关注类别中心距离，Fisher 同时关注类间分离和类内紧凑。

4.2 实际数据集与参数计算

本实验使用 FEMALE.TXT 和 MALE.TXT 中的 100 个训练样本，不使用 test1.txt 和 test2.txt 参与 K-L 方向计算，只用于分类性能测试。将男女训练样本合并后，计算得到总体均值：

$$\mu = [168.380, 59.049]^T.$$

总体协方差矩阵为

$$C = \begin{bmatrix} 63.036 & 55.041 \\ 55.041 & 87.155 \end{bmatrix}.$$

其特征值为

$$\lambda_1 = 131.442, \quad \lambda_2 = 18.748.$$

对应方差贡献率为

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = 87.52\%, \quad \frac{\lambda_2}{\lambda_1 + \lambda_2} = 12.48\%.$$

第一主成分方向为

$$\mathbf{u}_1 = [0.626888, 0.779109]^T,$$

第二主成分方向为

$$\mathbf{u}_2 = [-0.779109, 0.626888]^T.$$

类均值差方向为

$$\mathbf{w}_{mean} = [0.651392, 0.758742]^T, \quad t = 154.484247.$$

可以看到，第一主成分方向与类均值差方向非常接近，说明该数据集中总体方差最大的方向基本也是男女类别均值分离的方向。

扩展实验继续使用实验一、实验二中已经准备好的模拟高斯数据、UCI Iris、MNIST、CIFAR-10 和 CIFAR-100 数据集。为了突出 K-L/PCA 的特征提取作用，各数据集统一按如下方式处理：

- 连续低维数据先进行标准化，再计算 PCA 主方向，并比较 PC1、前若干主成分和 LDA 监督投影的分类效果。
- 图像数据先展开为像素向量并标准化，再计算前 64 个 PCA 主成分，使用最近质心分类器在 PCA 空间中分类。
- 同时绘制 PCA 二维投影和 LDA 二维投影，用于比较“最大方差方向”和“类别判别方向”的差异。
- MNIST、CIFAR-10 和 CIFAR-100 额外绘制部分测试图像预测结果，观察 PCA 特征下哪些样本容易分类错误。

4.3 程序框图与代码实现

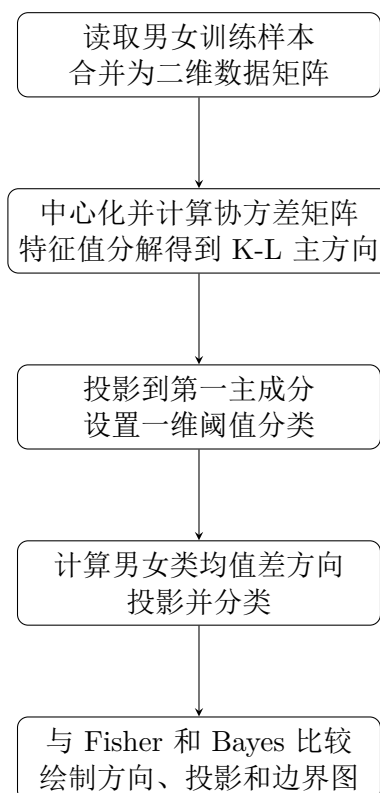


图 34: 实验三程序流程图

实验三程序位于 `scripts/experiment3_kl.py`。K-L/PCA 的核心代码如下：相关实现见附录代码 11。

一维投影分类器的实现如下。它既可以用于第一主成分方向，也可以用于类均值差方向。相关实现见附录代码 12。

类均值差方向直接利用类别标签计算：相关实现见附录代码 13。

扩展数据集实验位于 `scripts/experiment3_extended_datasets.py`，主要实现 PCA 特征提取、累计方差统计、最近质心分类、LDA 对比和图像预测样例绘制。相关实现见附录代码 14。

4.4 实验步骤与结果分析

4.4.1 步骤一：不考虑类别信息的 K-L/PCA 变换

首先将男女训练样本合并，不使用类别标签，计算总体协方差矩阵及其特征向量。图 35 中 PC1 表示第一主成分方向，PC2 表示第二主成分方向。可以看到 PC1 沿着身高和体重共同增大的方向，这说明训练样本的主要变化来自“更高更重”和“更矮更轻”的差异。

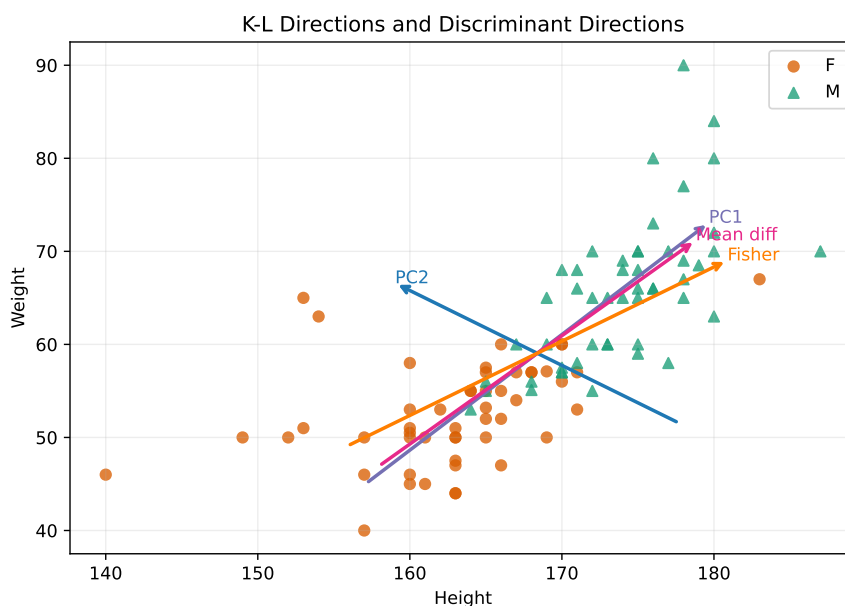


图 35: K-L 主方向、类均值差方向与 Fisher 方向比较

将样本投影到第一主成分方向后，得到图 36。女生投影值整体偏小，男生投影值整体偏大，中间存在少量重叠，因此可以用一维阈值完成分类。

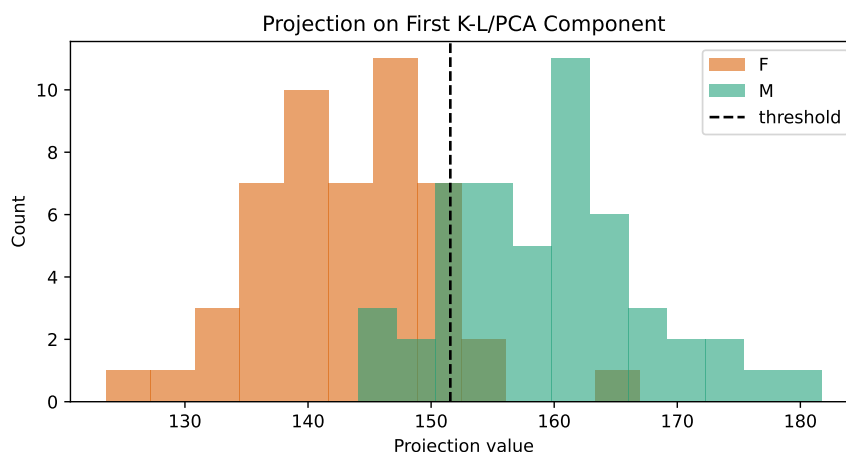


图 36: 训练样本在 K-L 第一主成分方向上的投影分布

K-L/PCA 第一主成分分类在训练集上的准确率为 86.00%，在 `test1.txt` 上为 100.00%，在 `test2.txt` 上为 89.67%。这说明虽然 PCA 没有使用类别信息，但由于数据主要方差方向与性别差异方向接近，第一主成分仍具有较强判别能力。

4.4.2 步骤二：利用类平均向量提取判别信息

第二步利用类别标签，计算男生均值与女生均值的差向量，并将其作为投影方向。该方向直接连接两类中心，因此比 PCA 更具有明确的判别含义。投影结果如图 37 所示。

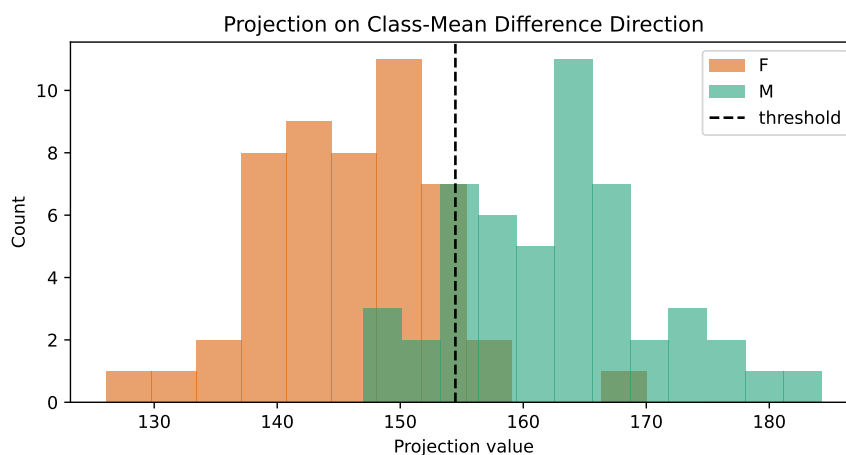


图 37: 训练样本在类均值差方向上的投影分布

本实验中，类均值差方向的训练准确率、test1 准确率和 test2 准确率分别为 86.00%、100.00% 和 89.67%，与 K-L/PCA 第一主成分完全相同。这是因为两者方向非常接近，得到的一维投影排序和阈值划分几乎一致。

4.4.3 步骤三：与 Fisher、Bayes 分类方法比较

为了比较不同方法的分类面，将 K-L/PCA 主方向、类均值差方向、Fisher 线性判别边界和二维高斯 Bayes 边界画在同一平面上，如图 38 所示。

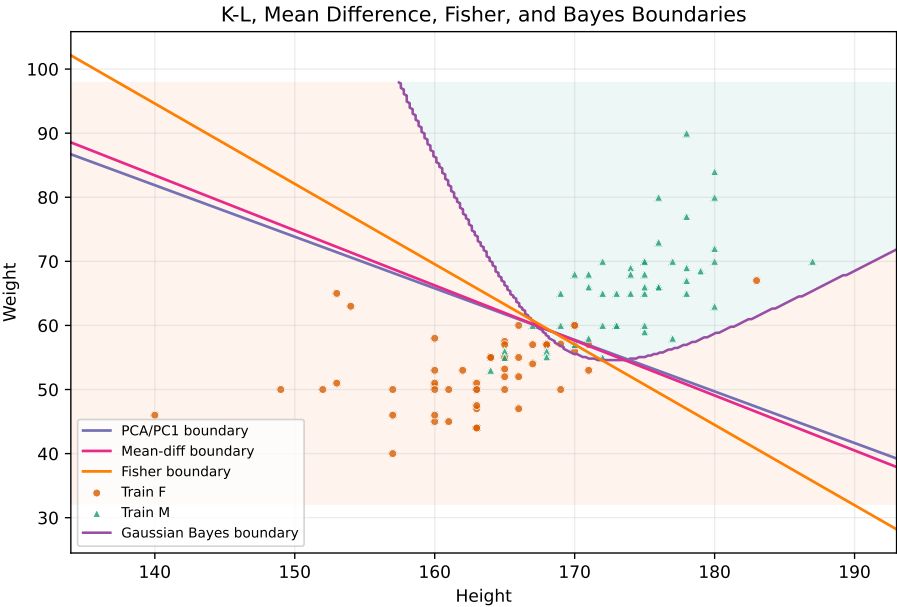


图 38: K-L、类均值差、Fisher 与 Bayes 分类边界比较

从图中可以看出，K-L/PCA 与类均值差方向对应的线性边界几乎重合，Fisher 边界略有旋转，这是因为 Fisher 还考虑了类内散度；Bayes 边界由概率密度估计得到，可以呈现非线性形状。各方法的准确率对比如表 13 所示。

表 13: 实验三 K-L 变换与相关分类方法结果汇总

方法	训练准确率	test1 准确率	test2 准确率
K-L/PCA 主方向	86.00%	100.00%	89.67%
类均值差方向	86.00%	100.00%	89.67%
Fisher 线性判别	88.00%	97.14%	90.33%
二维高斯 Bayes	88.00%	97.14%	89.33%

4.5 扩展数据集实验

扩展实验用于观察 K-L/PCA 在不同类型数据上的特征提取效果。评价时不只看分类准确率，还观察第一主成分的方差贡献率、前若干主成分的累计方差贡献率，以及 PCA 投影与 LDA 投影的差异。PCA 不使用类别标签，LDA 使用类别标签，因此二者的对比可以反映“保留总体变化”和“增强类别可分性”之间的区别。

4.5.1 模拟高斯数据

图 39 给出了模拟二类 and 三类高斯数据的 PCA 与 LDA 投影结果。二类数据中，第一主成分解释 77.04% 的方差，PC1 分类准确率达到 97.50%，说明数据的最大方差方向与类别分离方向基本一致。三类数据中，第一主成分只解释 54.05% 的方差，PC1 分类准确率降为 64.17%；使用前两个 PCA 主成分后准确率提高到 89.17%，接近 LDA 的 90.00%。

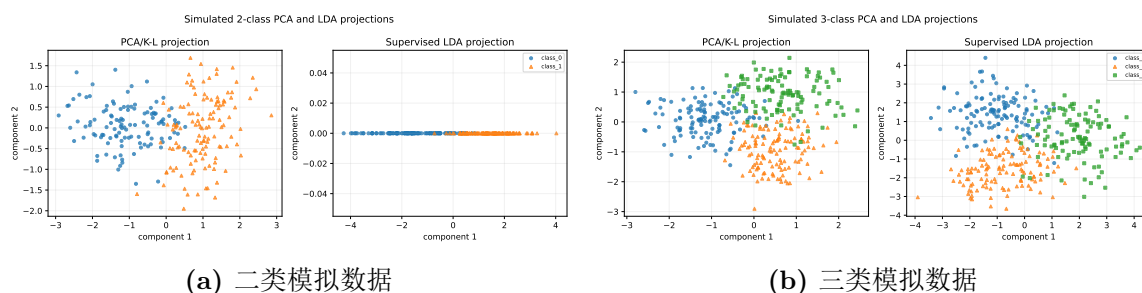


图 39: 模拟数据的 PCA/K-L 投影与 LDA 投影比较

二类模拟数据的 PC1 投影分布如图 40 所示。两类样本在 PC1 上分离明显，因此一维主成分已经可以完成较好分类。这与男女身高体重数据中的现象相似：当最大方差方向同时也是类间分离方向时，K-L/PCA 能得到较好的分类特征。

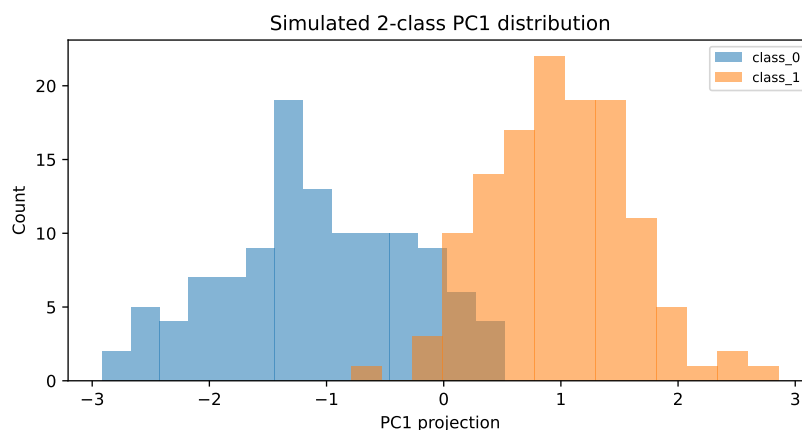


图 40: 模拟二类高斯数据在第一主成分上的投影分布

4.5.2 UCI Iris 数据

Iris 数据的第一主成分解释 72.93% 的方差，使用 PC1 分类准确率为 93.33%。图 41 中 PCA 平面已经能较好地地区分 Setosa 类，但另外两类在 PCA 空间中仍有一定接近；LDA 投影由于使用类别标签，类间分离更明显，测试准确率达到 100.00%。这说明 PCA 的最大方差方向不一定完全等同于最优判别方向。

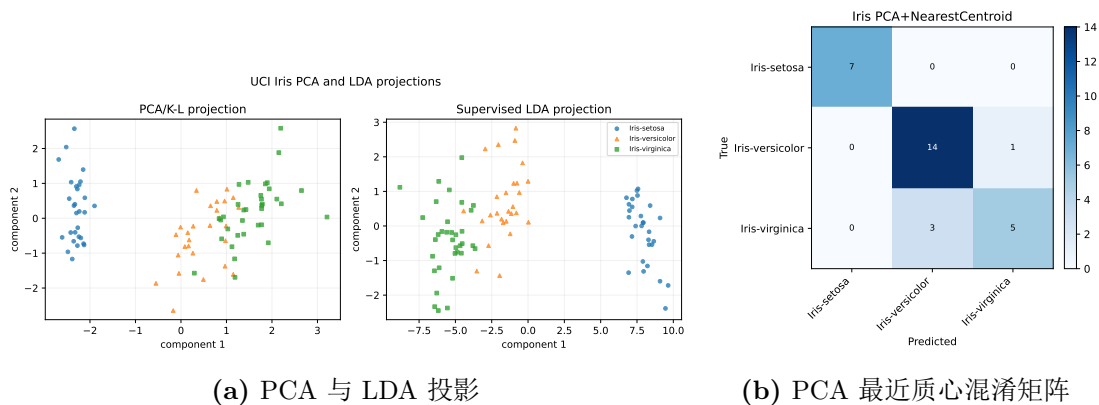


图 41: UCI Iris 数据的 K-L/PCA 特征提取结果

4.5.3 MNIST 与 CIFAR 图像数据

图像数据维数高，单个主成分只能描述整体像素变化的一部分。图 42 给出了 MNIST、CIFAR-10 和 CIFAR-100 前 64 个主成分的累计方差曲线。MNIST 前 64 个主成分累计解释 67.28% 的方差，CIFAR-10 和 CIFAR-100 分别为 86.36% 和 87.58%。CIFAR 数据的累计方差较高并不代表分类更容易，因为这些方差中包含背景、颜色和纹理等与类别不完全一致的变化。

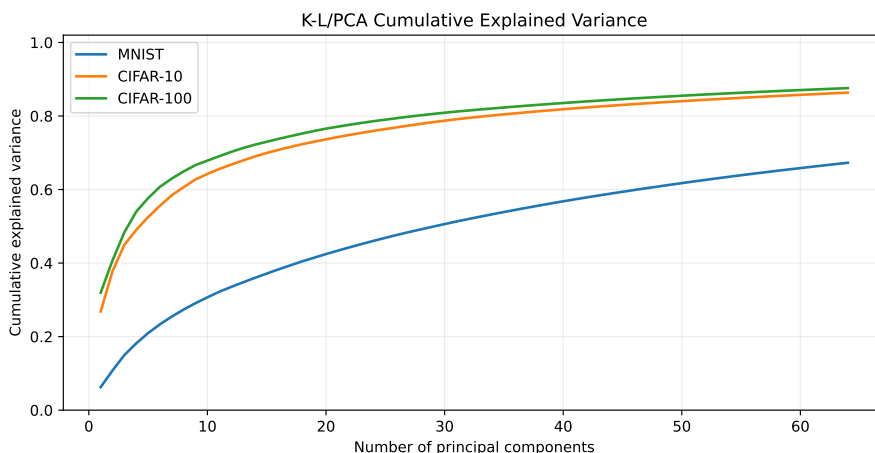


图 42: 图像数据 K-L/PCA 累计方差贡献率

MNIST 的 PCA 与 LDA 投影如图 43 所示。PC1 只解释 6.26% 的方差，单独使用 PC1 分类准确率为 27.80%；使用 64 维 PCA 特征和最近质心分类器后，准确率提高到 74.30%；LDA 监督投影达到 82.25%。这说明手写数字识别不能只依赖一个最大方差方向，多个主成分共同保留的笔画结构才具有更强判别能力。

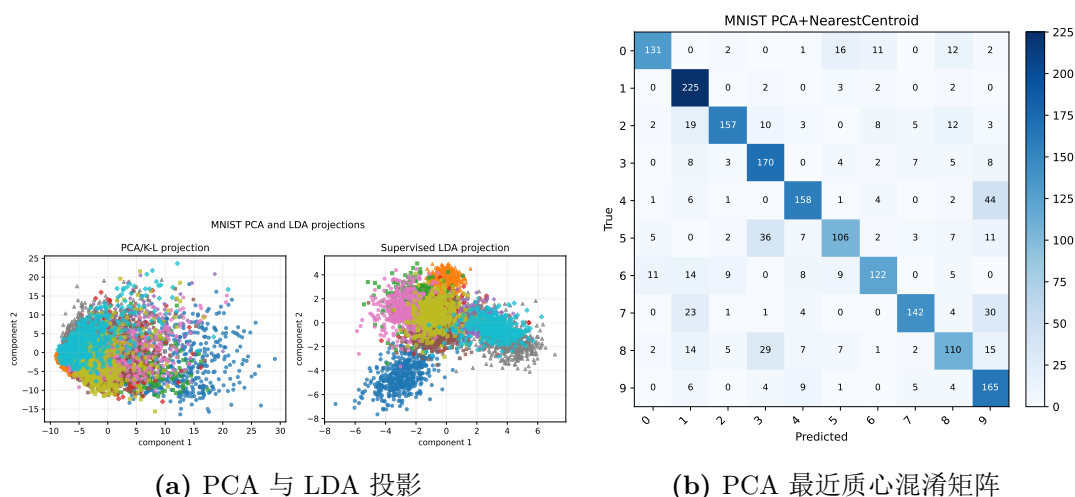


图 43: MNIST 数据的 K-L/PCA 特征提取结果

图 44 给出了 MNIST 的部分测试样本预测结果。绿色标题表示预测正确，红色标题表示预测错误。错误样本多出现在书写形态接近或笔画不完整的数字之间，说明 PCA 像素特征对局部笔画差异的表达仍有限。

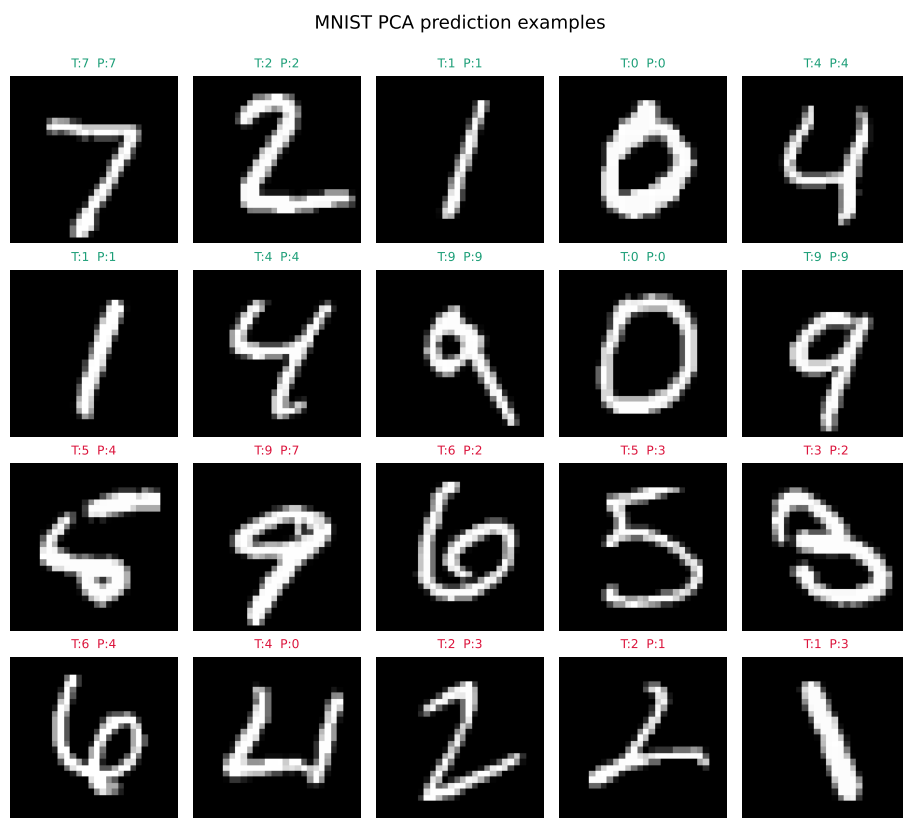


图 44: MNIST 数据 PCA 特征分类的部分测试样本预测结果

CIFAR 图像数据的 PCA 与 LDA 投影如图 45 所示。与 MNIST 相比，CIFAR 的二维投影中正确和错误样本混杂更明显，说明自然图像的类别结构很难仅由像素 PCA

特征展开。CIFAR-10 在 64 维 PCA 特征上的最近质心准确率为 26.72%，CIFAR-100 为 11.13%，明显低于低维数据和 MNIST。

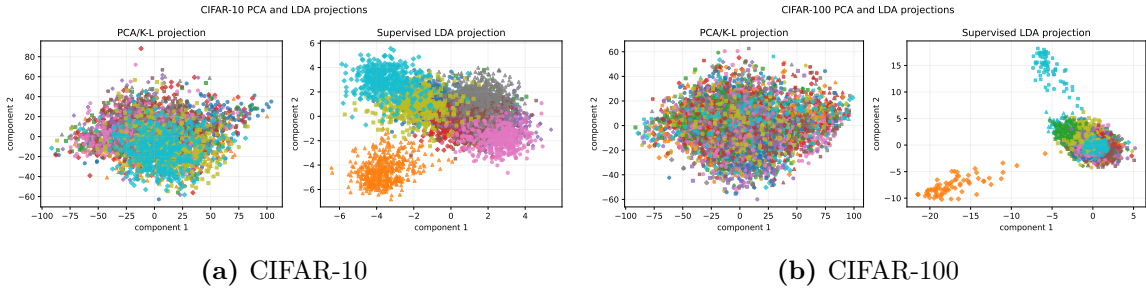


图 45: CIFAR 数据的 PCA/K-L 投影与 LDA 投影比较

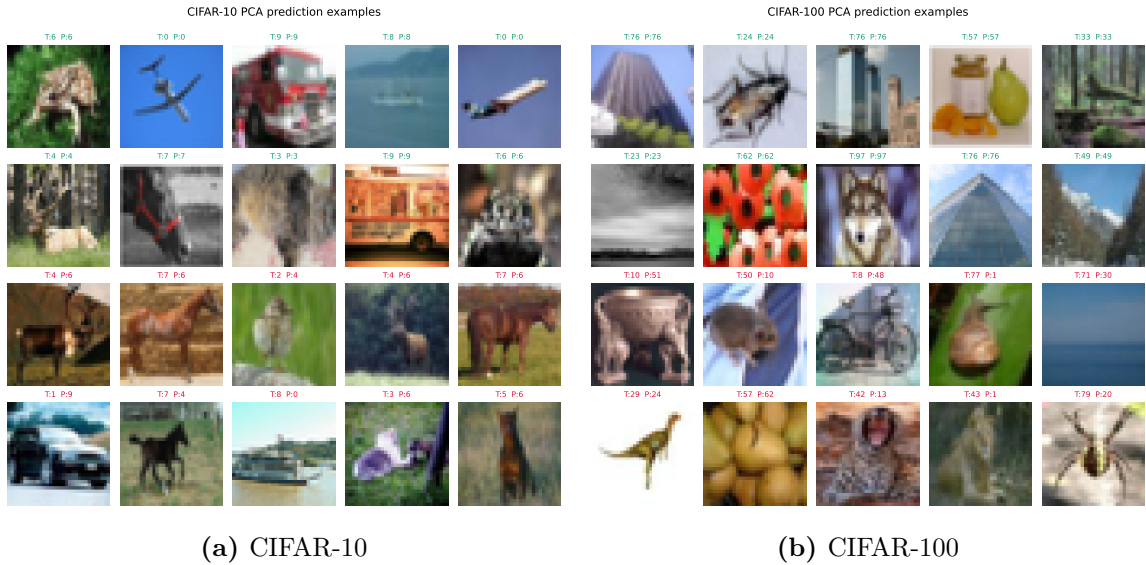


图 46: CIFAR 数据 PCA 特征分类的部分测试样本预测结果

表 14: 实验三扩展数据集 K-L/PCA 特征提取结果

数据集	PCA 维数	PC1 方差	PCA 累计方差	PC1 准确率	PCA 分类准确率	LDA 准确率
模拟二类高斯	2	77.04%	100.00%	97.50%	97.50%	96.25%
模拟三类高斯	2	54.05%	100.00%	64.17%	89.17%	90.00%
UCI Iris	4	72.93%	100.00%	93.33%	86.67%	100.00%
MNIST	64	6.26%	67.28%	27.80%	74.30%	82.25%
CIFAR-10	64	26.83%	86.36%	15.89%	26.72%	32.56%
CIFAR-100	64	31.94%	87.58%	2.00%	11.13%	11.33%

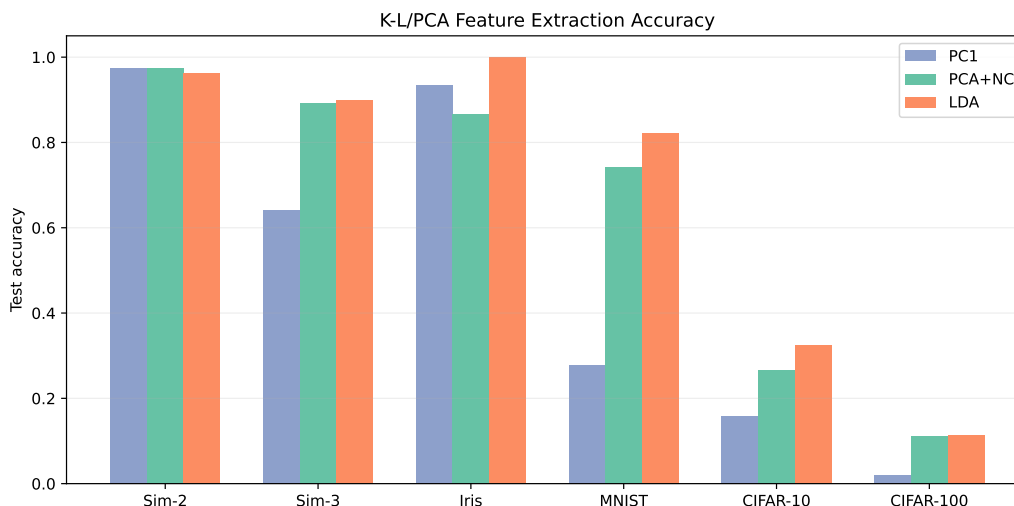


图 47: 扩展数据集上 PC1、PCA 特征和 LDA 的分类准确率比较

4.6 综合实验结果与分析总结

综合结果可以看出：

- K-L/PCA 第一主成分保留了 87.52% 的总体方差信息，并在 test2 上达到 89.67% 的准确率，说明它提取到的主要变化方向具有较强性别判别能力。
- 类均值差方向与 PCA 第一主成分方向接近，因此分类结果完全相同。这说明本数据集中“总体变化最大方向”和“男女均值分离方向”基本一致。
- Fisher 线性判别在 test2 上达到 90.33%，略高于 PCA 和类均值差方向，因为 Fisher 同时考虑了类间分离和类内散度。
- 二维高斯 Bayes 在 test2 上为 89.33%，与 PCA 方法接近，但它基于概率密度估计，分类边界可以是非线性的。
- 扩展数据集表明，PC1 分类效果强烈依赖“最大方差方向是否与类别差异一致”。模拟二类高斯和男女数据中该条件基本满足，因此 PC1 效果较好；三类数据和图像数据中，仅使用 PC1 容易丢失重要判别信息。
- 图像数据上，64 维 PCA 特征能明显优于单个主成分，但仍弱于使用类别信息的 LDA 或局部邻域方法。PCA 保留的是总体方差，保留下来的变化不一定是类别相关变化。

K-L 变换适合作为降维和特征提取工具，但它本身不是专门为分类设计的。对于身高体重、模拟二类高斯等数据，PCA 主方向刚好接近判别方向，因此分类效果较好；在 MNIST 和 CIFAR 等更复杂数据中，最大方差可能来自书写风格、背景、颜色、姿态等因素，此时仅靠 PCA 主方向分类不够理想。PCA 更适合作为特征压缩和可视化工具，若目标是提高分类性能，通常还需要结合 Fisher/LDA、Bayes、近邻方法或更有效的特征表达。

4.7 体会

本实验让我更清楚地区分了无监督特征提取和监督判别分析。K-L/PCA 不使用类别标签，得到的是最能表示数据整体变化的方向；类均值差方向和 Fisher/LDA 使用类别标签，更关注样本是否容易被分开。男女身高体重数据中三种方向非常接近，说明身高和体重的主要变化本身就与性别差异相关。

扩展数据集进一步说明，方差大并不一定等于分类信息多。MNIST 中单个主成分分类效果较弱，但多个主成分组合后能保留较多笔画结构；CIFAR 中即使累计方差较高，分类准确率仍然不高，因为许多方差来自背景、颜色和纹理变化，而不是类别本身。之后分析特征提取结果时，不能只看累计方差比例，还要结合类别分布、投影图和分类结果一起判断。

5 实验四：基于 K-L 变换的应用实验

5.1 模式识别理论知识

5.1.1 人脸图像的向量表示

本实验按照实验要求完成 K-L 变换在人脸识别中的应用。人脸识别与前面男女身高体重分类的主要区别在于特征维数更高：一张灰度人脸图像可以看成由像素组成的矩阵，将矩阵按行展开后即可得到一个高维向量。若图像大小为 64×64 ，则每张人脸图像对应

$$\mathbf{x} = [x_1, x_2, \dots, x_{4096}]^T,$$

其中每个分量表示一个像素位置的灰度值。人脸图像具有较强的整体结构：眼睛、鼻子、嘴巴和脸部轮廓的位置相对稳定，因此适合用 K-L 变换从大量人脸样本中提取公共变化方向。

直接在原始像素空间中进行分类存在两个问题。第一，维数较高，样本之间的距离容易受噪声、光照和表情变化影响；第二，相邻像素之间存在相关性，很多维度包含重复信息。因此需要先进行特征提取，把原始人脸图像转换为更紧凑的低维表示。

5.1.2 K-L 变换与特征脸

K-L 变换在实际计算中通常表现为 PCA。设训练图像向量为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ ，先计算训练集均值

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i,$$

再对每个样本做中心化 $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \boldsymbol{\mu}$ 。训练集协方差矩阵可写为

$$C = \frac{1}{N} \sum_{i=1}^N \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^T.$$

对协方差矩阵进行特征值分解：

$$C \mathbf{u}_j = \lambda_j \mathbf{u}_j, \quad \lambda_1 \geq \lambda_2 \geq \dots.$$

其中 $\mathbf{u}_j$ 是第 j 个主成分方向， λ_j 表示该方向上的方差大小。取前 k 个特征向量组成矩阵 $U_k = [\mathbf{u}_1, \dots, \mathbf{u}_k]$ ，则图像在 K-L 特征空间中的表示为

$$\mathbf{z} = U_k^T (\mathbf{x} - \boldsymbol{\mu}),$$

维数从原来的 4096 降为 k 。将每个主成分方向 $\mathbf{u}_j$ 重新排列成 64×64 图像后，就得到“特征脸”。特征脸不是某一个人的真实照片，而是训练集中人脸灰度变化的基方向。

不同人脸可以看成平均脸加上若干特征脸的线性组合：

$$\mathbf{x} \approx \boldsymbol{\mu} + U_k \mathbf{z}.$$

因此，识别时不再比较原始图像，而是比较每张人脸在特征脸空间中的系数向量 $\mathbf{z}$ 。

5.1.3 低维特征空间中的分类

完成 K-L 变换后，分类器不再直接处理原始图像，而是在 PCA 投影后的低维空间中工作。本实验采用最近质心、1 近邻和线性 SVM 三种分类器作为对比。

最近质心分类器先计算每一类在 PCA 空间中的均值向量：

$$\mathbf{m}_c = \frac{1}{N_c} \sum_{y_i=c} \mathbf{z}_i,$$

然后把测试样本分到距离最近的类别：

$$\hat{y} = \arg \min_c \|\mathbf{z} - \mathbf{m}_c\|_2.$$

该方法参数少、计算快，但它默认每个人的脸样本可以用一个中心表示。

1 近邻分类器不显式估计类别分布，而是保存训练样本的 PCA 特征。对测试样本 $\mathbf{z}$ ，寻找距离最近的训练样本：

$$i^* = \arg \min_i \|\mathbf{z} - \mathbf{z}_i\|_2, \quad \hat{y} = y_{i^*}.$$

该方法能够保留训练样本的局部信息，但对噪声和维数选择比较敏感。

线性 SVM 在特征脸空间中学习分类超平面。对多类人脸识别，程序采用一对多或一对一的多类扩展形式，由分类器内部完成。SVM 不只比较距离，还利用类别标签寻找具有间隔的判别边界，因此通常比单纯最近邻更稳定。

5.2 人脸数据集说明

实验使用 Olivetti Faces 人脸数据集。该数据集形式与 ORL/AT&T 风格的人脸库接近，包含 40 个人，每人 10 张灰度人脸图像，共 400 张，图像大小为 64×64 。每个人的脸图像包含轻微姿态、表情和光照变化。实验按人员划分训练集和测试集：每人前 6 张作为训练样本，后 4 张作为测试样本，因此训练集 240 张，测试集 160 张。这种划分保证每个身份都同时出现在训练集和测试集中，符合人脸识别中的闭集识别设置。

表 15: Olivetti Faces 人脸数据集基本信息

数据集	人数	每人图像数	图像大小
Olivetti Faces	40	10	64×64

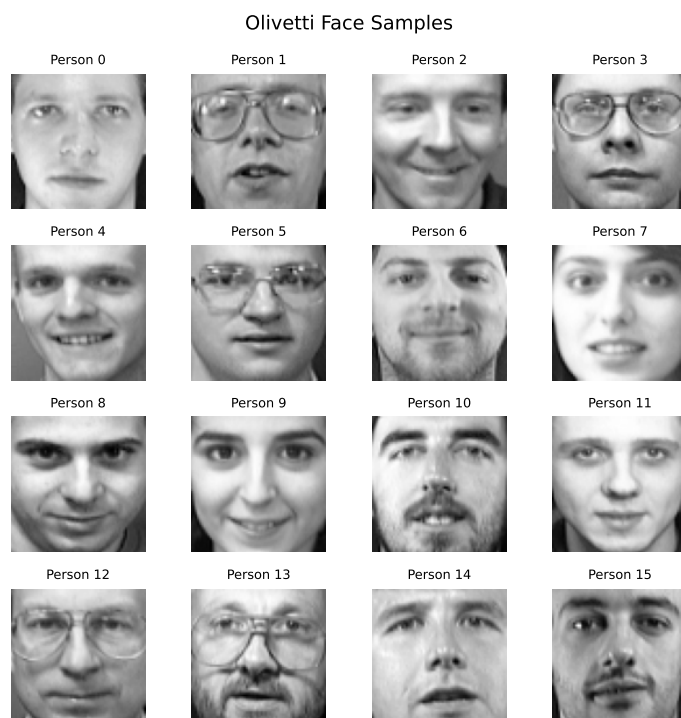


图 48: Olivetti Faces 数据集样本示例

图 48 展示了部分人脸样本。与飞机图像相比，人脸图像背景较简单、目标位置较统一，面部结构也更稳定，因此更适合作为 K-L 变换和特征脸方法的教学实验数据。

5.3 实验设置与程序实现

实验流程为：读取人脸数据，按身份构造训练集和测试集，将 64×64 图像展开为 4096 维向量，对训练集进行标准化，再用 K-L/PCA 提取特征脸。测试图像使用训练集得到的均值、标准差和 PCA 投影矩阵进行同样变换，避免测试信息泄漏。

主成分数分别取 $k = 10, 20, 40, 60, 80, 120, 160$ 。其中 $k = 120$ 作为主要展示设置，累计方差贡献率为 96.90% 左右。分类器包括 Eigenfaces+ 最近质心、Eigenfaces+1NN 和 Eigenfaces+SVM。实验程序位于 `scripts/experiment4_eigenfaces.py`，数据划分、特征脸提取和分类评价核心实现见附录代码 15 和 16。

5.4 程序框图

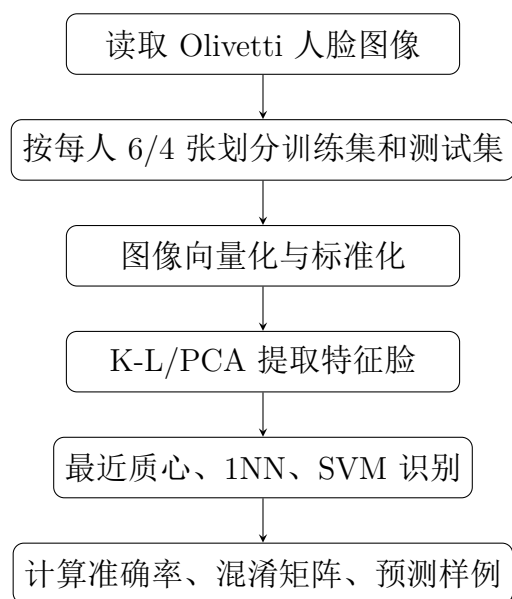


图 49: 基于 K-L 变换的特征脸识别流程

5.5 实验步骤与结果分析

5.5.1 平均脸与特征脸提取结果

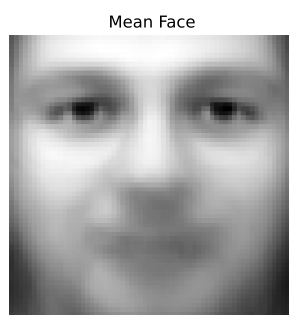


图 50: 训练集平均脸

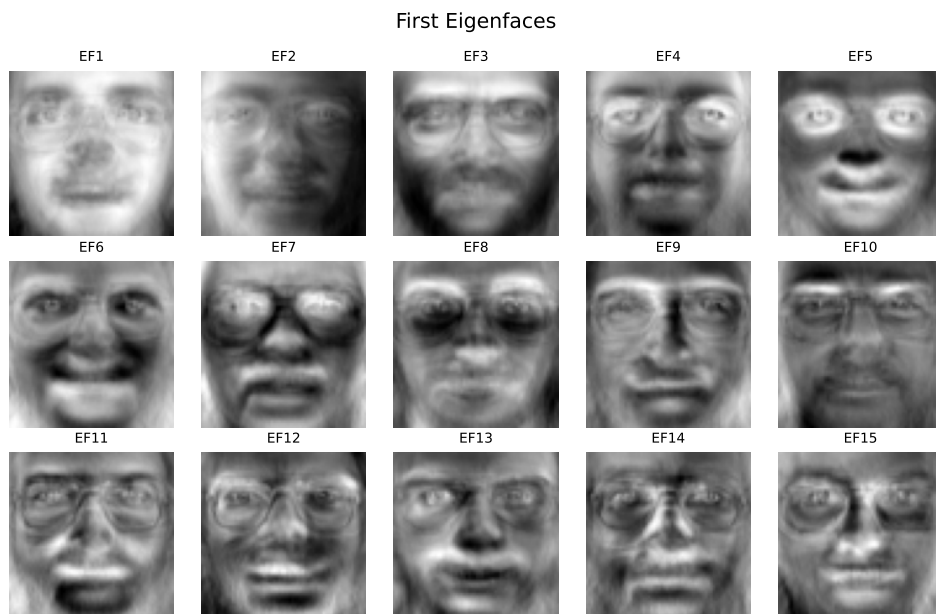


图 51: K-L/PCA 提取的前 15 个特征脸

图 50 为训练样本的平均脸，可以看到人脸的大致轮廓、眼睛和嘴部位置。图 51 为前 15 个特征脸。特征脸通常表现为正负灰度变化组成的面部模式，而不是某个人的真实照片。前几个特征脸主要描述脸部整体亮度、左右光照、头部轮廓和五官位置变化；后续特征脸包含更细的局部纹理。测试人脸被投影到这些特征脸方向后，就得到用于分类的低维系数。

5.5.2 特征脸数量与识别性能

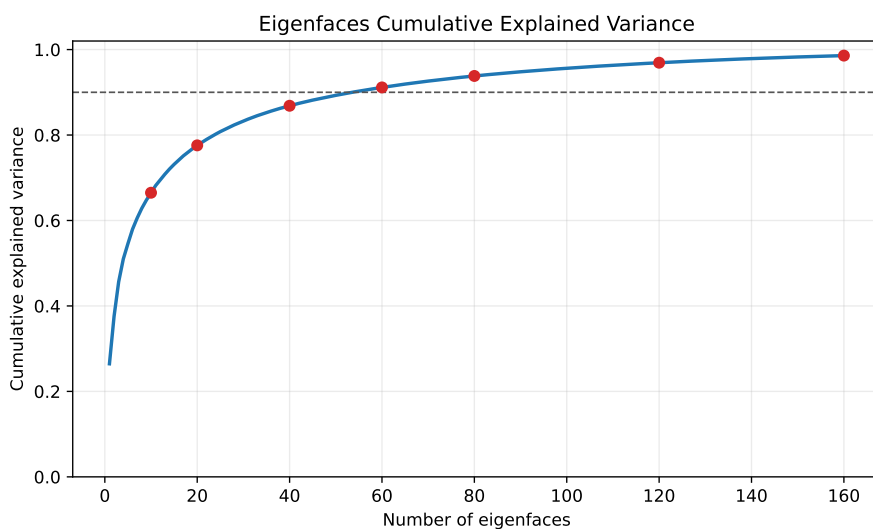


图 52: 特征脸数量与累计方差贡献率

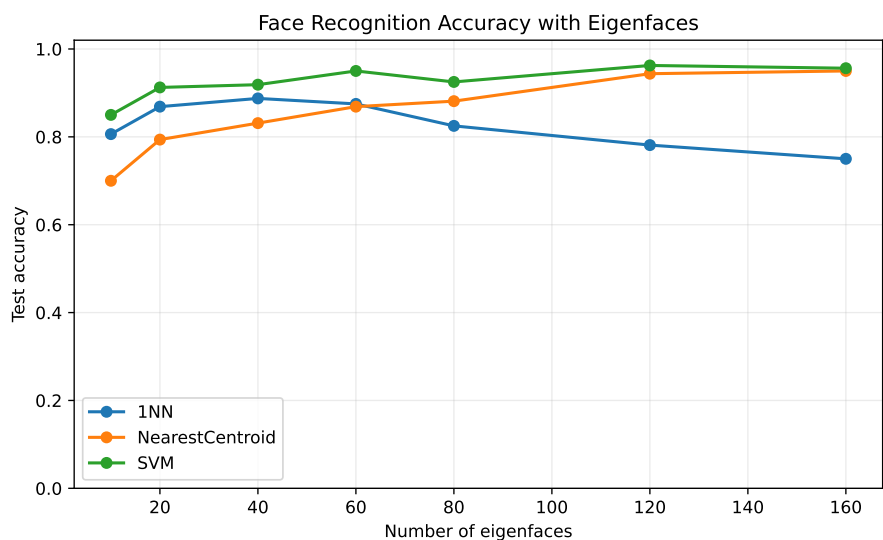


图 53: 不同特征脸数量下的人脸识别准确率

表 16: 不同特征脸数量下的人脸识别结果

主成分数	累计方差	最近质心	1NN	SVM
10	66.49%	70.00%	80.62%	85.00%
20	77.58%	79.38%	86.88%	91.25%
40	86.85%	83.12%	88.75%	91.88%
60	91.14%	86.88%	87.50%	95.00%
80	93.82%	88.12%	82.50%	92.50%
120	96.92%	94.38%	78.12%	96.25%
160	98.59%	95.00%	75.00%	95.62%

图 52 显示，前 10 个特征脸已经保留 66.49% 的训练集方差，前 60 个特征脸保留 91.14%，前 120 个特征脸保留 96.90% 左右。人脸图像具有较强相关结构，因此 K-L 变换可以用远少于 4096 的维数表示主要变化。

表 16 和图 53 显示，特征脸数量对识别性能有明显影响。特征脸数量从 10 增加到 60 时，三种分类器总体上都有提升；继续增加到 120 后，SVM 测试准确率达到 96.25%，最近质心达到 94.38%。1NN 在特征脸数量较多时反而下降，说明高维低样本情况下，距离分类器容易受到细小噪声维度影响。

表 17: 120 个特征脸下的人脸识别结果

方法	主成分数	训练准确率	测试准确率
Eigenfaces+NearestCentroid	120	100.00%	94.38%
Eigenfaces+1NN	120	100.00%	78.12%
Eigenfaces+SVM	120	100.00%	96.25%

5.5.3 特征脸空间分布与混淆情况

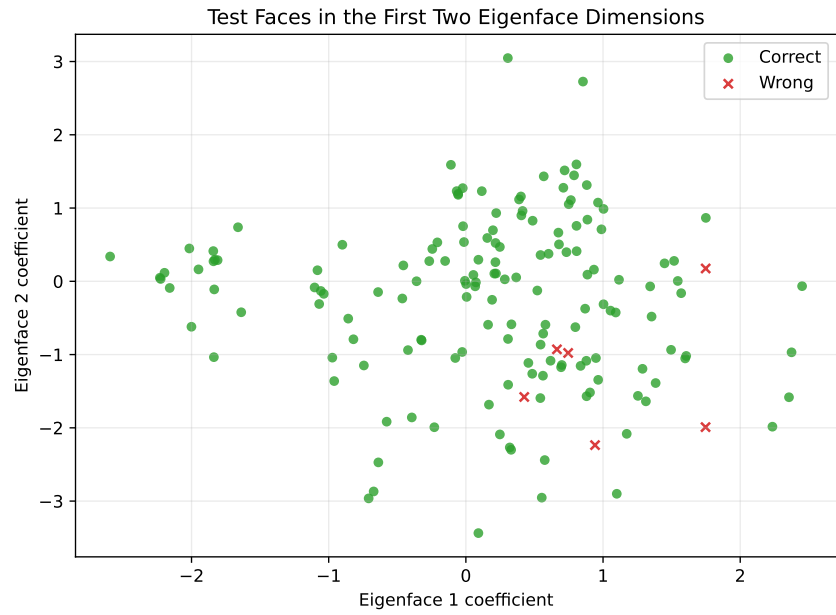


图 54: 测试人脸在前两个特征脸系数维度上的预测分布

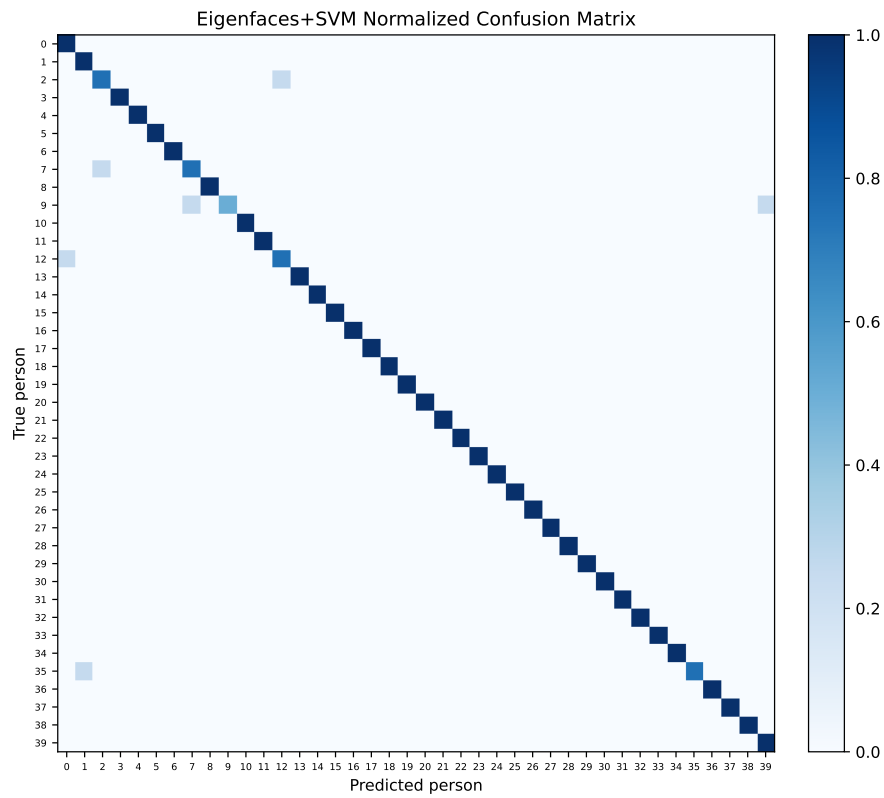


图 55: Eigenfaces+SVM 在人脸测试集上的归一化混淆矩阵

图 54 只显示前两个特征脸系数，因此不同身份仍有重叠；但预测错误样本数量明显少于飞机补充实验。图 55 中大部分能量集中在对角线附近，说明多数人的测试图像

被正确识别。少量错误通常发生在面部姿态、光照或表情差异较大的样本上。

5.5.4 预测样例分析

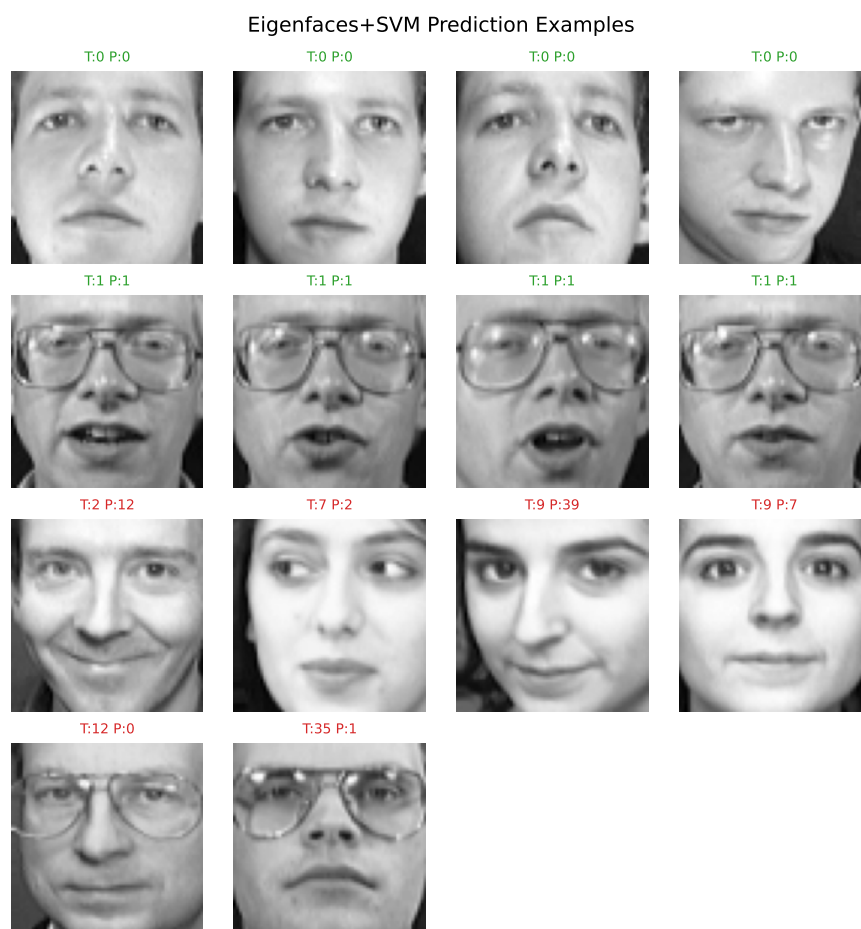


图 56: Eigenfaces+SVM 人脸预测正确与错误样例

图 56 给出了部分测试样本预测结果。绿色标题为识别正确样本，红色标题为识别错误样本。正确样本通常保持较典型的正面或近正面人脸结构；错误样本往往存在表情、头部角度或阴影变化，使其在特征脸空间中更接近其他人的训练样本。

5.6 补充实验：飞机图像数据集上的 K-L 变换

为比较 K-L 变换在不同图像任务中的表现，保留仓库中的飞机分类数据集作为补充实验。飞机图像数据集位于 `experiment/plane_dataset_4_1/`，包含 16 类飞机，训练集 2530 张，测试集 636 张。实验将飞机图像灰度化并缩放为 32×32 ，再展开为 1024 维向量，使用 PCA+ 最近质心和 PCA+1NN 分类器进行识别。

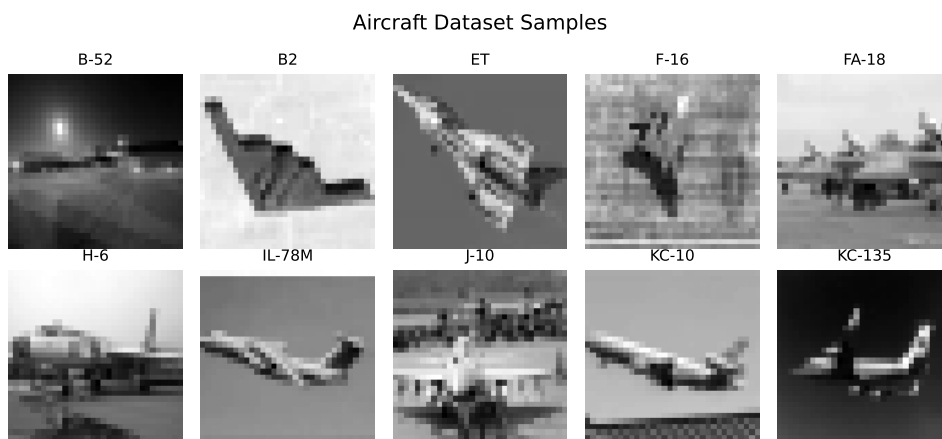


图 57: 飞机分类补充数据集样本示例

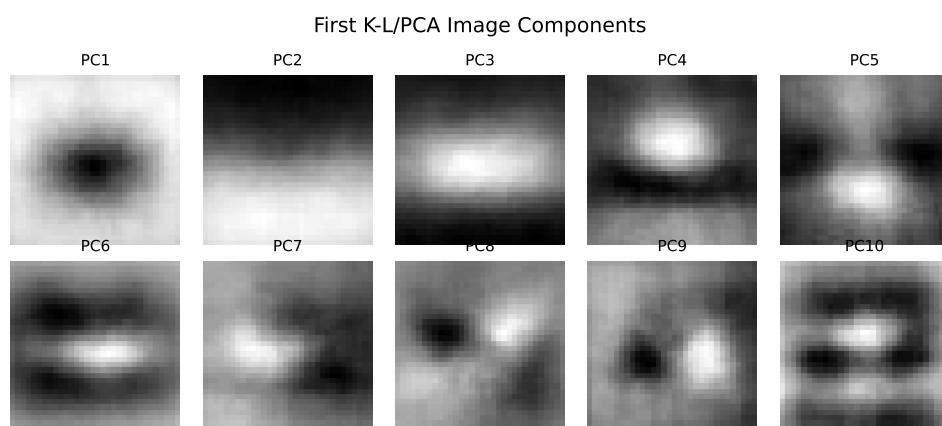


图 58: 飞机图像补充实验中提取的前 10 个主成分

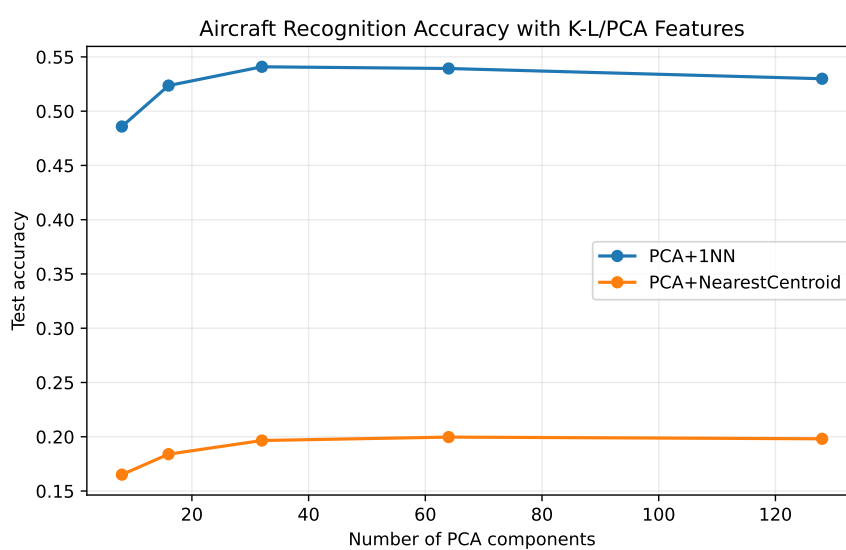


图 59: 飞机图像补充实验中不同主成分数下的测试准确率

表 18: 飞机图像补充实验结果

主成分数	累计方差	最近质心测试准确率	1NN 测试准确率
8	71.08%	16.51%	48.58%
16	78.51%	18.40%	52.36%
32	84.46%	19.65%	54.09%
64	89.89%	19.97%	53.93%
128	94.27%	19.81%	52.99%

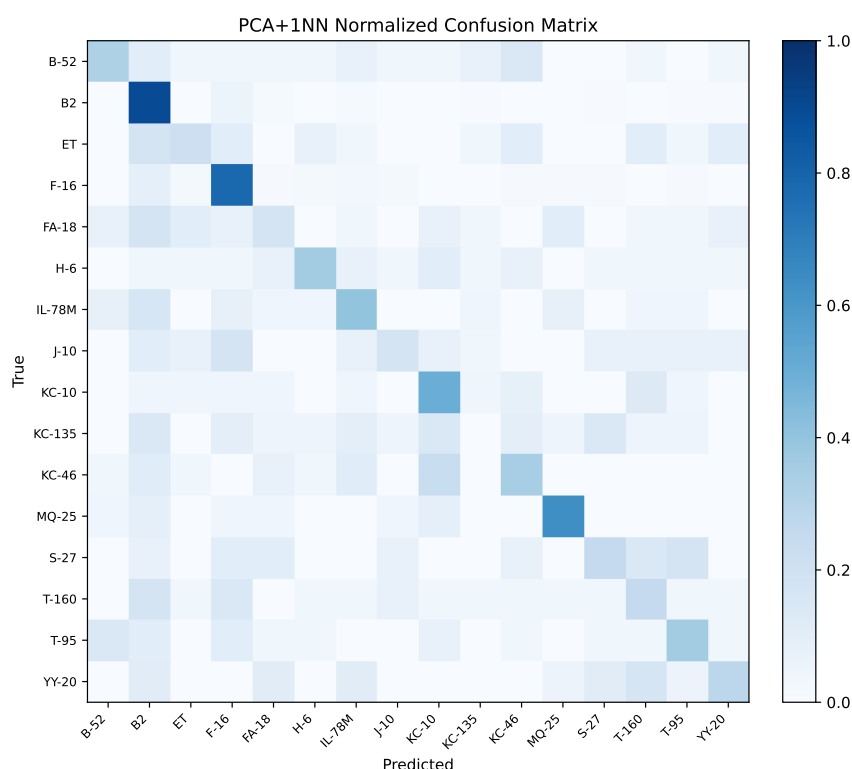


图 60: 飞机图像补充实验 PCA+1NN 归一化混淆矩阵

飞机补充实验中，64 个主成分保留约 89.88% 的方差信息，但 PCA+1NN 测试准确率为 53.93%，最近质心分类器约为 19.97%。与人脸数据相比，飞机图像背景复杂、目标姿态和尺度变化更大，类别数量也更多且样本不均衡。PCA 提取到的高方差方向中包含大量背景、亮度和拍摄角度变化，不一定是区分类别的关键结构，因此识别率低于特征脸实验。

5.7 实验结果综合分析

人脸主实验中，120 个特征脸将原始 4096 维像素压缩到 120 维，累计方差贡献率约为 96.90%。在该设置下，最近质心、1NN 和 SVM 的测试准确率分别为 94.38%、78.12% 和 96.25%。SVM 准确率最高，说明在特征脸空间中进一步学习判别边界，比

单纯距离比较更稳定。最近质心也取得较好结果，是因为 Olivetti 人脸图像姿态较统一，每个人的样本在特征脸空间中具有较紧凑分布。

飞机补充实验与人脸实验形成对比。两者都使用 K-L/PCA 降维，但人脸图像对齐程度高、结构稳定，PCA 保留的主要方差与身份差异有较强关系；飞机图像包含更多背景、尺度和姿态干扰，高方差方向不一定等于高判别方向。该对比说明，K-L 变换适合结构较稳定、样本对齐较好的图像识别问题；对于复杂自然图像，通常还需要更强的局部特征或监督学习模型。

5.8 体会

特征脸实验让我更容易理解 K-L 变换为什么能够用于图像识别。原始像素维数很高，但人脸之间存在共同结构，很多变化可以由少数主成分表示。把主成分重新显示成图像后，“特征脸”把抽象的协方差特征向量变得直观：它们描述的是人脸中最常见的明暗、轮廓和五官位置变化。

同时，补充飞机实验也提醒我不能只看累计方差贡献率。人脸数据上 120 个特征脸能得到较高识别率，是因为数据本身比较规整；飞机数据即使保留较多方差，分类准确率仍然有限。后续处理图像识别问题时，除了降维，还要考虑目标是否对齐、背景是否干净、类别差异是否能被当前特征表达出来。K-L 变换适合作为基础特征提取方法，但复杂图像任务还需要结合更有判别性的特征和分类模型。

6 实验五：聚类分析实验

6.1 模式识别理论知识

前面几个实验主要是监督分类：训练样本带有类别标签，分类器利用标签学习判别规则。聚类分析属于无监督学习，算法只使用样本特征，不在训练过程中使用真实性别标签。聚类的目标是根据样本之间的相似性自动形成若干组，再通过真实标签评价这些组是否具有明确的类别含义。本实验使用身高和体重两个特征观察男女样本在二维空间中的自然分组结构。

本实验使用二维特征

$$\mathbf{x} = [\text{身高}, \text{体重}]^T.$$

由于身高和体重的量纲及数值范围不同，聚类前先进行标准化：

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

这样可以避免某个数值尺度较大的特征在欧氏距离中占据过大影响。

6.1.1 C 均值聚类

C 均值聚类也常称为 k -means 聚类。给定聚类数 C ，算法希望找到 C 个聚类中心，使样本到所属中心的平方距离总和最小。目标函数为

$$J = \sum_{j=1}^C \sum_{\mathbf{x}_i \in \omega_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2.$$

其中 ω_j 表示第 j 个簇， $\boldsymbol{\mu}_j$ 表示该簇中心。算法迭代执行两步：

$$\omega_i = \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2,$$

$$\boldsymbol{\mu}_j = \frac{1}{N_j} \sum_{\mathbf{x}_i \in \omega_j} \mathbf{x}_i.$$

第一步把样本分配给最近中心，第二步用簇内样本均值更新中心，直到中心变化很小或达到最大迭代次数。由于目标函数是非凸的，随机初始中心可能影响最终结果。因此本实验会对 $C = 2$ 重复不同初值，观察 SSE 和性别匹配率的波动。

C 均值的优点是简单、快速、易实现；缺点是需要预先指定类别数 C ，对初始中心和特征尺度敏感，并且倾向于发现近似球形的簇。

6.1.2 聚类指标

为了分析聚类效果，本实验使用以下指标：

- SSE: 类内平方误差和, 即 C 均值目标函数。SSE 越小表示簇内越紧凑, 但它会随着聚类数增加而自然下降, 不能单独用于确定类别数。
- 轮廓系数: 综合衡量簇内紧凑性和簇间分离性, 取值越大通常说明聚类结构越清晰。
- ARI: 调整兰德指数, 用真实标签与聚类结果比较。ARI 越大表示聚类结果越接近真实类别。
- 性别匹配率: 聚类编号本身没有性别含义, 将两个簇与男女标签做最佳匹配后计算准确率, 仅用于解释聚类结果与性别标签的对应程度。

6.1.3 分级聚类

分级聚类通过逐步合并或分裂样本形成层次结构。本实验采用凝聚式 Ward 分级聚类: 初始时每个样本单独成簇, 然后不断合并使类内平方误差增加最小的两个簇。Ward 方法不需要随机初始中心, 可以用树状图观察样本逐层合并的过程; 缺点是计算量较大, 并且一旦完成某次合并, 后续不能再撤销。

6.2 实际数据集与实验设置

实验五使用两组数据:

- 训练集: FEMALE.TXT 与 MALE.TXT 合并, 共 100 个样本, 其中女生 50 个、男生 50 个。
- 合并数据: 训练集再加入 test2.txt, 共 400 个样本。由于 test2.txt 中有 50 个女生、250 个男生, 因此合并后类别比例变为女生 100 个、男生 300 个。

表 19: 实验五使用的数据集统计

数据集	样本数	女生数	男生数	平均身高	平均体重
训练集	100	50	50	168.38	59.05
test2	300	50	250	172.79	64.50
训练集 + test2	400	100	300	171.69	63.14

实验步骤如下:

- 对训练集设 $C = 2$, 独立实现 C 均值聚类, 并考察不同初始值的影响。
- 对训练集分别设置 $C = 2, 3, 4, 5$, 画出 SSE 和轮廓系数随类别数变化的曲线。
- 对训练集使用 Ward 分级聚类, 绘制树状图和二维聚类结果。
- 将训练集与 test2.txt 合并, 重复 C 均值和 Ward 聚类实验, 观察样本规模和类别比例变化带来的影响。

6.3 程序框图与代码实现

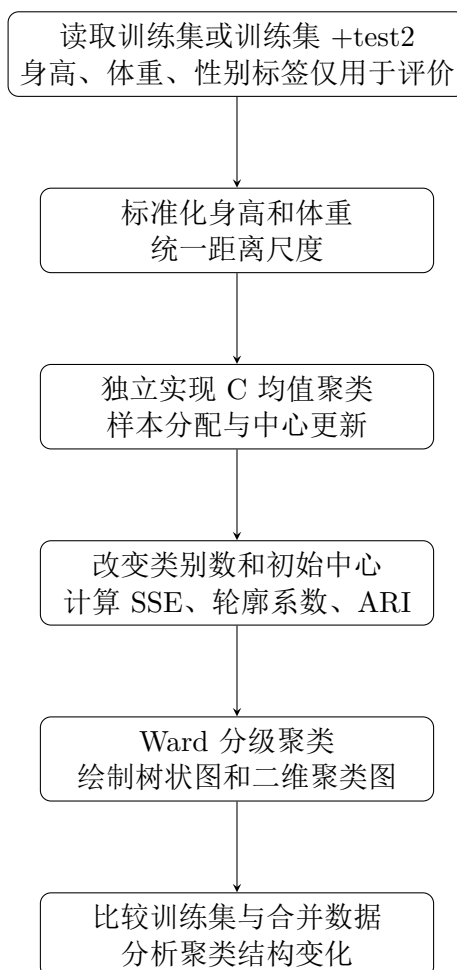


图 61: 实验五聚类分析流程图

实验五程序位于 `scripts/experiment5_clustering.py`。C 均值部分为独立编程实现，相关实现见附录代码 19。聚类指标计算与不同类别数实验见附录代码 20。Ward 分级聚类使用已有程序完成，相关实现见附录代码 21。

6.4 实验步骤与结果分析

6.4.1 步骤一：训练集上 $C=2$ 的 C 均值聚类与初始值影响

首先将 `FEMALE.TXT` 和 `MALE.TXT` 合并，使用身高和体重两个特征，设 $C = 2$ 进行 C 均值聚类。结果如图 62 所示。

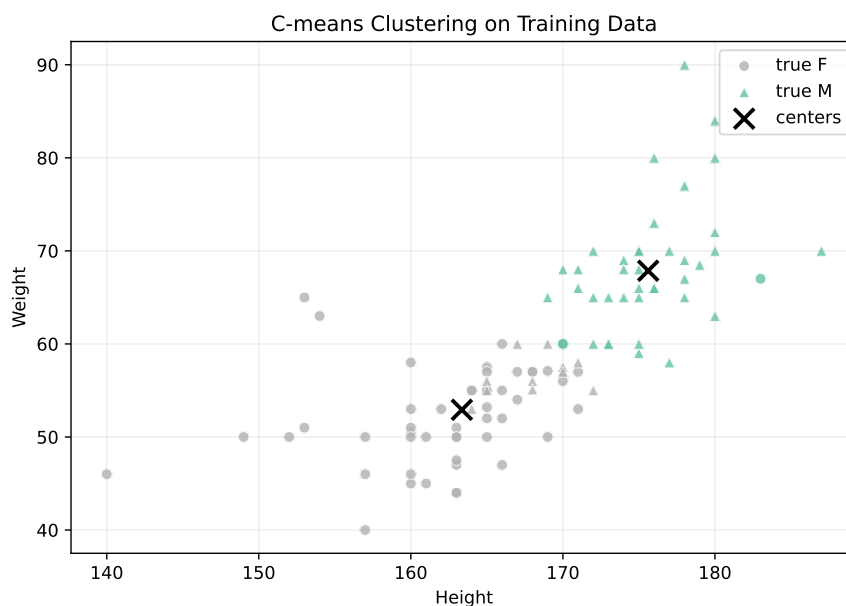


图 62: 训练集 C 均值聚类结果, $C = 2$

图中颜色表示聚类编号, 圆点和三角形表示真实性别。聚类结果大致形成“较矮较轻”和“较高较重”两个簇, 与男女样本的统计差异基本一致。虽然聚类算法没有使用性别标签, 但聚类结果与性别标签具有较强对应关系。

表 20: 训练集 C 均值聚类与真实性别的对应关系

真实类别	判为女生簇	判为男生簇
女生	47	3
男生	12	38

表 20 显示, 50 个女生中有 47 个落入“女生簇”, 50 个男生中有 38 个落入“男生簇”, 性别匹配率为 85.00%。错误样本主要集中在身高体重接近两类交界的位置, 说明无监督聚类可以发现数据几何结构, 但不能完全等同于真实性别分类。

为考察初始中心影响, 对 $C = 2$ 使用 10 个不同随机种子重复实验, 结果如图 63 所示。

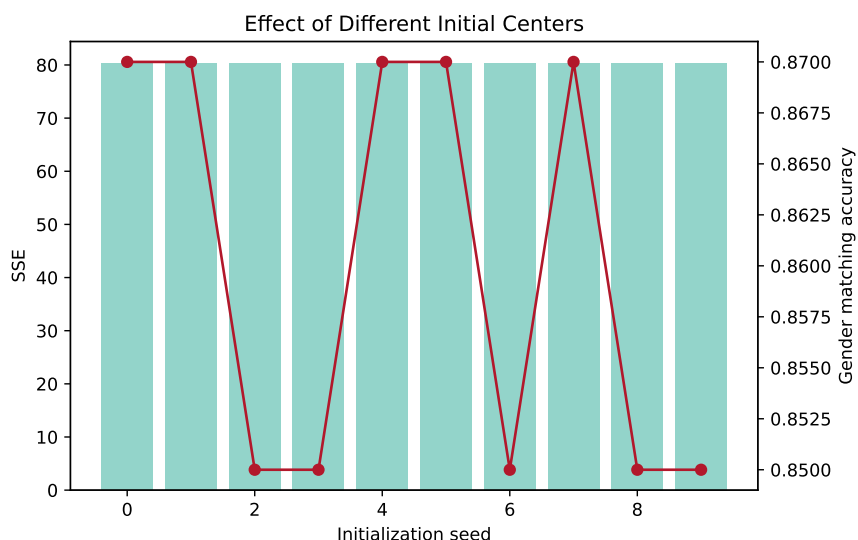


图 63: 不同初始中心对 C 均值聚类结果的影响

表 21: 训练集 C 均值不同初始值实验结果

随机种子	SSE	轮廓系数	ARI	性别匹配率
0	80.3193	0.5176	0.5432	87.00%
1	80.3193	0.5176	0.5432	87.00%
2	80.3950	0.5108	0.4850	85.00%
3	80.3950	0.5108	0.4850	85.00%
4	80.3193	0.5176	0.5432	87.00%
5	80.3193	0.5176	0.5432	87.00%
6	80.3950	0.5108	0.4850	85.00%
7	80.3193	0.5176	0.5432	87.00%
8	80.3950	0.5108	0.4850	85.00%
9	80.3950	0.5108	0.4850	85.00%

不同初始值下 SSE 在 80.3193 到 80.3950 之间变化，性别匹配率在 85.00% 到 87.00% 之间变化。说明该数据集聚类结构较稳定，但边界附近样本仍可能因初始中心不同而改变归属。实际使用 C 均值时，应多次初始化并选择 SSE 较小、轮廓系数较高的结果。

6.4.2 步骤二：不同类别数下的聚类指标

在训练集上分别设置 $C = 2, 3, 4, 5$ ，记录 SSE 和轮廓系数，如图 64 所示。

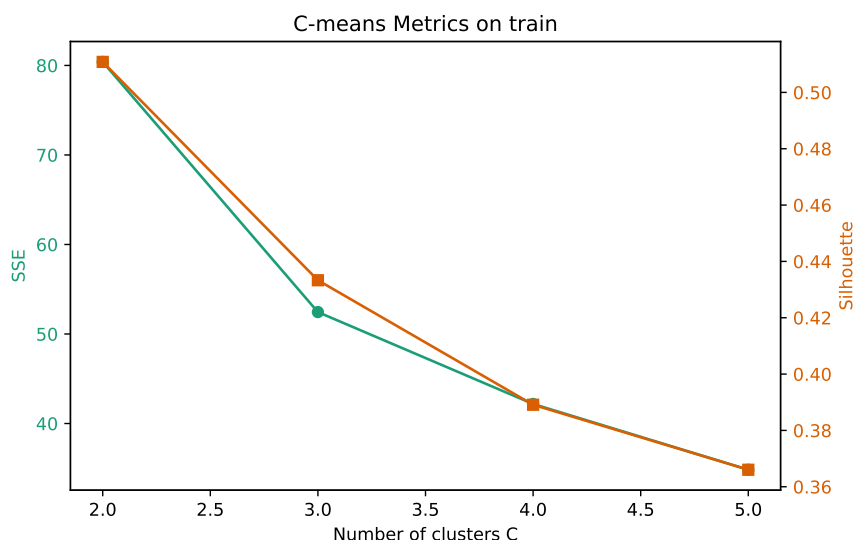


图 64: 训练集上不同类别数的 C 均值聚类指标

表 22: 训练集不同类别数下的 C 均值聚类指标

类别数	SSE	轮廓系数	ARI	迭代次数
2	80.3950	0.5108	0.4850	5
3	52.4481	0.4333	0.3185	16
4	42.1838	0.3891	0.3141	12
5	34.8358	0.3660	0.2381	4

训练集上 $C = 2, 3, 4, 5$ 的 SSE 分别为 80.3950、52.4481、42.1838、34.8358。SSE 随类别数增加单调下降，这是因为更多中心必然能降低类内平方误差。但轮廓系数分别为 0.5108、0.4333、0.3891、0.3660，随类别数增加下降。 $C = 2$ 的轮廓系数最高，且与男女两类的实际标签结构一致，因此训练集上合理类别数更倾向于 2。

6.4.3 步骤三：Ward 分级聚类

对训练集使用 Ward 分级聚类，树状图如图 65 所示，取两类后的二维聚类结果如图 66 所示。

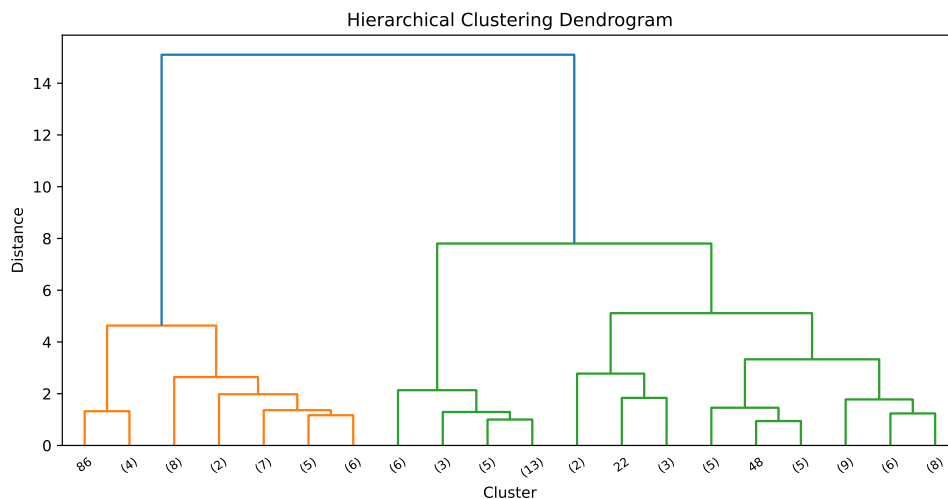


图 65: 训练集 Ward 分级聚类树状图

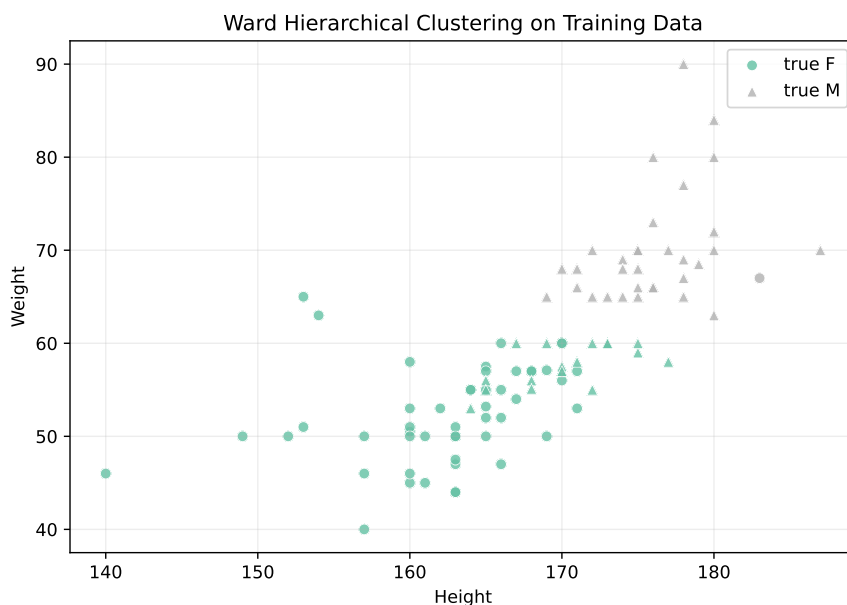


图 66: 训练集 Ward 分级聚类结果, $C = 2$

表 23: 训练集 Ward 分级聚类与真实性别的关系

真实类别	判为女生簇	判为男生簇
女生	49	1
男生	18	32

Ward 分级聚类在训练集上取两类时, 轮廓系数为 0.5049, ARI 为 0.3789, 性别匹配率为 81.00%。其性别匹配率略低于 C 均值, 但分级聚类的优势在于可以通过树状图观察样本逐层合并的过程, 不依赖随机初始中心。树状图中后期的大距离合并说明数据确实存在较明显的两大分支结构。

6.4.4 步骤四：加入 test2 后重复实验

最后将训练集与 `test2.txt` 合并，共 400 个样本，其中男生比例明显升高。对合并数据重复 $C = 2$ 的 C 均值聚类，结果如图 67 所示。

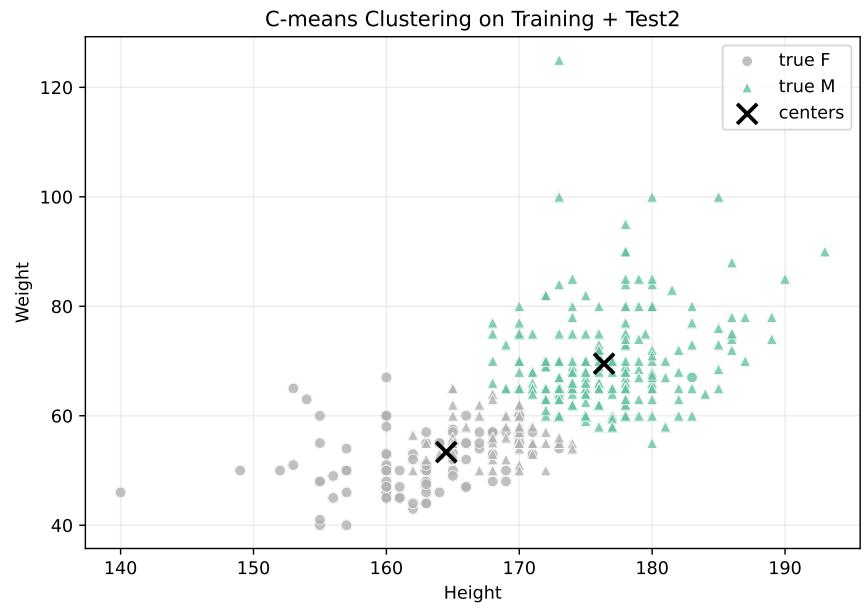


图 67: 训练集与 test2 合并后的 C 均值聚类结果， $C = 2$

合并数据上不同类别数的指标如图 68 所示。

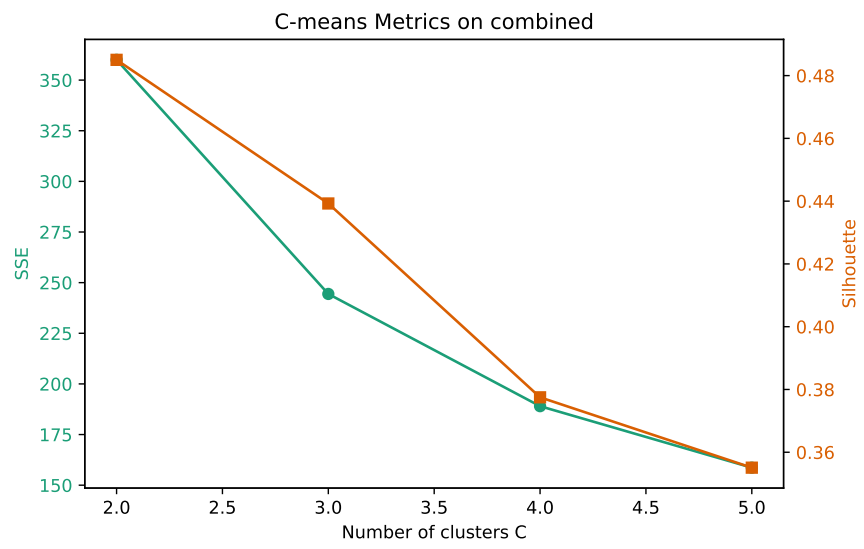


图 68: 训练集与 test2 合并后不同类别数的 C 均值聚类指标

表 24: 训练集与 test2 合并后不同类别数下的 C 均值聚类指标

类别数	SSE	轮廓系数	ARI	迭代次数
2	360.0075	0.4850	0.4693	6
3	244.4188	0.4393	0.3838	26
4	189.0085	0.3775	0.2840	18
5	158.6469	0.3551	0.2272	10

对合并数据使用 Ward 分级聚类并取两类，结果如图 69 所示。

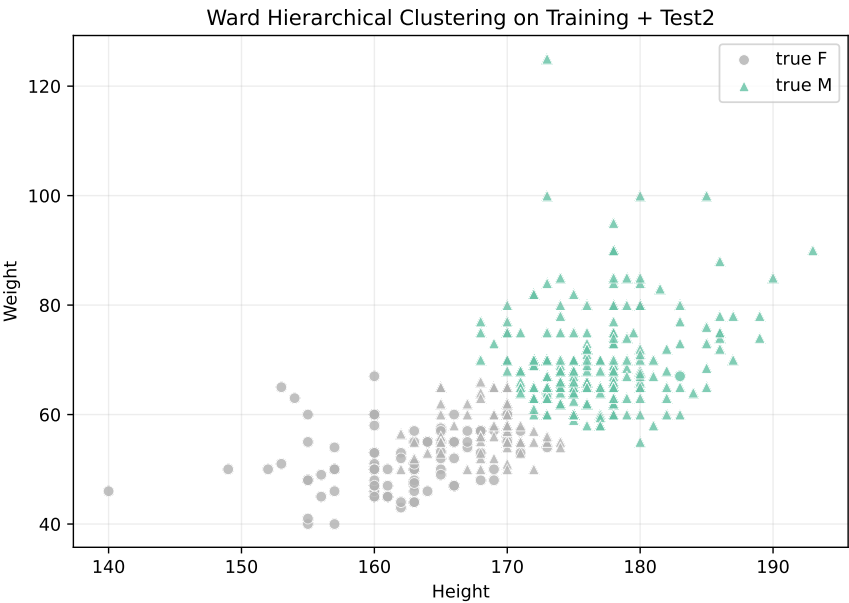


图 69: 训练集与 test2 合并后的 Ward 分级聚类结果， $C = 2$

表 25: 合并数据 C 均值聚类与真实性别的关系

真实类别	判为女生簇	判为男生簇
女生	98	2
男生	60	240

表 26: 合并数据 Ward 分级聚类与真实性别的关系

真实类别	判为女生簇	判为男生簇
女生	98	2
男生	68	232

合并数据中 C 均值 $C = 2$ 的轮廓系数为 0.4850，ARI 为 0.4693，性别匹配率为 84.50%；Ward 分级聚类的轮廓系数为 0.4739，性别匹配率为 82.50%。与训练集相比，

合并数据的轮廓系数略有下降，说明新增样本增加了类内变化和边界重叠；但 $C = 2$ 仍然具有最高轮廓系数，合理类别数仍可认为是 2。

表 27: 实验五聚类结果汇总

数据集	方法	类数	轮廓系数	ARI	性别匹配率
训练集	C 均值	2	0.5108	0.4850	85.00%
训练集	Ward 分级聚类	2	0.5049	0.3789	81.00%
训练集 +test2	C 均值	2	0.4850	0.4693	84.50%
训练集 +test2	Ward 分级聚类	2	0.4739	0.4172	82.50%

6.5 综合实验结果与分析总结

表 27 汇总了主要实验结果。C 均值在训练集上取 $C = 2$ 时性别匹配率为 85.00%，说明身高体重空间中存在与性别相关的自然聚类结构。不同初始中心会造成少量边界样本变化，但 SSE 和性别匹配率波动较小，说明该数据集上的 C 均值结果较稳定。

类别数实验中，SSE 随 C 增大持续下降，但轮廓系数从 $C = 2$ 开始下降。这说明单看 SSE 会倾向于选择更多簇，而结合轮廓系数和真实任务背景， $C = 2$ 更能反映男女身高体重数据的主要结构。Ward 分级聚类不需要随机初始化，能够展示层次关系，但本数据集上性别匹配率略低于 C 均值。加入 `test2.txt` 后，样本量增大且类别比例变为男生占多数，聚类结构仍以两类为主，但边界重叠更加明显。

聚类结果与前面监督分类实验相互印证：身高和体重确实具有性别区分信息，但男女样本在边界区域存在重叠，因此无监督聚类无法完全恢复真实性别标签。聚类算法发现的是数据几何结构，而不是语义标签本身。

6.6 体会

通过本实验可以体会到，聚类与分类的目标不同。分类器利用标签学习判别边界，而聚类只根据距离和分布自动分组。即使聚类结果与性别标签较一致，也只能说明特征空间中存在相应结构，不能说明聚类算法真正理解了“性别”这个类别含义。

C 均值算法实现简单、效率高，适合低维连续特征；但它对类别数、初始中心和标准化处理敏感。分级聚类能够展示更丰富的层次关系，适合观察样本合并过程。实验中两种方法都支持 $C = 2$ 是较合理类别数，这与男女数据集的真实类别数一致，也说明身高和体重是较有效的聚类特征。

7 补充实验一：基于 DT-ProtoYOLO 的开集目标检测

7.1 模式识别理论知识

7.1.1 闭集识别与开集识别

传统目标检测通常采用闭集假设：训练集和测试集类别集合相同，模型只需要在已知类别之间进行分类。在实际环境中，测试图像中可能出现训练阶段没有见过的类别。若闭集检测器仍然强制把未知目标分到某个已知类，就会产生开放空间风险。开集目标检测要求模型既能检测已知类目标，也能把未知类目标识别为 unknown，而不是错误地归入某个已知类。

设检测框对应的类别置信度为 $p(c_i|x)$ ，闭集检测器通常取

$$\hat{c} = \arg \max_i p(c_i|x). \quad (1)$$

开集检测还需要增加拒识机制。若一个候选框虽然具有目标外观，但与所有已知类别都不够匹配，则应输出 unknown。这样，分类决策不再只是“选哪一类”，还包括“是否属于已知类”。

7.1.2 原型表示

DT-ProtoYOLO 在检测头中加入特征投影分支，将候选目标的局部特征映射为原型空间中的向量 $z \in \mathbb{R}^{128}$ 。每个已知类别维护一个可学习原型 p_i 。对于预测类别 $\hat{c}$ ，计算目标特征与该类别原型之间的距离

$$d = \|z - p_{\hat{c}}\|_2. \quad (2)$$

若样本确实属于已知类，则其投影特征应靠近对应类别原型；若样本是未知类，即使分类分支给出较高置信度，它与已知类原型的距离也可能较大。

原型正则化由吸引项和排斥项组成。吸引项使同类样本靠近本类原型：

$$L_{\text{attr}} = \|z - p_y\|_2^2. \quad (3)$$

排斥项使样本远离非目标类别原型，可写为

$$L_{\text{rep}} = \sum_{j \neq y} \max(0, m - \|z - p_j\|_2)^2, \quad (4)$$

其中 m 是间隔参数。总的原型损失可表示为

$$L_{\text{proto}} = \lambda_{\text{attr}} L_{\text{attr}} + \lambda_{\text{rep}} L_{\text{rep}} + \lambda_{\text{cls}} L_{\text{proto_cls}}. \quad (5)$$

7.1.3 双线性阈值判别

本实验中的开集决策同时使用分类置信度和原型距离。设

$$s = \max_i p(c_i|x), \quad d = \|z - p_{\hat{c}}\|_2. \quad (6)$$

双线性阈值规则为

$$\begin{cases} \text{known}, & s \geq \alpha \text{ and } d \leq \delta, \\ \text{unknown}, & \beta \leq s < \alpha \text{ or } d > \delta, \\ \text{background}, & s < \beta. \end{cases} \quad (7)$$

其中 α 控制已知类确认阈值， β 控制目标候选与背景的分界， δ 控制原型空间中已知类可接受的距离范围。相比只看 softmax 置信度，加入原型距离后可以降低未知目标被误判为已知类的概率。

训练阶段还在潜在未知区间 $\beta \leq s \leq \alpha$ 内引入不确定性最大化约束。设类别概率分布为 $p = (p_1, p_2, \dots, p_K)$ ，不确定性损失写为

$$L_{\text{unc}} = -H(p) = -\sum_{i=1}^K p_i \log p_i. \quad (8)$$

最小化该损失等价于增大预测熵，使处于灰色区间的候选框不被强行压到某个已知类别上。模型总损失可概括为

$$L_{\text{total}} = \lambda_1 L_{\text{box}} + \lambda_2 L_{\text{dfl}} + \mathbb{I}(s > \alpha) \lambda_3 L_{\text{cls}} + \mathbb{I}(\beta \leq s \leq \alpha) \lambda_4 L_{\text{unc}} + \lambda_5 (L_{\text{attr}} + L_{\text{rep}}). \quad (9)$$

其中检测损失保证定位和已知类分类能力，不确定性损失控制概率层面的过度自信，原型损失控制特征空间中的开放空间风险。

7.2 数据集与实验设置

补充实验一使用仓库 yolov11_osod 中的 VOC 开集检测设置。数据集来源为 PASCAL VOC 2007，训练时只使用已知类别，评估时保留未知类别用于测试开集能力。仓库文档中给出的设置为：VOC 20 类中编号 0 ~ 14 作为已知类，编号 15 ~ 19 作为未知类；训练标签生成已知类子集，验证集保留完整 20 类标签，以便统计 unknown 发现情况和误检情况。

表 28: 补充实验一主要模块与设置

模块	代码或配置位置	作用
闭集检测基线	scripts/train/train.py, YOLO11n 权重	在 15 个已知类上训练常规 YOLO 检测器, 作为开集检测对比基线。
特征投影分支	Detect 检测头, proto_dim=128	将检测位置对应的特征映射到原型空间, 为类别原型距离计算提供特征表示。
可学习类别原型	Detect.prototypes	为每个已知类维护一个原型向量, 用样本特征与原型距离描述“是否像该类”。
双线性阈值	osod_inference.py, $\alpha = 0.37$, $\beta = 0.10$, $\delta = 16.5$	同时利用分类置信度和原型距离, 将候选框划分为已知目标、未知目标和背景。
原型正则化	utils/loss.py 中的原型损失	通过吸引项、排斥项和原型分类损失, 使同类特征靠近本类原型, 不同类特征彼此分离。

模型采用 YOLO11n 作为检测主干, 在检测头中加入 128 维投影特征和可学习类别原型。主要超参数为 $\alpha = 0.37$ 、 $\beta = 0.10$ 、 $\delta = 16.5$, 原型损失权重采用 $\lambda_{\text{proto}} = 0.05$, 不确定性损失权重采用 $\lambda_{\text{unc}} = 0.01$ 。训练流程分为闭集基线训练、原型初始化、DT-ProtoYOLO 训练、阈值校准和开集评估五个阶段。

论文实验环境为 Ubuntu 20.04, 单张 NVIDIA RTX 4060 Laptop GPU, 深度学习框架为 PyTorch。训练时输入图像统一缩放为 640×640 , 端到端训练 100 个 epoch, batch size 为 12。已知类训练结束后, 在完整 VOC 20 类验证集上统计闭集基线与 DT-ProtoYOLO 的开集性能。

7.3 方法设计与程序框图

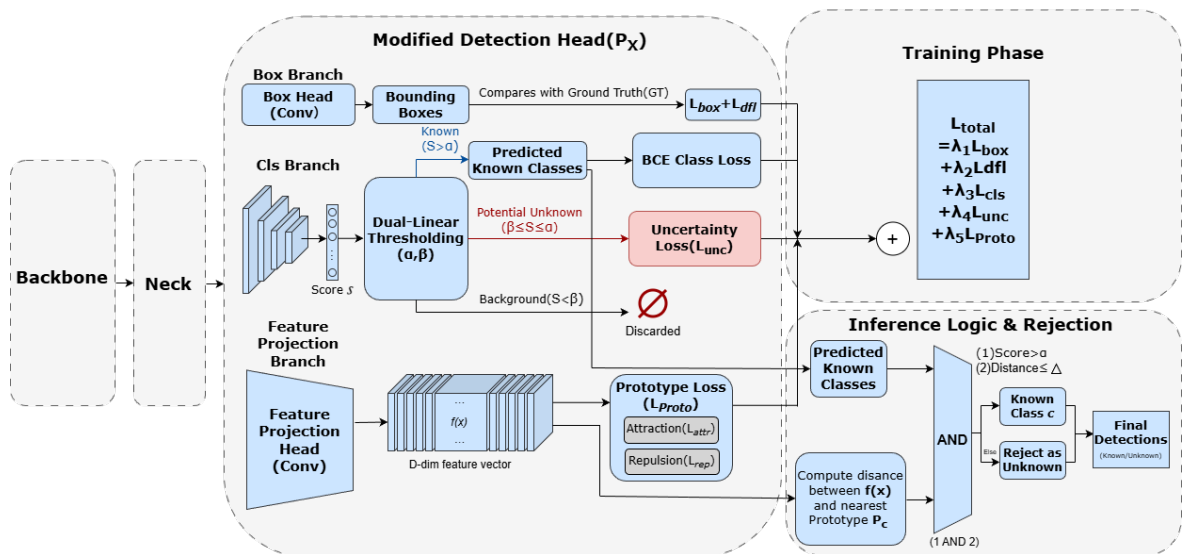


图 70: DT-ProtoYOLO 总体架构图

如图 70 所示, DT-ProtoYOLO 保留 YOLO11 的 Backbone 和 Neck, 重点重构检测头。检测头包含三个分支: Box Branch 负责边界框回归, Cls Branch 负责类别置信

度，Feature Projection Branch 将候选框特征映射到 D 维原型空间。训练阶段，已知高置信样本进入 BCE 分类损失，潜在未知样本进入不确定性损失，投影特征同时受到原型吸引和排斥约束。推理阶段，候选框必须同时满足分类置信度和原型距离两个条件，才能被确认为已知类；若任一条件不满足，则进入 unknown 拒识分支。

7.4 实验结果与分析

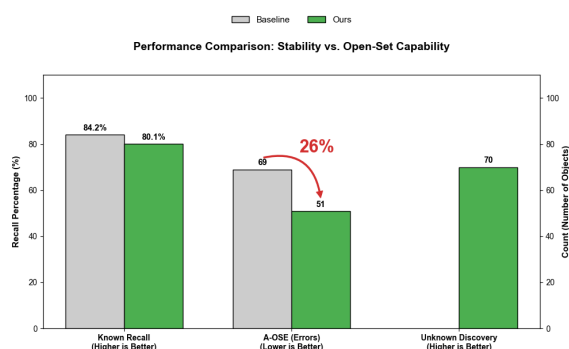
表 29: 补充实验一开集检测评价指标

指标	含义	评价方向
Known Recall	已知类真实目标被正确检出的比例。	越高越好
A-OSE	未知类真实目标被错误识别为某个已知类的数量。	越低越好
Unknown Discovery	未知类真实目标被检测并标记为 unknown 的比例。	越高越好
Known mAP	已知类检测框定位和分类的平均精度。	越高越好
阈值敏感性	改变 α, β, δ 后未知发现率和误检数量的变化。	观察稳定性

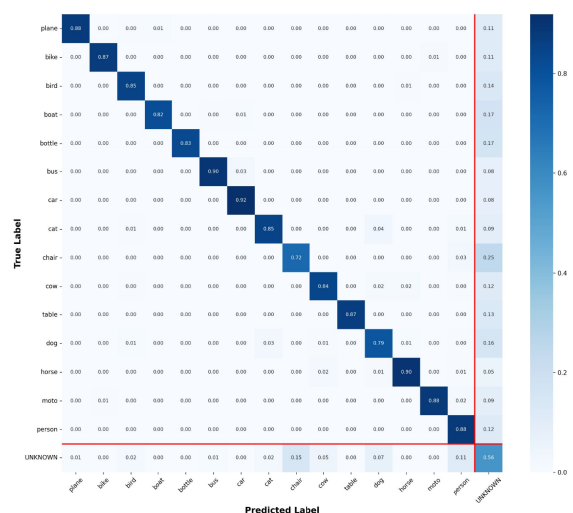
论文实验以 YOLO11n 闭集检测器作为 Baseline，并与加入双线性阈值和原型正则化后的 DT-ProtoYOLO 进行比较。主要结果如表 30 和图 71a 所示。

表 30: 补充实验一论文定量结果

指标	Baseline	DT-ProtoYOLO	变化	结果说明
Known Recall	84.2%	80.1%	-4.1 个百分点	已知类召回略有下降，但保留了基线约 95.1% 的相对召回能力。
A-OSE	69	51	-26.1%	未知目标被误识别为已知类的错误数明显减少，开集安全性提升。
Unknown Discovery	0	70	显著提升	闭集基线无法输出 unknown, DT-ProtoYOLO 能发现并标记未知目标。



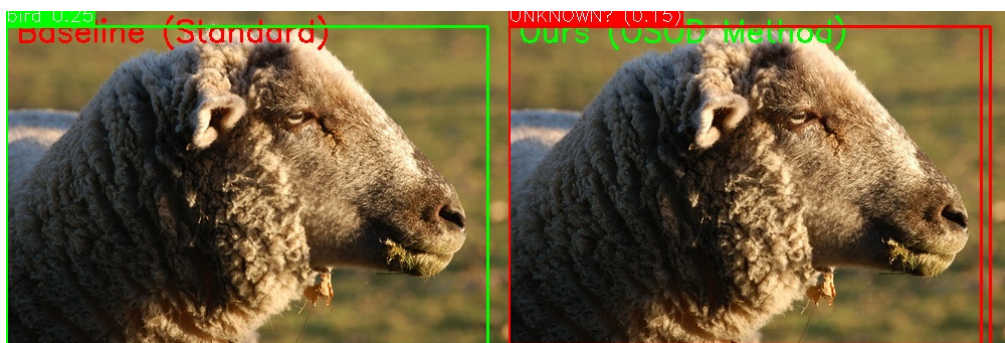
(a) 开集性能对比



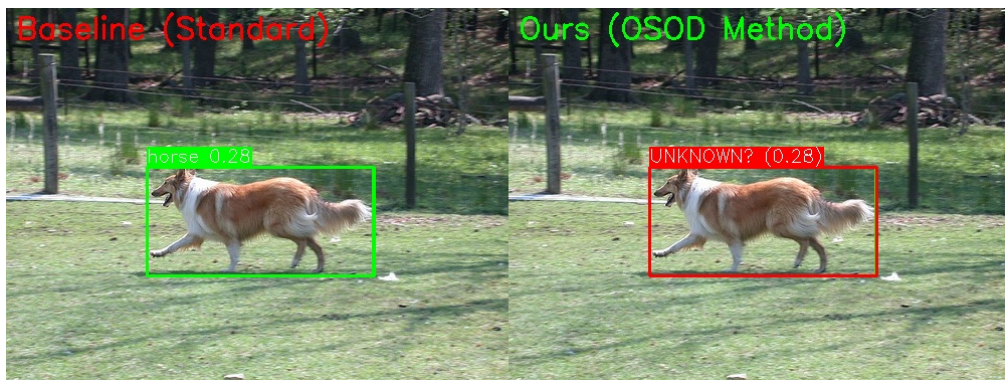
(b) 开集测试混淆矩阵

图 71: DT-ProtoYOLO 定量结果可视化

Baseline 的 Known Recall 为 84.2%，DT-ProtoYOLO 为 80.1%，下降 4.1 个百分点。该下降来自更保守的拒识策略：部分遮挡、光照差或外观不典型的已知类目标会因为原型距离超过阈值而被拒识。不过，DT-ProtoYOLO 仍保留了基线约 95.1% 的相对已知类召回能力。相比之下，A-OSE 从 69 降到 51，下降 26.1%，说明未知目标被错误归入已知类的情况明显减少。Unknown Discovery 从 0 增加到 70，说明闭集基线完全没有 unknown 输出能力，而 DT-ProtoYOLO 能够把一部分新类别目标显式标记为未知目标。



(a) Semantic Novelty: sheep 场景



(b) Semantic Confusion: collie 场景



(c) Small Object Discovery: Chihuahua 场景

图 72: DT-ProtoYOLO 开集检测定性示例

图 71b 中的混淆矩阵显示，真实 unknown 样本中有 56% 被正确分配到 UNKNOWN 类，而 Baseline 对 unknown 的发现率为 0。矩阵最后一列也反映出部分已知类被拒识为 unknown，例如 chair 和 dog 类中存在一定比例的误拒识。这说明开集检测存在安全性和稳定性的权衡：模型越谨慎，未知类误判为已知类的风险越低，但一部分边界已知类样本也可能被拒识。

定性检测结果进一步说明了双线性阈值和原型距离的作用。在 sheep 场景中，Baseline 将未知类别错误识别为 bird；在 collie 场景中，Baseline 受外观相似性影响，将目标识别为 horse；DT-ProtoYOLO 虽然分类分支可能受到纹理或姿态干扰，但原型距离提供了第二道判别条件，使这些样本被重新标记为 UNKNOWN。在小目标 Chihuahua 场景中，低阈值 $\beta = 0.1$ 保留了不确定候选框，再由原型距离判定其偏离已知 dog 原型，从而把原本可能被漏检的风险目标转化为 unknown 发现。

从方法结构上分析，双线性阈值主要影响推理阶段的概率拒识边界。若 α 过高，已知类目标可能被拒识为 unknown，Known Recall 会下降；若 α 过低，未知目标更容易被当作已知类，A-OSF 会升高。 β 影响低置信候选是否直接作为背景过滤， δ 则控制

原型空间的拒识边界。原型正则化主要影响特征空间结构：类内特征越紧凑，距离阈值越稳定；类间原型间隔越大，未知目标越容易与已知类区分。

与课程中的 Bayes 分类、Fisher 判别和 K-L 变换相比，开集目标检测面对的是更复杂的图像空间。课程实验中分类器通常在固定类别集合中工作，而本实验增加了 unknown 决策，评价目标也从单纯准确率扩展到 Known Recall、A-OSE 和 Unknown Discovery。它对应模式识别中的拒识分类、距离度量学习和开放空间风险控制问题。

8 补充实验二：视觉分类器奖励引导的无人机数字轨迹

8.1 模式识别理论知识

8.1.1 视觉分类器作为模式评价函数

该课外实验来自仓库 Drone-Digit-RL。任务目标不是直接分类静态图像，而是让无人机在二维平面上飞出类似数字的轨迹。环境把一段连续运动轨迹渲染成 28×28 灰度图，再交给 CNN 数字分类器评价。分类器输出 11 类概率：数字 0 ~ 9 和 OOD/乱码类。若目标数字为 k ，则视觉评价可以写为

$$p = \text{CNN}(I_\tau), \quad p_k = p(y = k|I_\tau), \quad (10)$$

其中 I_τ 是轨迹 τ 渲染得到的图像。为了避免轨迹虽然得到某个数字概率但整体不可读，引入 OOD 概率 p_{ood} 和置信间隔

$$\text{margin} = p_k - \max_{j \neq k, j \neq \text{ood}} p_j. \quad (11)$$

奖励函数可以由目标数字概率、混淆类别概率和 OOD 概率共同构成。这样，CNN 分类器相当于一个模式识别评分器，把连续控制问题中的轨迹质量转化为可优化的数值信号。

8.1.2 CNN 数字识别的理论基础

卷积神经网络属于监督学习分类器，其输入是图像矩阵，输出是各类别的判别得分或后验概率估计。与传统 Bayes 分类器需要显式估计 $p(x|\omega_i)$ 不同，CNN 直接从标注样本中学习从图像到类别的非线性映射

$$f_\theta : I \rightarrow z, \quad z = [z_0, z_1, \dots, z_{10}]^T, \quad (12)$$

其中 I 为轨迹渲染图像， z_i 为第 i 类的分类得分， θ 为卷积核和全连接层权值。最后一层通常使用 Softmax 将得分转化为概率形式：

$$p(y = i|I) = \frac{\exp(z_i)}{\sum_{j=0}^{10} \exp(z_j)}. \quad (13)$$

因此，CNN 的输出可以看作对后验概率 $P(\omega_i|x)$ 的近似估计，分类时取概率最大的类别：

$$\hat{y} = \arg \max_i p(y = i|I). \quad (14)$$

CNN 适合处理轨迹图像，主要原因在于卷积层具有局部感受野和权值共享。局部感受野使卷积核只关注图像中的局部笔画结构，例如短直线、弯折、交叉点和闭环边

缘；权值共享使同一个卷积核可以在整幅图像不同位置检测相同模式，减少参数数量并增强平移泛化能力。对于数字轨迹来说，数字 6、8、9 等类别的区分往往依赖闭合区域、上下笔画连接和交叉位置，这些结构都可以由多层卷积逐步组合得到。

池化层用于对局部区域进行下采样，保留主要响应并降低小幅位置变化带来的影响。经过若干卷积和池化后，图像被表示为高层特征向量；全连接层再根据这些特征完成类别判别。若训练样本为 (I_n, y_n) ，模型参数通常通过最小化交叉熵损失学习：

$$L_{\text{cls}} = -\frac{1}{N} \sum_{n=1}^N \sum_{i=0}^{10} \mathbb{I}(y_n = i) \log p(y = i | I_n). \quad (15)$$

该损失会提高真实类别的预测概率，降低其他类别概率，使网络逐渐学习到数字图像中具有判别性的笔画模式。加入 OOD/乱码类后，分类器还具备一定拒识能力：当轨迹不像任何规范数字时，理想输出应偏向 OOD 类，而不是强行归入 0 ~ 9 中的某一类。

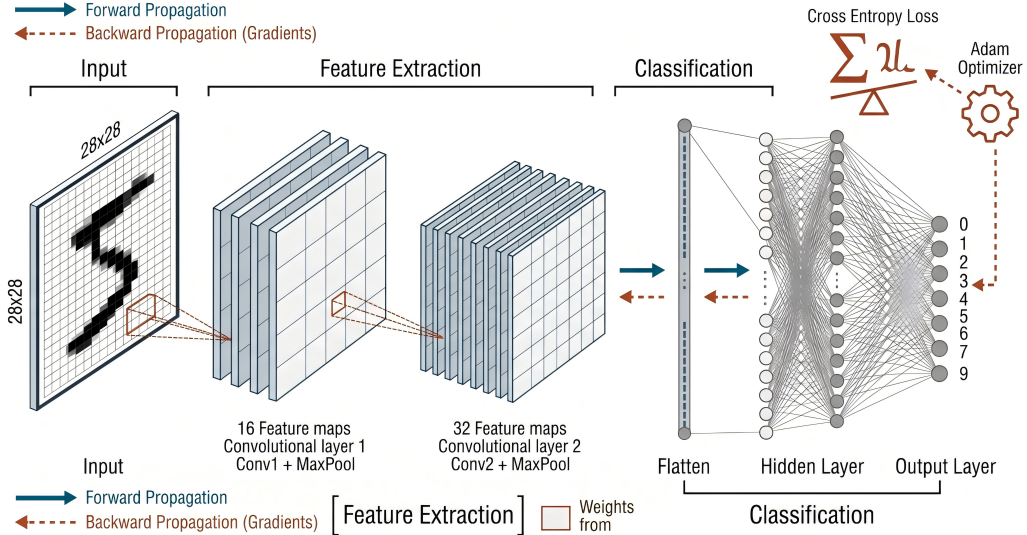


图 73: 基于 CNN 的轨迹图像视觉分类器结构

图 73 展示了本实验中视觉分类器的基本结构。输入为轨迹渲染后的 28×28 灰度图，卷积层负责提取笔画边缘、局部弯折和闭环等空间特征，池化层降低小幅位移带来的影响，全连接层将图像特征映射为数字类别概率。该结构与 MNIST 数字识别中的 CNN 思路一致，但在本实验中它不仅用于离线分类，还被进一步用作强化学习奖励的一部分。换言之，CNN 的模式识别结果从“最终评价指标”变成了“策略优化目标”：轨迹越接近目标数字，目标类别概率越高；轨迹越像乱码，OOD 概率越高，奖励越低。

8.1.3 无监督技能发现与下游迁移

无监督强化学习先不指定具体数字，而是鼓励策略学习多样化可控技能。预训练结束后，再通过 zero-shot 技能搜索或 few-shot 微调寻找能画出目标数字的技能。zero-

shot 阶段可采用 CEM 在技能空间中搜索，使目标数字概率和奖励最大；few-shot 阶段则继续用目标数字奖励对策略做短时间微调。

该实验与模式识别的联系主要体现在两个方面：第一，CNN 对轨迹图像进行数字模式识别，决定轨迹是否符合目标类别；第二，OOD/乱码类提供拒识能力，避免策略利用分类器漏洞生成不可读但高置信的图像。

8.2 数据集与实验设置

实验环境为 DroneDigit-v0。智能体输出二维连续动作，环境对位置进行积分得到轨迹；每个 episode 结束后，轨迹被渲染成灰度图并输入 11 类 CNN 分类器。仓库文档给出的评价流程包括：加载预训练快照、搜索或选择技能、固定技能 rollout、保存奖励、目标类别概率、OOD 概率、置信间隔、轨迹图和分类器输入图。

表 31: 补充实验二主要模块与设置

模块	实验设置	模式识别作用
轨迹环境	DroneDigit-v0, 二维连续动作控制轨迹生成	将无人机控制问题转化为可渲染的数字轨迹模式。
图像表示	episode 结束后渲染为 28×28 灰度图	与手写数字识别输入形式一致，便于使用 CNN 判别轨迹形状。
视觉分类器	11 类 CNN, 数字 0 ~ 9 加 OOD/乱码类	输出目标数字概率、混淆概率和 OOD 概率，作为轨迹质量评价。
无监督预训练	HID-APT/BeCL 风格技能发现	在不指定目标数字的情况下学习多样化运动技能。
技能搜索与微调	zero-shot CEM 选技能, few-shot 目标数字微调	根据分类器得分选择或优化能画出指定数字的轨迹。

对比实验可设置为 CIC、BeCL、CESD 等无监督技能发现算法；目标数字为 0 ~ 9；每个目标数字在多个随机种子下测试。实验分为 zero-shot 和 few-shot 两种情况：zero-shot 不更新策略，只搜索技能；few-shot 在预训练策略基础上针对目标数字进行短步数微调。

8.3 方法设计与程序框图

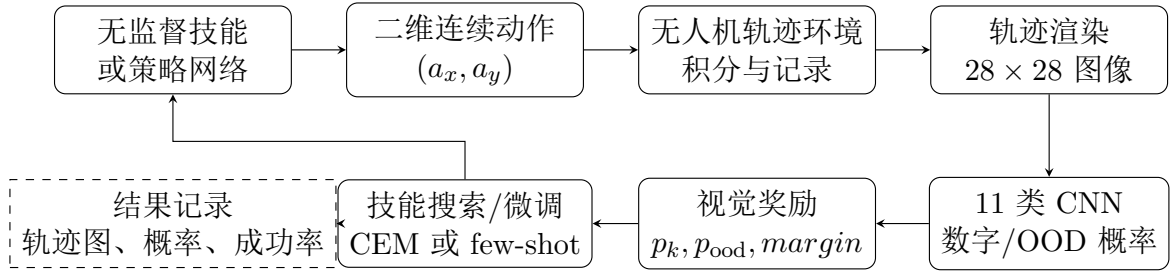


图 74: 视觉分类器奖励引导的无人机数字轨迹生成流程

图 74 中形成了一个由模式识别模型参与的闭环：策略负责生成轨迹，CNN 负责识别轨迹像哪个数字，识别结果再反过来构成奖励。若目标是数字 8，则一条好的轨迹应具有较高的 p_8 、较低的 p_{ood} ，并且 p_8 与其他数字概率之间有足够大的 $margin$ 。

8.4 实验结果与分析

当前报告根据仓库文档和已有可视化结果整理该实验的评价指标和结果记录格式。表 32 记录了实验结果应包含的关键量；对应数值可由 `evaluate_zero_shot_skill.py`、`finetune.py` 和可视化脚本生成。

表 32: 补充实验二评价指标与结果记录

记录量	含义	分析用途
<code>episode_reward</code>	一条完整轨迹得到的分类器引导奖励。	衡量策略总体表现。
<code>prob_target</code>	CNN 将轨迹图像判为目标数字的概率。	判断轨迹是否像目标数字。
<code>prob_ood</code>	CNN 将轨迹判为 OOD/乱码的概率。	衡量轨迹是否可识别。
<code>margin</code>	目标数字概率与最强非目标类概率之差。	衡量识别置信间隔。
<code>success_rate</code>	满足目标概率和 OOD 约束的轨迹比例。	比较不同算法和目标数字。
轨迹图与分类器输入图	原始飞行轨迹和 28×28 渲染图。	检查数值指标是否符合视觉效果。

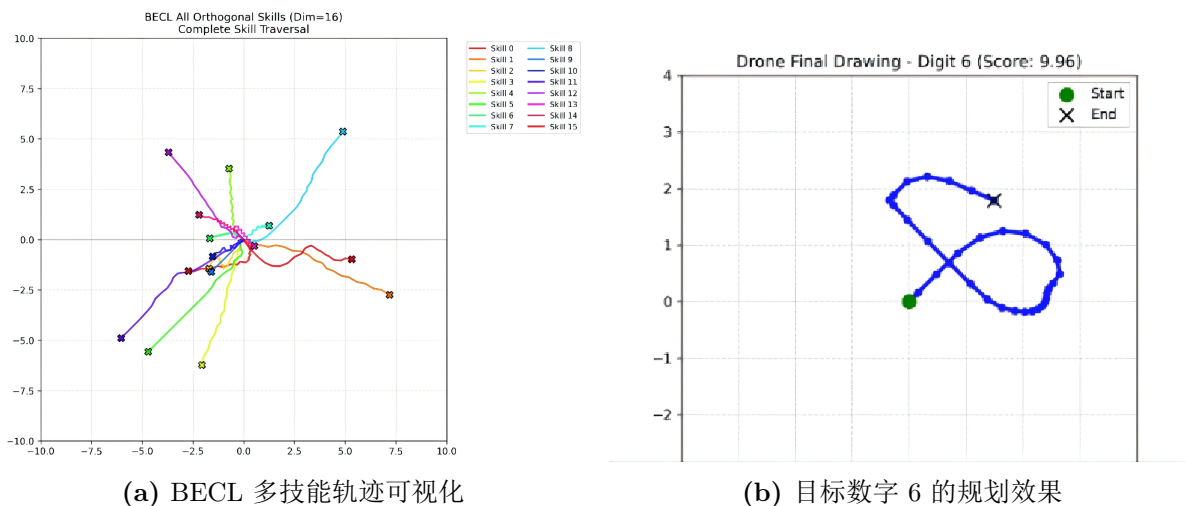


图 75: 补充实验二轨迹生成与技能可视化结果

图 75a 给出了 BECL 方法学习到的多条技能轨迹。不同颜色代表不同技能，可以看到各技能在方向、长度和弯曲方式上存在明显差异，说明无监督预训练阶段并不是只学习单一运动模式，而是在技能空间中形成了多样化的轨迹原型。这样的多样性对后续 zero-shot 搜索很重要：如果预训练技能库中已经包含弧线、折线、闭合环等基本运动片段，搜索目标数字时就更容易找到接近数字形状的技能组合。

图 75b 展示了目标数字 6 的规划效果。蓝色曲线为无人机最终绘制轨迹，绿色点表示起点，黑色叉号表示终点。该轨迹形成了较清晰的上部弯折和下部闭环，因此 CNN 分类器给出较高的数字 6 评分。这个结果说明视觉分类器奖励能够把抽象的“数字形状相似性”转化为强化学习可以优化的反馈信号；同时，闭环是否完整、笔画是否连续也会直接影响分类器置信度。

在结果分析时，需要同时观察数值指标和轨迹图像。若只看 `prob_target`，策略可能生成局部纹理或不完整轨迹，使分类器短暂偏向目标数字；加入 `prob_ood` 和 `margin` 后，可以过滤掉不可读轨迹。zero-shot 结果主要反映无监督技能库是否包含接近目标数字的运动模式；few-shot 结果则反映预训练技能是否便于迁移到具体数字任务。若某些数字如 1、7 较容易生成，而 8、9 较难生成，通常与轨迹拓扑结构有关：闭环和交叉结构对连续控制策略要求更高。

与前面 MNIST/CIFAR 图像分类实验相比，本实验中的图像不是固定数据集样本，而是由策略主动生成。分类器不只是评价最终结果，还直接影响学习方向。因此，视觉分类器的鲁棒性会影响强化学习策略：分类器若对乱码过于自信，策略可能学到投机轨迹；加入 OOD 类和 `margin` 约束后，奖励更符合人的视觉判断。

9 总体总结

五个课内实验从参数 Bayes 分类、非参数估计、Fisher 线性判别、K-L 特征提取、K-L 应用和聚类分析等角度覆盖了模式识别课程中的主要方法。实验一除男女身高体重数据外，还补充了模拟高斯数据、UCI Iris、MNIST、CIFAR-10 和 CIFAR-100 上的 Bayes 分类实验，用于比较参数估计、非参数估计、最小错误率和最小风险准则在不同复杂度数据集上的表现。男女身高体重数据和 Iris 数据结构清晰，适合分析分类边界与概率模型；MNIST 和 CIFAR 等图像数据对特征表示要求更高，简单 PCA 特征和传统 Bayes 分类器性能有限。

两个补充实验把课程方法扩展到更接近实际应用的视觉任务中。DT-ProtoYOLO 开集检测实验强调 unknown 拒识、原型距离和开放空间风险控制；无人机数字轨迹实验强调 CNN 视觉分类器作为奖励模型，把轨迹生成和模式识别联系起来。前者说明分类器需要知道“哪些样本不属于已知类”，后者说明分类器还可以参与决策与控制过程。通过这些实验可以看到，模式识别方法既可以用于低维统计分类，也可以作为现代视觉系统和智能体学习系统中的关键模块。

A 核心程序代码

本附录集中列出正文实验中的关键程序片段。

A.1 实验一核心代码

Listing 1: Exp.1 data loading and label normalization

```
1 def normalize_label(label: str) -> str:
2     # 将 Female/Female-like 标签归一化为 F, Male/Male-like 标签归一化为 M
3     value = label.strip().upper()
4     if value.startswith("F"):
5         return "F"
6     if value.startswith("M"):
7         return "M"
8     raise ValueError(f"Unknown label: {label}")
9
10
11 def read_test_file(path: Path) -> Tuple[np.ndarray, np.ndarray]:
12     # 测试文件每行包含: 身高 体重 性别
13     rows = []
14     labels = []
15     with path.open("r", encoding="utf-8") as f:
16         for line in f:
17             if not line.strip():
18                 continue
19             height, weight, label = line.split()[:3]
20             rows.append((float(height), float(weight)))
21             labels.append(normalize_label(label))
22     return np.array(rows, dtype=float), np.array(labels)
```

Listing 2: Exp.1 Gaussian parameter estimation

```
1 def fit_gaussian(x, y, features, covariance_mode):
2     # 对每一类分别估计均值向量和协方差矩阵
3     model = {}
4     for label in LABELS:
5         class_x = x[y == label][:, features]
6         if class_x.ndim == 1:
7             class_x = class_x[:, None]
8         mean = class_x.mean(axis=0)
9         centered = class_x - mean
```

```

10     cov = centered.T @ centered / len(class_x) # MLE: 分母为 N
11     cov = np.atleast_2d(cov)
12     if covariance_mode == "diag":
13         cov = np.diag(np.diag(cov)) # 不相关假设: 只保留方差
14     cov = cov + np.eye(cov.shape[0]) * 1e-6
15     model[label] = {"mean": mean, "cov": cov}
16     return model

```

Listing 3: Exp.1 minimum-error Bayes decision

```

1 def predict_gaussian(model, x, features, priors):
2     # 比较  $\log p(x|w_i) + \log P(w_i)$ , 得分更大的类别即预测类别
3     x_selected = x[:, features]
4     if x_selected.ndim == 1:
5         x_selected = x_selected[:, None]
6     scores = []
7     for label, prior in zip(LABELS, priors):
8         params = model[label]
9         log_px = log_gaussian_density(x_selected, params["mean"], params["cov"]
10 ])
11         scores.append(log_px + math.log(prior))
12     score_matrix = np.vstack(scores).T
13     return np.array([LABELS[idx] for idx in np.argmax(score_matrix, axis=1)])

```

Listing 4: Exp.1 minimum-risk Bayes decision

```

1 def predict_min_risk(model, x, features, priors,
2                       loss_predict_f_when_m=1.0,
3                       loss_predict_m_when_f=2.0):
4     # 计算两个类别的对数后验得分
5     x_selected = x[:, features]
6     log_posts = []
7     for label, prior in zip(LABELS, priors):
8         params = model[label]
9         log_posts.append(
10             log_gaussian_density(x_selected, params["mean"], params["cov"])
11             + math.log(prior)
12         )
13     log_posts = np.vstack(log_posts).T
14
15     # 对数得分归一化为后验概率
16     max_log = np.max(log_posts, axis=1, keepdims=True)

```

```

17 posts = np.exp(log_posts - max_log)
18 posts = posts / posts.sum(axis=1, keepdims=True)
19
20 # 根据损失表计算条件风险，选择风险更小的判决
21 posterior_f = posts[:, 0]
22 posterior_m = posts[:, 1]
23 risk_predict_f = loss_predict_f_when_m * posterior_m
24 risk_predict_m = loss_predict_m_when_f * posterior_f
25 return np.where(risk_predict_f <= risk_predict_m, "F", "M")

```

Listing 5: Exp.1 Parzen and kNN classifiers

```

1 def predict_parzen(train_x, train_y, x, features, priors, bandwidth):
2     # 对每个类别估计 Parzen 密度，再代入 Bayes 判别函数
3     x_selected = x[:, features]
4     class_scores = []
5     for label, prior in zip(LABELS, priors):
6         samples = train_x[train_y == label][:, features]
7         density = parzen_density(x_selected, samples, bandwidth)
8         class_scores.append(np.log(density + 1e-300) + math.log(prior))
9     scores = np.vstack(class_scores).T
10    return np.array([LABELS[idx] for idx in np.argmax(scores, axis=1)])
11
12
13 def predict_knn(train_x, train_y, x, features, k):
14     # 找到距离测试样本最近的 k 个训练样本，用多数类别作为预测
15     train_selected = train_x[:, features]
16     query = x[:, features]
17     preds = []
18     for row in query:
19         distances = np.linalg.norm(train_selected - row, axis=1)
20         nearest = np.argsort(distances)[:k]
21         labels, counts = np.unique(train_y[nearest], return_counts=True)
22         preds.append(sorted(labels[counts == counts.max()])[0])
23     return np.array(preds)

```

Listing 6: Exp.1 extension unified Bayes evaluation

```

1 def evaluate_dataset(name, train_x, train_y, test_x, test_y,
2                     use_pca, param_cov, nonparam):
3     # 图像数据先 PCA 降维，低维数据直接标准化后分类
4     train_x, test_x, prep_note = preprocess(train_x, test_x, use_pca=use_pca)

```



```

5 labels = np.unique(train_y)
6
7 # 构造损失矩阵：正确分类损失为 0，一般错分损失为 1，
8 # 第一类样本被错分时损失为 2
9 loss = loss_matrix(len(labels))
10
11 # 参数估计：高斯 Bayes，低维用完整协方差，高维图像用对角协方差
12 param = GaussianBayes(param_cov).fit(train_x, train_y)
13 pred_param = param.predict(test_x)
14 pred_param_risk = param.predict_risk(test_x, loss)
15
16 # 非参数估计：低维数据用 Parzen 窗，图像数据用 kNN 估计后验概率
17 if nonparam == "parzen":
18     non = ParzenBayes(bandwidth=0.8).fit(train_x, train_y)
19 else:
20     non = KNNBayes(k=5).fit(train_x, train_y)
21 pred_non = non.predict(test_x)
22 pred_non_risk = non.predict_risk(test_x, loss)
23
24 # 记录最小错误率和最小风险两类结果
25 return [
26     metric_row(name, "参数最小错误率", prep_note, test_y, pred_param),
27     metric_row(name, "参数最小风险", prep_note, test_y, pred_param_risk),
28     metric_row(name, "非参数最小错误率", prep_note, test_y, pred_non),
29     metric_row(name, "非参数最小风险", prep_note, test_y, pred_non_risk),
30 ]

```

A.2 实验二核心代码

Listing 7: Exp.2 Parzen density estimation

```

1 def parzen_density(query, samples, bandwidth):
2     # query 是待分类样本，samples 是某一类训练样本
3     # 使用高斯核函数估计 query 在该类别下的概率密度
4     diff = query[:, None, :] - samples[None, :, :]
5     exponent = -0.5 * np.sum((diff / bandwidth) ** 2, axis=2)
6     coef = 1.0 / ((2 * math.pi) ** (samples.shape[1] / 2)
7                 * bandwidth ** samples.shape[1])
8     return coef * np.exp(exponent).mean(axis=1)
9

```

```

10
11 def predict_parzen(train_x, train_y, query, bandwidth, priors=PRIOR_EQUAL):
12     # 分别估计女生和男生的 Parzen 密度, 再比较 Bayes 对数得分
13     scores = []
14     for label, prior in zip(LABELS, priors):
15         samples = train_x[train_y == label]
16         density = parzen_density(query, samples, bandwidth)
17         scores.append(np.log(density + 1e-300) + math.log(prior))
18     score_matrix = np.vstack(scores).T
19     return np.array([LABELS[idx] for idx in np.argmax(score_matrix, axis=1)])

```

Listing 8: Exp.2 Fisher linear discriminant

```

1 def fit_fisher(x, y):
2     # 取出女生和男生样本
3     f = x[y == "F"]
4     m = x[y == "M"]
5     mean_f = f.mean(axis=0)
6     mean_m = m.mean(axis=0)
7
8     # 类内散度矩阵 Sw, 反映同类样本内部的离散程度
9     sw = (f - mean_f).T @ (f - mean_f) + (m - mean_m).T @ (m - mean_m)
10
11     # Fisher 最优投影方向: Sw 的伪逆乘以类均值差
12     w = np.linalg.pinv(sw) @ (mean_m - mean_f)
13
14     # 把训练样本投影到一维, 并取两类投影均值中点作为阈值
15     proj_f = f @ w
16     proj_m = m @ w
17     threshold = 0.5 * (proj_f.mean() + proj_m.mean())
18     direction = 1.0 if proj_m.mean() > proj_f.mean() else -1.0
19     return {"w": w, "threshold": np.array([threshold]), "direction": np.array([direction])}
20
21
22 def predict_fisher(model, x):
23     # 计算投影值, 超过阈值则判为男生, 否则判为女生
24     scores = x @ model["w"]
25     threshold = float(model["threshold"][0])
26     direction = float(model["direction"][0])
27     is_m = scores >= threshold if direction > 0 else scores < threshold

```

```
28 return np.where(is_m, "M", "F")
```

Listing 9: Exp.2 leave-one-out validation

```
1 def loo_error_fisher(x, y):
2     # 对训练集中的每一个样本做一次“留一”验证
3     errors = 0
4     for idx in range(len(x)):
5         mask = np.ones(len(x), dtype=bool)
6         mask[idx] = False
7
8         # 用剩余 N-1 个样本重新训练 Fisher 分类器
9         model = fit_fisher(x[mask], y[mask])
10
11        # 用训练好的模型预测被留下的这个样本
12        pred = predict_fisher(model, x[idx : idx + 1])[0]
13        errors += int(pred != y[idx])
14    return errors / len(x)
```

Listing 10: Exp.2 extended datasets: LDA and non-parametric comparison

```
1 def fit_lda(x, y):
2     # 多分类 Fisher 的实现形式为 LDA
3     # shrinkage 用于提高高维 PCA 特征上的数值稳定性
4     model = LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto")
5     model.fit(x, y)
6     return model
7
8
9 def evaluate_dataset(train_x, train_y, test_x, test_y, nonparam):
10    # 图像数据先标准化并用 PCA 降到 64 维，低维数据只标准化
11    train_z, test_z = preprocess(train_x, test_x)
12
13    # 参数 Bayes 基线
14    gaussian = GaussianNB().fit(train_z, train_y)
15
16    # 非参数方法：低维数据用 Parzen 窗，图像数据用 kNN
17    if nonparam == "parzen":
18        non_model = ParzenBayes(bandwidth=0.8).fit(train_z, train_y)
19    else:
20        non_model = KNeighborsClassifier(n_neighbors=5).fit(train_z, train_y)
21
```

```

22     # Fisher/LDA 线性分类器
23     lda = fit_lda(train_z, train_y)
24
25     pred_gaussian = gaussian.predict(test_z)
26     pred_nonparam = non_model.predict(test_z)
27     pred_lda = lda.predict(test_z)
28     return pred_gaussian, pred_nonparam, pred_lda
29
30
31 def loo_error(x, y, fit_predict_one):
32     # 在训练集或分层训练子集上进行留一法估计
33     errors = 0
34     for idx in range(len(x)):
35         mask = np.ones(len(x), dtype=bool)
36         mask[idx] = False
37         pred = fit_predict_one(x[mask], y[mask], x[idx : idx + 1])
38         errors += int(pred != y[idx])
39     return errors / len(x)

```

A.3 实验三核心代码

Listing 11: Exp.3 K-L/PCA computation

```

1 def fit_pca_kl(x):
2     # 计算总体均值，并对样本中心化
3     mean = x.mean(axis=0)
4     centered = x - mean
5
6     # 最大似然形式的总体协方差矩阵，分母取样本数 N
7     cov = centered.T @ centered / len(x)
8
9     # 对协方差矩阵做特征值分解
10    eigvals, eigvecs = np.linalg.eigh(cov)
11
12    # 按特征值从大到小排列，第一列就是第一主成分方向
13    order = np.argsort(eigvals)[::-1]
14    eigvals = eigvals[order]
15    eigvecs = eigvecs[:, order]
16    return {"mean": mean, "cov": cov, "eigvals": eigvals, "eigvecs": eigvecs}

```

Listing 12: Exp.3 projection classifier

```
1 def fit_projection_classifier(x, y, direction):
2     # 将二维样本投影到给定方向上, 得到一维特征
3     proj = x @ direction
4
5     # 分别计算女生和男生在该方向上的投影均值
6     mean_f = proj[y == "F"].mean()
7     mean_m = proj[y == "M"].mean()
8
9     # 阈值取两个类别投影均值的中点
10    threshold = 0.5 * (mean_f + mean_m)
11    sign = 1.0 if mean_m > mean_f else -1.0
12    return {"direction": direction, "threshold": np.array([threshold]), "sign":
13           : np.array([sign])}
14
15 def predict_projection(model, x):
16     # 根据投影值与阈值的关系进行二分类
17     scores = x @ model["direction"]
18     threshold = float(model["threshold"][0])
19     sign = float(model.get("sign", np.array([1.0]))[0])
20     is_m = scores >= threshold if sign > 0 else scores < threshold
21     return np.where(is_m, "M", "F")
```

Listing 13: Exp.3 mean-difference direction

```
1 def fit_mean_direction(x, y):
2     # 分别计算男女类别均值
3     mean_f = x[y == "F"].mean(axis=0)
4     mean_m = x[y == "M"].mean(axis=0)
5
6     # 类均值差方向, 并归一化为单位向量
7     direction = mean_m - mean_f
8     direction = direction / np.linalg.norm(direction)
9
10    # 阈值取两个类均值中点在该方向上的投影
11    midpoint = 0.5 * (mean_f + mean_m)
12    threshold = float(midpoint @ direction)
13    return {"direction": direction, "threshold": np.array([threshold])}
```

Listing 14: Exp.3 extended datasets: PCA feature extraction

```

1 def evaluate_dataset(train_x, train_y, test_x, test_y, pca_k):
2     # 先标准化特征，避免量纲或像素取值范围影响协方差矩阵
3     train_z, test_z = standardize(train_x, test_x)
4
5     # 只取第一主成分，观察最大方差方向本身的分类能力
6     pred_pc1, pca1 = pca_nearest_centroid(train_z, train_y, test_z, 1)
7
8     # 取前 pca_k 个主成分，观察多维 K-L/PCA 特征的分类能力
9     pred_pca_k, pca_k_model = pca_nearest_centroid(
10         train_z, train_y, test_z, pca_k
11     )
12
13     # 使用类别标签训练 LDA，作为监督判别投影的对比
14     pred_lda = lda_predict(train_z, train_y, test_z)
15
16     return {
17         "pc1_variance": pca_k_model.explained_variance_ratio_[0],
18         "pca_k_variance": pca_k_model.explained_variance_ratio_.sum(),
19         "pc1_accuracy": accuracy_score(test_y, pred_pc1),
20         "pca_k_accuracy": accuracy_score(test_y, pred_pca_k),
21         "lda_accuracy": accuracy_score(test_y, pred_lda),
22     }

```

A.4 实验四核心代码

Listing 15: Exp.4 face dataset split for Eigenfaces

```

1 def load_faces():
2     # Olivetti Faces 与 ORL/AT&T 数据集形式接近：
3     # 40 人，每人 10 张，64x64 灰度图
4     data = fetch_olivetti_faces(data_home=DATA_HOME, shuffle=False)
5     return (
6         data.data.astype(np.float32),
7         data.images.astype(np.float32),
8         data.target.astype(int),
9     )
10
11
12 def split_by_person(x, images, y):
13     # 每个人前 6 张作为训练样本，后 4 张作为测试样本

```

```

14 train_indices = []
15 test_indices = []
16 for person in sorted(np.unique(y)):
17     indices = np.flatnonzero(y == person)
18     train_indices.extend(indices[:TRAIN_PER_PERSON])
19     test_indices.extend(indices[TRAIN_PER_PERSON:])
20
21 train_indices = np.array(train_indices)
22 test_indices = np.array(test_indices)
23 return {
24     "x_train": x[train_indices],
25     "y_train": y[train_indices],
26     "img_train": images[train_indices],
27     "x_test": x[test_indices],
28     "y_test": y[test_indices],
29     "img_test": images[test_indices],
30 }

```

Listing 16: Exp.4 Eigenfaces feature extraction and classifiers

```

1 def fit_eval_models(x_train, y_train, x_test, y_test, n_components):
2     # 在标准化后的图像向量上进行 K-L/PCA 特征脸提取
3     models = {
4         "Eigenfaces+NearestCentroid": make_pipeline(
5             StandardScaler(),
6             PCA(n_components=n_components, whiten=True, random_state=2026),
7             NearestCentroid(),
8         ),
9         "Eigenfaces+1NN": make_pipeline(
10             StandardScaler(),
11             PCA(n_components=n_components, whiten=True, random_state=2026),
12             KNeighborsClassifier(n_neighbors=1),
13         ),
14         "Eigenfaces+SVM": make_pipeline(
15             StandardScaler(),
16             PCA(n_components=n_components, whiten=True, random_state=2026),
17             SVC(kernel="linear", C=1.0, random_state=2026),
18         ),
19     }
20
21 rows = []

```

```

22 for name, model in models.items():
23     model.fit(x_train, y_train)
24     pred_train = model.predict(x_train)
25     pred_test = model.predict(x_test)
26     rows.append({
27         "method": name,
28         "n_components": n_components,
29         "train_accuracy": accuracy_score(y_train, pred_train),
30         "test_accuracy": accuracy_score(y_test, pred_test),
31     })
32 return rows

```

Listing 17: Exp.4 supplementary aircraft image loading

```

1 def read_image(path):
2     # 使用 OpenCV 读取飞机图像，并转换为灰度图
3     data = np.fromfile(str(path), dtype=np.uint8)
4     image = cv2.imdecode(data, cv2.IMREAD_GRAYSCALE)
5     if image is None:
6         raise ValueError(f"Could not read image: {path}")
7
8     # 将所有图像缩放到统一大小，便于展开为固定维数向量
9     image = cv2.resize(image, IMAGE_SIZE, interpolation=cv2.INTER_AREA)
10
11    # 像素归一化到 [0,1]，再在 load_split 中展平为 1024 维向量
12    return image.astype(np.float32) / 255.0
13
14
15 def load_split(split):
16     rows, labels, paths = [], [], []
17     for path, label in image_paths(split):
18         rows.append(read_image(path).reshape(-1))
19         labels.append(label)
20         paths.append(path)
21    return np.vstack(rows), np.array(labels), paths

```

Listing 18: Exp.4 supplementary aircraft PCA classifiers

```

1 def fit_and_eval(x_train, y_train, x_test, y_test, n_components):
2     # K-L/PCA 主成分数不能超过样本数和原始特征维数
3     n_components = min(n_components, x_train.shape[0], x_train.shape[1])
4

```



```

5 # 两种分类器都在标准化后的 PCA 特征空间中训练和预测
6 models = {
7     "PCA+NearestCentroid": make_pipeline(
8         StandardScaler(),
9         PCA(n_components=n_components, random_state=2026),
10        NearestCentroid(),
11    ),
12    "PCA+1NN": make_pipeline(
13        StandardScaler(),
14        PCA(n_components=n_components, random_state=2026),
15        KNeighborsClassifier(n_neighbors=1),
16    ),
17 }
18
19 rows = []
20 for name, model in models.items():
21     model.fit(x_train, y_train)
22     pred_train = model.predict(x_train)
23     pred_test = model.predict(x_test)
24     rows.append(
25         {
26             "method": name,
27             "n_components": n_components,
28             "train_accuracy": accuracy_score(y_train, pred_train),
29             "test_accuracy": accuracy_score(y_test, pred_test),
30         }
31     )
32 return rows

```

A.5 实验五核心代码

Listing 19: Exp.5 C-means implementation

```

1 def c_means(x, n_clusters, seed, max_iter=200, tol=1e-6):
2     # 随机选择 C 个样本作为初始聚类中心
3     rng = np.random.default_rng(seed)
4     indices = rng.choice(len(x), size=n_clusters, replace=False)
5     centers = x[indices].copy()
6     labels = np.zeros(len(x), dtype=int)
7

```

```

8   for iteration in range(1, max_iter + 1):
9       # 计算每个样本到每个中心的欧氏距离
10      distances = np.linalg.norm(x[:, None, :] - centers[None, :, :], axis
=2)
11
12      # 将样本分配给距离最近的中心
13      new_labels = distances.argmin(axis=1)
14      new_centers = centers.copy()
15
16      # 根据当前簇内样本均值更新中心
17      for cluster in range(n_clusters):
18          members = x[new_labels == cluster]
19          if len(members) == 0:
20              new_centers[cluster] = x[rng.integers(0, len(x))]
21          else:
22              new_centers[cluster] = members.mean(axis=0)
23
24      # 若中心移动量很小, 则认为算法收敛
25      shift = np.linalg.norm(new_centers - centers)
26      centers = new_centers
27      labels = new_labels
28      if shift < tol:
29          break
30
31      # SSE: 样本到所属中心的平方距离总和
32      inertia = float(np.sum((x - centers[labels]) ** 2))
33      return labels, centers, inertia, iteration

```

Listing 20: Exp.5 cluster-number metrics

```

1  def evaluate_k_range(x_scaled, y, dataset_name):
2      # 分别测试 C=2,3,4,5, 记录 SSE、轮廓系数和 ARI
3      rows = []
4      for n_clusters in range(2, 6):
5          labels, centers, inertia, iterations = c_means(
6              x_scaled, n_clusters, seed=2026 + n_clusters
7          )
8          rows.append({
9              "dataset": dataset_name,
10             "method": "C-means",
11             "n_clusters": n_clusters,

```

```

12         "inertia": inertia,
13         "silhouette": silhouette_score(x_scaled, labels),
14         "ari": adjusted_rand_score(y, labels),
15         "iterations": iterations,
16     })
17     return rows

```

Listing 21: Exp.5 Ward hierarchical clustering

```

1 def evaluate_hierarchical(x_scaled, y, dataset_name):
2     # Ward 分级聚类: 每次合并使类内平方误差增加最小的簇
3     clustering = AgglomerativeClustering(n_clusters=2, linkage="ward")
4     labels = clustering.fit_predict(x_scaled)
5     return {
6         "dataset": dataset_name,
7         "method": "Hierarchical Ward",
8         "n_clusters": 2,
9         "silhouette": silhouette_score(x_scaled, labels),
10        "ari": adjusted_rand_score(y, labels),
11        "gender_accuracy_if_2": best_label_accuracy(y, labels),
12    }

```

A.6 补充实验一核心代码

Listing 22: Supplement 1 open-set decision rule

```

1 def decide_status(score, distance, alpha=0.37, beta=0.10, delta=16.5):
2     # score: 检测框最大类别置信度
3     # distance: 投影特征与预测类别原型之间的欧氏距离
4     # alpha: 确认已知类的高置信阈值
5     # beta: 区分目标候选和背景的低置信阈值
6     # delta: 原型空间中的已知类距离阈值
7     if score >= alpha and distance <= delta:
8         return "known"
9     if score >= beta and (score < alpha or distance > delta):
10        return "unknown"
11    return "background"

```

Listing 23: Supplement 1 prototype regularization

```

1 def prototype_regularization(z, y, prototypes, margin=1.0,
2                             lambda_attr=1.0, lambda_rep=0.2):

```

```

3 # z: 目标候选框的投影特征, 形状为 [batch, proto_dim]
4 # y: 已知类标签
5 # prototypes: 所有已知类的可学习原型
6
7 # 吸引项: 同类样本靠近本类原型
8 target_proto = prototypes[y]
9 l_attr = ((z - target_proto) ** 2).sum(dim=1).mean()
10
11 # 排斥项: 样本与非目标类原型保持一定间隔
12 all_dist = torch.cdist(z, prototypes)
13 mask = torch.ones_like(all_dist, dtype=torch.bool)
14 mask[torch.arange(len(y)), y] = False
15 negative_dist = all_dist[mask].view(len(y), -1)
16 l_rep = torch.relu(margin - negative_dist).pow(2).mean()
17
18 return lambda_attr * l_attr + lambda_rep * l_rep

```

A.7 补充实验二核心代码

Listing 24: Supplement 2 classifier-guided reward

```

1 def classifier_guided_reward(trajecory, target_digit, classifier):
2     # 将无人机连续轨迹渲染成 28x28 灰度图
3     image = render_trajectory_to_28x28(trajecory)
4
5     # CNN 输出 11 类概率: 0-9 为数字类, 10 为 OOD/乱码类
6     probs = classifier.predict_proba(image)
7     p_target = probs[target_digit]
8     p_ood = probs[10]
9
10    # 计算目标类与最强非目标数字类之间的置信间隔
11    digit_probs = probs[:10].copy()
12    digit_probs[target_digit] = -1.0
13    p_confuse = digit_probs.max()
14    margin = p_target - p_confuse
15
16    # 奖励鼓励目标数字概率和间隔, 惩罚 OOD/乱码概率
17    reward = p_target + 0.5 * margin - 0.8 * p_ood
18    return reward, {
19        "prob_target": p_target,

```

```

20     "prob_confuse": p_confuse,
21     "prob_ood": p_ood,
22     "margin": margin,
23     "image": image,
24 }

```

Listing 25: Supplement 2 zero-shot skill evaluation

```

1  def evaluate_zero_shot_skill(agent, env, target_digit, classifier,
2      skill_candidates):
3
4      # 在预训练技能空间中搜索，不更新策略参数
5      for skill in skill_candidates:
6          obs, info = env.reset(skill=skill)
7          done = False
8          trajectory = []
9
10         while not done:
11             action = agent.act(obs, skill=skill, eval_mode=True)
12             obs, reward_env, terminated, truncated, info = env.step(action)
13             trajectory.append(info["position"])
14             done = terminated or truncated
15
16         reward, metrics = classifier_guided_reward(
17             trajectory, target_digit, classifier
18         )
19         if best_result is None or reward > best_result["reward"]:
20             best_result = {"skill": skill, "reward": reward, **metrics}
21
22     return best_result

```